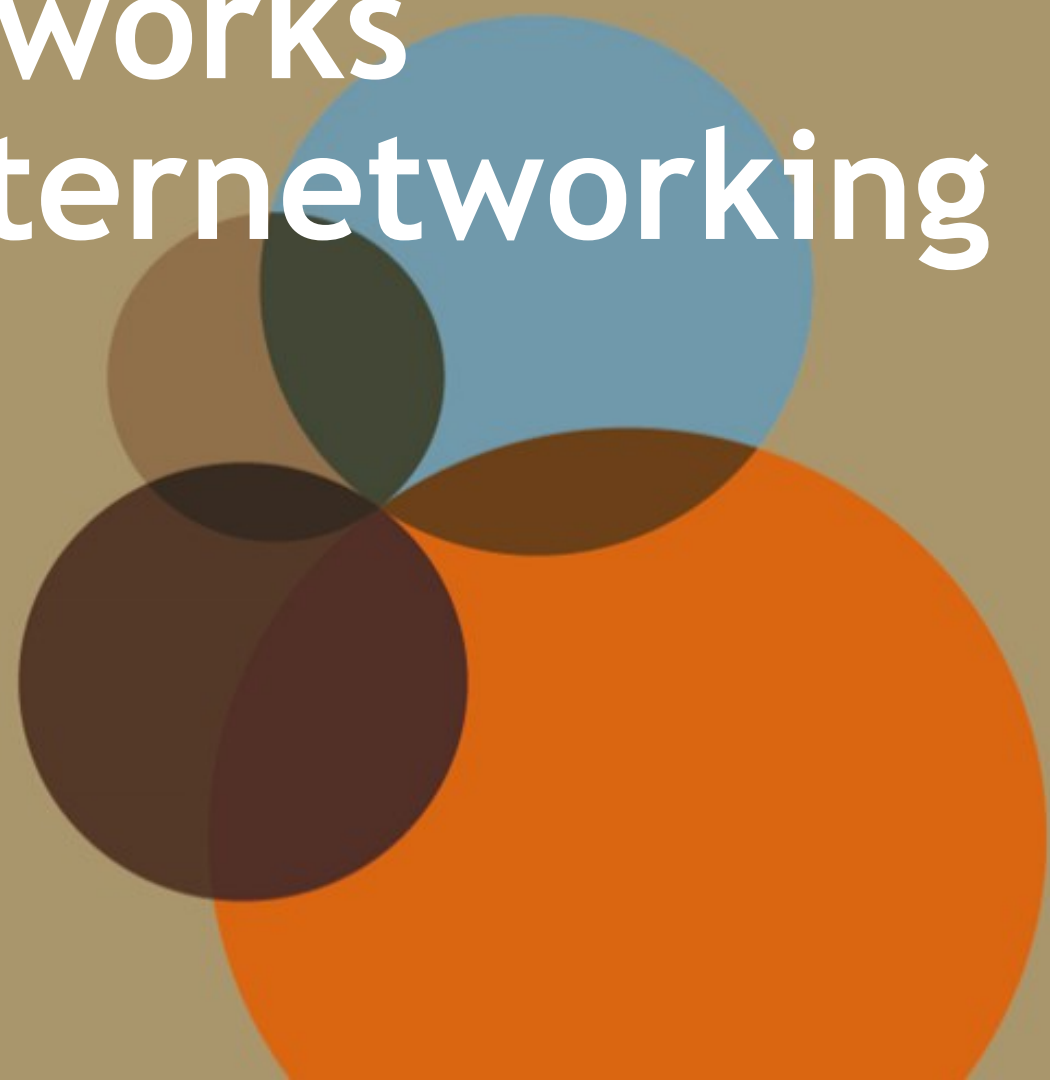


Computer Networks

Chapter 3 - Internetworking

Prof. Marino Miculan



Problems

- In Chapter 2 we saw how to connect one node to another, or to an existing network. How do we build networks of global scale?
- How do we interconnect different types of networks to build a large global network?
- Chapter Outline
 - Switching and Bridging
 - Basic Internetworking (IP)
 - Routing

Chapter Goal

- Understanding the functions of **switches, bridges and routers**
 - Connection between different networks
- Discussing **Internet Protocol (IP)** for interconnecting networks
 - Abstraction over the kind of networks
- Understanding the concept of **routing**
 - in an internet, many paths are possible...how to choose, how to follow?



Switching and Forwarding

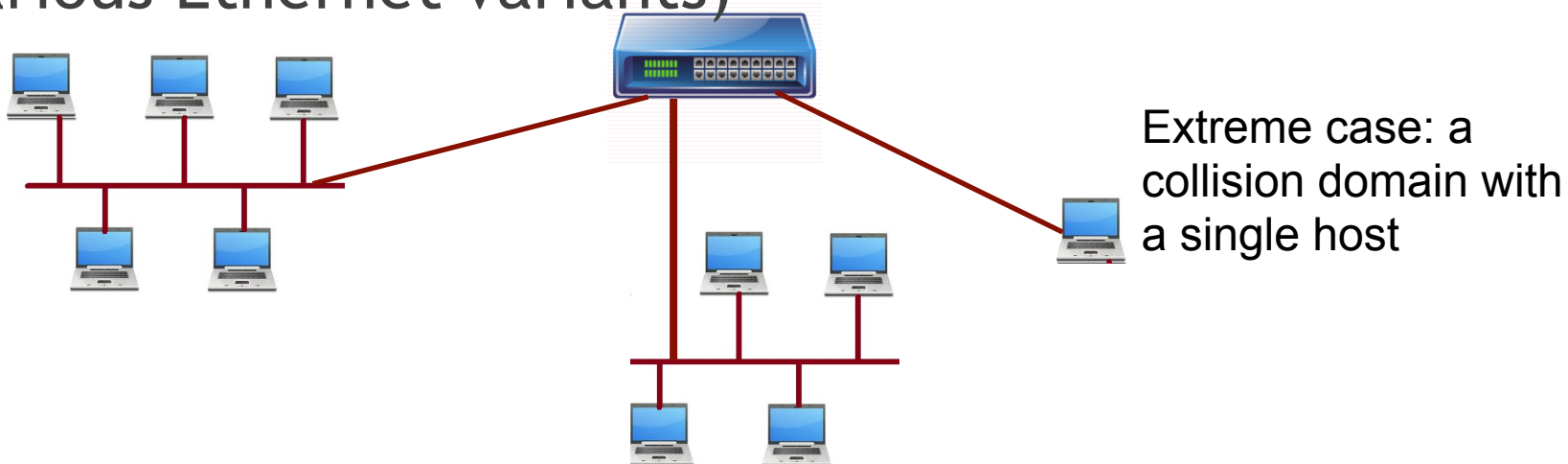
Switching and Forwarding

- Store-and-Forward Switches
- Bridges and Extended LANs
- Cell Switching
- Segmentation and Reassembly

Switching and Forwarding

• Switch

- A mechanism that allows us to interconnect links to form a large network
- A multi-input, multi-output device which transfers packets from an input to one or more outputs
- Usually, these links are homogenous: of the same type, with the same address space, but possibly with different technologies (e.g.: various Ethernet variants)

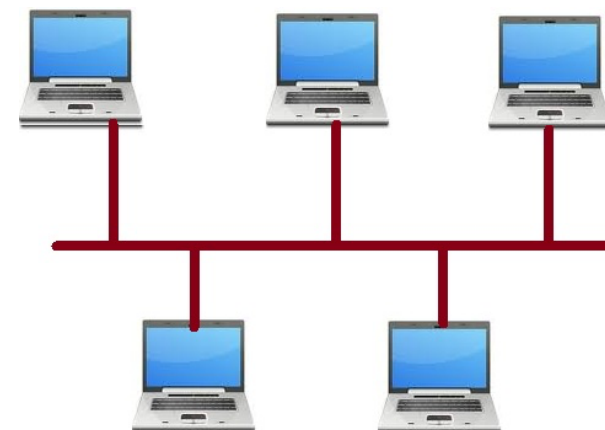
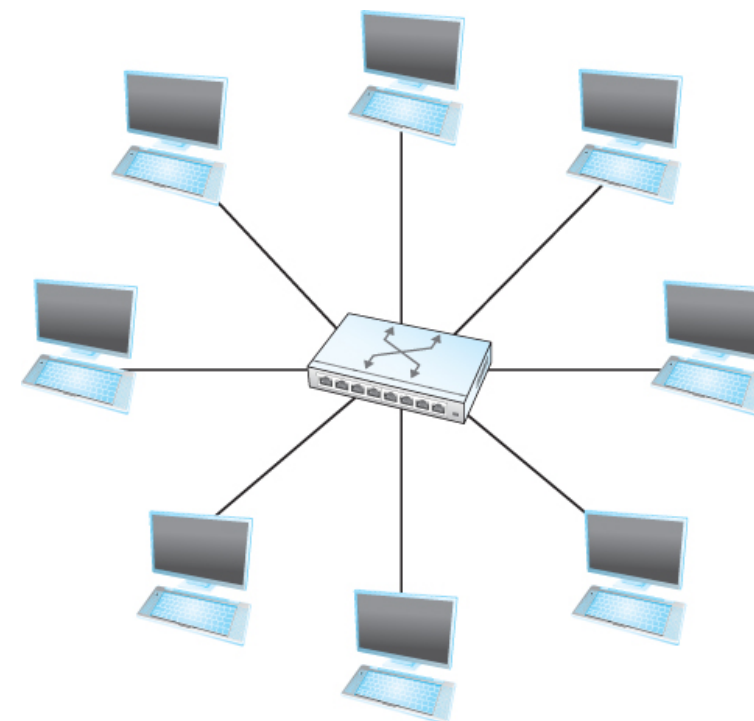


Switching and Forwarding

- A switch is connected to a set of links and for each of these links, runs the appropriate data link protocol to communicate
- A switch's primary job is **switching and forwarding**: to receive incoming packets on one of its links and to transmit them on some other link
 - According to OSI architecture this is the main function of the network layer (3), but can be implemented also at layer 2 (datalink)
- Most common strategy is **store-and-forward**:
 1. Receive a whole frame on some link
 2. Check it (CRC, etc.); if invalid, drop it (without notice)
 3. Using some internal table, choose which link forward the frame to
 4. Resend the frame to the chosen link
- (More modern switches use a faster strategy, called *cut through*)

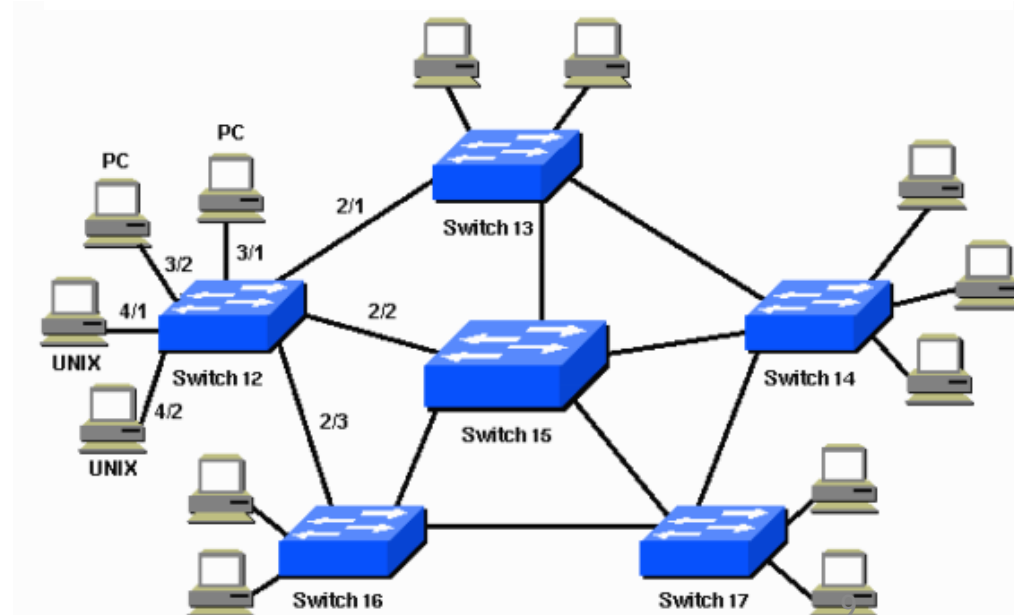
The star topology

- The use of switches adds the **star topology** to the point-to-point link (PPP, modems), bus (Ethernet), and ring (802.5 and FDDI) topologies
- Each spoke of the hub can be seen as a separate collision domain, so no collisions between host and star center
 - In full duplex links, data can be transferred between host and switch in both directions at maximum speed
 - However, the switch still has a limited bandwidth and memory...
- can be used to simulate a shared medium



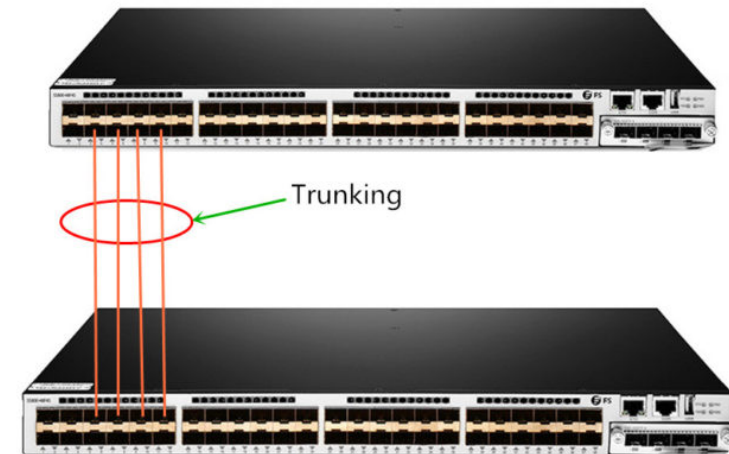
The star topology

- Properties of star topology
 - A switch has a fixed number of inputs and outputs, which limits the number of hosts that can be connected to a single switch (64+)
 - Larger networks can be built by interconnecting a number of switches
 - The whole network appears as a single LAN to each host (although delays and latencies are not uniform)



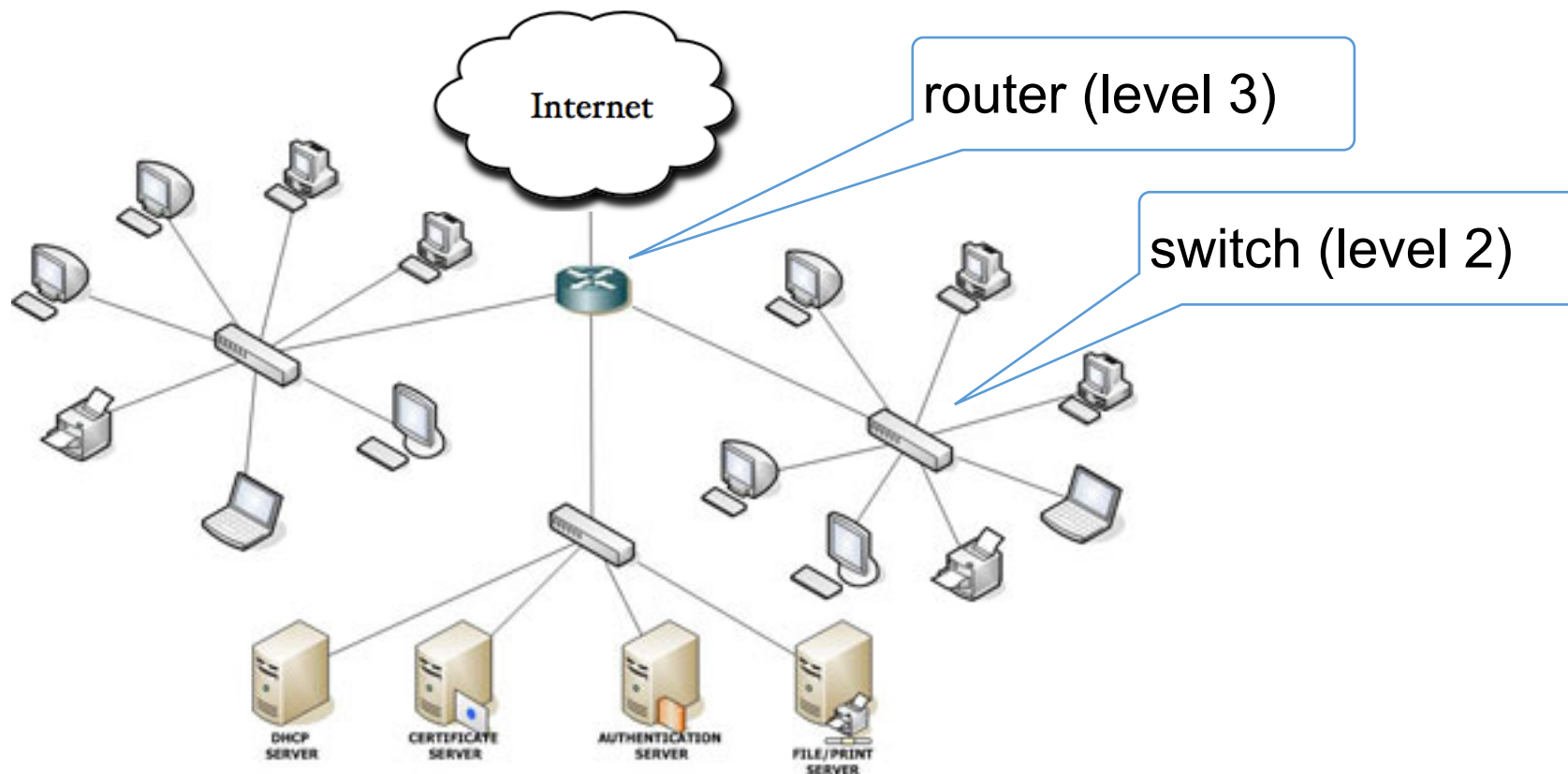
The star topology

- Properties of star topology (cont.)
 - hosts/other stars can be connected by point-to-point links
 - These can be normal links on normal ports (called *uplinks*), possibly tied together to appear as a single link (*trunking*)
 - Not to be confused with *stackable switches*, whose backplanes can be connected together via dedicated, proprietary high-speed links
 - Stacked switches act as a single one, not as separate units



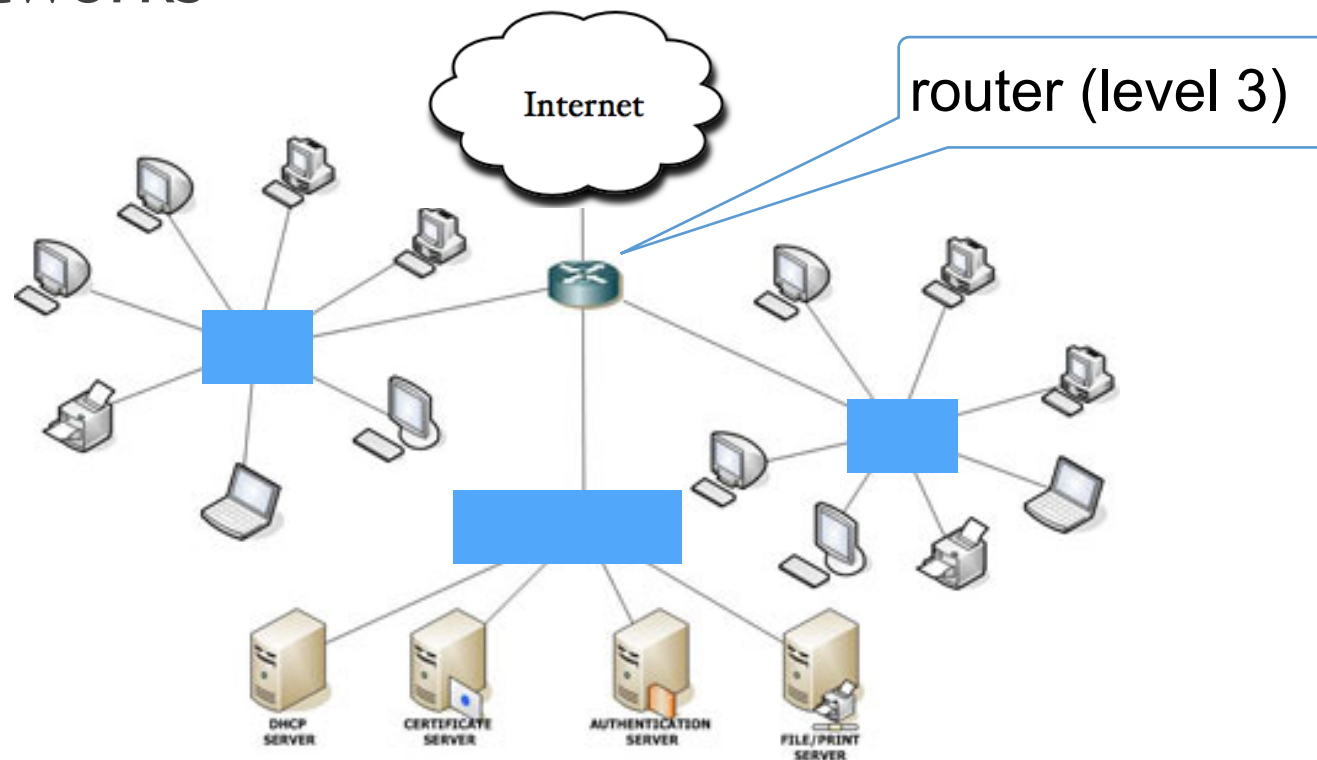
The star topology

- Properties of star topology
 - Different stars/set of stars (i.e., different LANs) can be connected to routers, which are level-3 switches



The star topology

- Properties of star topology
 - From the level-3, switched networks are just another form of local networks



The star topology

- Properties of star topology (cont.)
 - networks of large geographic scope can be built
 - MANs/WANs can be built using optic fiber or ATM switches
 - ISPs use xDSL switches (DSLAM) to connect end users over telephonic lines



ATM switch



Optic switch



DSLAM¹³

The star topology

- Properties of star topology (cont.)
 - Adding a new host to the network by connecting it to a switch does not necessarily mean that the hosts already connected will get worse performance from the network
 - Every host on a switched network has its own link to the switch, so it is possible for many hosts to transmit at the full link speed (bandwidth)
 - The limit is given by the internal *maximum bandwidth* and the *switching speed* (or *forwarding rate*)
 - In Ethernet switches, internal bandwidth ranges from 10Gbps at ~1M packet per second (pps) in cheap devices, to up to 10-20 Tbps at tens of Gpps for high-end devices
 - This is not true for shared media networks
 - Two hosts on the same Ethernet cannot transmit continuously at full speed because there are collisions on the shared transmission medium
 - Collisions can be avoided, e.g. with TDM techniques, but then each host receives a fraction of the total bandwidth of the shared media

Switching and Forwarding

- How does the switch decide which output port to place each packet on?
 - It looks at the header of the packet for an identifier that it uses to make the decision
 - Three approaches
 - *Datagram* or *Connectionless* approach
 - *Virtual circuit* or *Connection-oriented* approach
 - *Source routing* (less common)
 - All the switches of the network must adhere to the same method
- Exchanged units: Frames (level 2), Packets (level 3), Datagrams (level 4), Messages (level 7),...

Switching and Forwarding

- Assumptions
 - Each host has a globally unique address
 - level 2: MAC address
 - level 3: IP address, etc.
 - There is some way to identify the input and output ports of each switch
 - We can use numbers (one for each port)
 - We can use names (of the nodes the port is connected to)
 - For sake of simplicity, let us use numbers

Switching and Forwarding: Connectionless

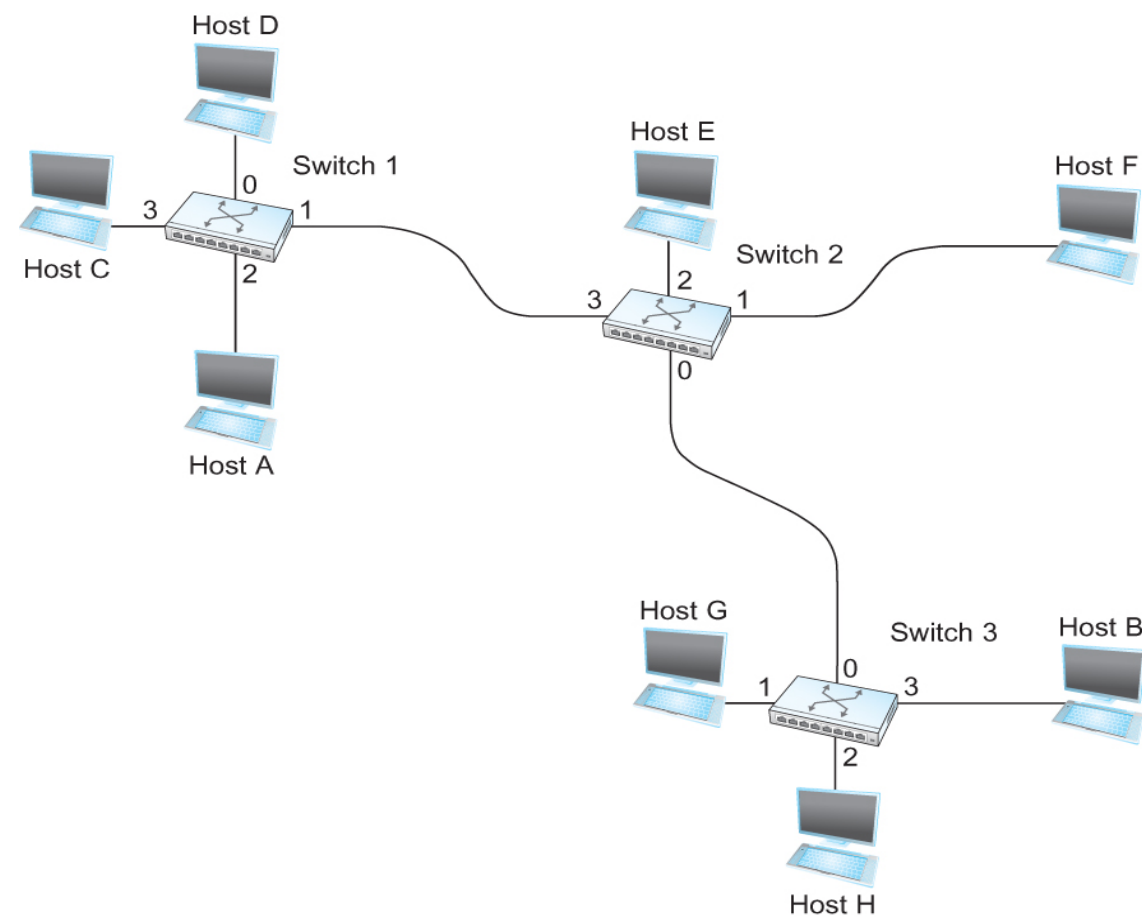
- Key Idea
 - Every packet contains enough information to enable any switch to decide how to get it to destination
 - Every packet contains the complete destination address
 - Each switch forwards each packet just by looking at its header
 - no state is kept in switches

Switching and Forwarding: Connectionless

- An example network is aside
- To decide how to forward a packet, a switch consults a **forwarding table** (sometimes called a **routing table**)
- Eg, Forwarding Table for Switch 2:

Dest.	Port

A	3
B	0
C	3
D	3
E	2
F	1
G	0
H	0



Switching and Forwarding: Connectionless

- Characteristics of Connectionless (Datagram) Network
 - A host can send a packet anywhere at any time, since any packet that turns up at the switch can be immediately forwarded (assuming a correctly populated forwarding table)
 - When a host sends a packet, it has no way of knowing if the network is capable of delivering it or if the destination host is even up and running
 - Each packet is forwarded independently of previous packets that might have been sent to the same destination.
 - Thus two successive packets from host A to host B may follow completely different paths
 - A switch or link failure might not have any serious effect on communication if it is possible to find an alternate route around the failure and update the forwarding table accordingly

Switching and Forwarding: Connectionless

- Characteristics of Connectionless (Datagram) Network (cont)
 - We expect the forwarding tables to be updated and correct
 - Who fills the tables?
 - Human: Viable only for small networks
 - Automatic: Routing algorithms running in the background, checking for link status and automatically updating tables
 - “Distributed autonomic system”

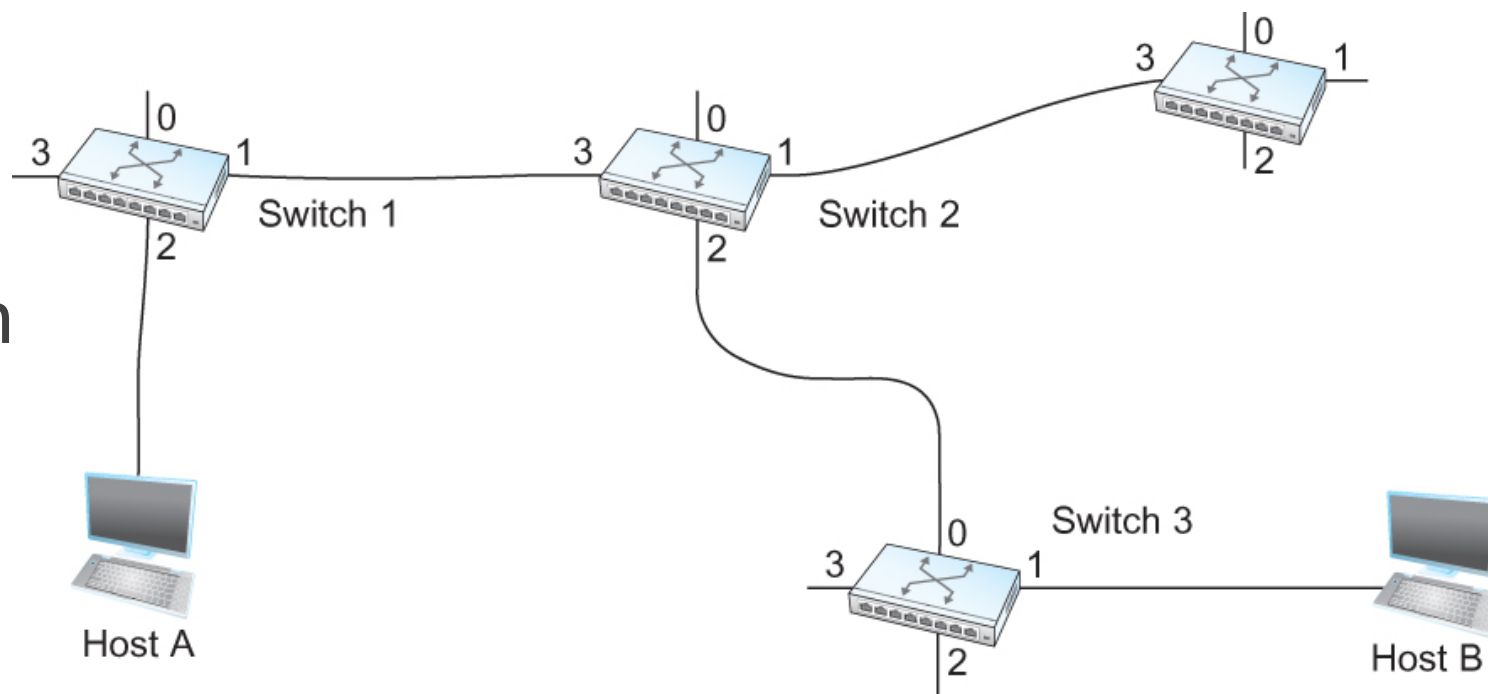
Switching and Forwarding: Virtual Circuit

- Virtual Circuit Switching
 - Widely used technique for packet switching
 - Uses the concept of virtual circuit (VC)
 - Also called a **connection-oriented model**
 - First set up a virtual connection from the source host to the destination host and then send the data

Switching and Forwarding: Virtual Circuit

- Host A wants to send packets to host B
- Three stage process

- 1.Connection setup
- 2.Data Transfer
- 3.Connection shutdown



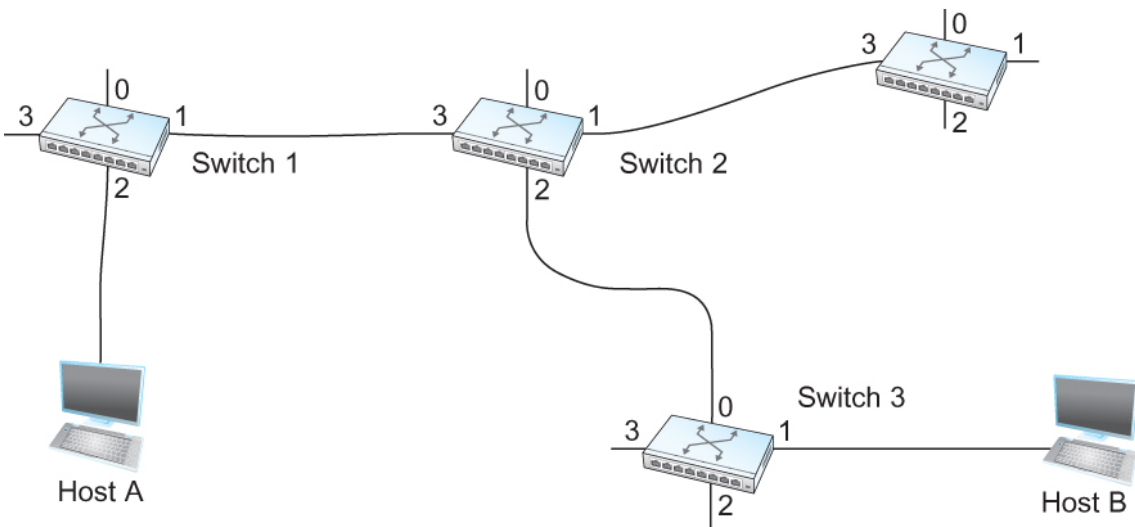
Switching and Forwarding: Virtual Circuit

- Connection setup
 - Establish “connection state” in each of the switches between the source and destination hosts
 - The connection state for a single connection consists of an entry in the “VC table” in each switch through which the connection passes
 - Sometimes configured permanently by network administrator (*Permanent VC*)
 - More often, established on demand by *signalling* (SVC):
 - A host can send messages into the network to cause the state to be established
 - A host may set up and delete such a VC dynamically without the involvement of a network administrator

Switching and Forwarding: Virtual Circuit

- One entry in the VC table on a single switch contains
 - A **virtual circuit identifier (VCI)** that uniquely identifies the connection at this switch and that will be carried inside the header of the packets that belong to this connection (cont.)
 - An incoming interface on which packets for this VC arrive at the switch
 - An outgoing interface in which packets for this VC leave the switch
 - A (potentially different) VCI that will be used for outgoing packets
- The semantics for one such entry is:
 - If a packet arrives on the designated incoming interface and that packet contains the designated VCI value in its header, then the packet should be sent out the specified outgoing interface with the specified outgoing VCI value first having been placed in its header

Switching and Forwarding: Virtual Circuit



Switch 1

Incoming Interface	Incoming VC	Outgoing Interface	Outgoing VC
2	5	1	11

Switch 2

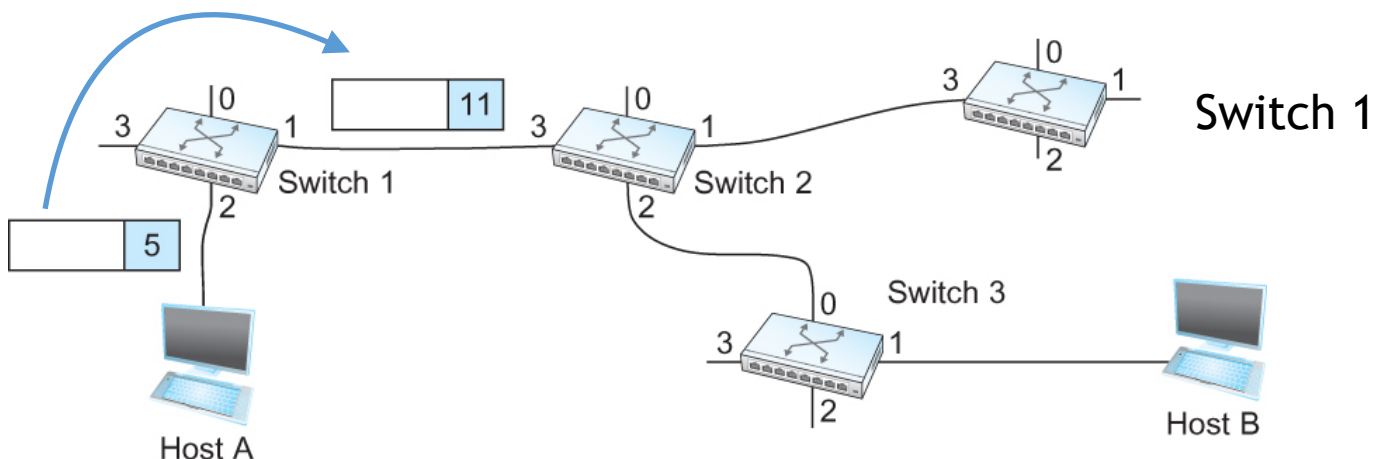
Incoming Interface	Incoming VC	Outgoing Interface	Outgoing VC
3	11	2	7

Switch 3

Incoming Interface	Incoming VC	Outgoing Interface	Outgoing VC
0	7	1	4

Switching and Forwarding: Virtual Circuit

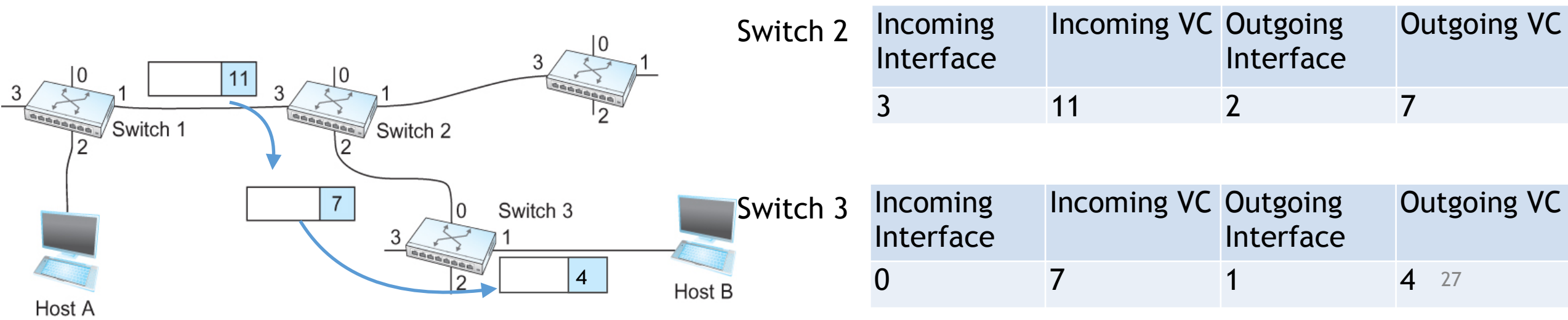
- For any packet that A wants to send to B, A puts the VCI value 5 in the header of the packet and sends it to switch 1
- Switch 1 receives any such packet on interface 2, and it uses the combination of the interface and the VCI in the packet header to find the appropriate VC table entry.
- The table entry on switch 1 tells the switch to forward the packet out of interface 1 and to put the VCI value 11 in the header



Incoming Interface	Incoming VC	Outgoing Interface	Outgoing VC
2	5	1	11

Switching and Forwarding: Virtual Circuit

- Packet will arrive at switch 2 on interface 3 bearing VCI 11
- Switch 2 looks up interface 3 and VCI 11 in its VC table and sends the packet on to switch 3 after updating the VCI value appropriately
- This process continues until it arrives at host B with the VCI value of 4 in the packet
- To host B, this identifies the packet as having come from host A



Switching and Forwarding: Virtual Circuit

- The combination of the VCI of the packets as they are received at the switch and the interface on which they are received *uniquely identifies* the virtual connection
- There may be many virtual connections established in the switch at one time
- Incoming and outgoing VCI values are not generally the same
 - VCI is not a globally significant identifier for the connection; rather it has significance only on a given link
- Whenever a new connection is created, we need to assign a new VCI for that connection on each link that the connection will traverse
 - We also need to ensure that the chosen VCI on a given link is not currently in use on that link by some existing connection.

Switching and Forwarding: Virtual Circuit

- Two broad classes of approach to establishing connection state
 - *Permanent* (PVC):
 - Network Administrator will configure the state
 - The network administrator can delete this
 - Can be thought of as a long-lived or administratively configured VC
 - Switched by *signalling* (SVC):
 - A host can send messages into the network to cause the state to be established, by means of specific *signaling protocols*
 - A host may set up and delete such a VC dynamically without the involvement of a network administrator

Switching and Forwarding: Virtual Circuit

- How does the signalling work
 - To start the signalling process, host A sends a setup message into the network (i.e. to switch 1)
 - The setup message contains (among other things) the complete destination address of B.
 - The setup message needs to get all the way to B to create the necessary connection state in every switch along the way
 - It is like sending a datagram to B where every switch knows which output to send the setup message so that it eventually reaches B
 - Assume that every switch knows the topology to figure out how to do that
 - When switch 1 receives the connection request, in addition to sending it on to switch 2, it creates a new entry in its VC table for this new connection
 - The entry is exactly the same shown in the previous table
 - Switch 1 picks the value 5 for this connection

Switching and Forwarding: Virtual Circuit

- How does the signalling work (contd.)
 - When switch 2 receives the setup message, it performs the similar process and it picks the value 11 as the incoming VCI
 - Similarly switch 3 picks 7 as the value for its incoming VCI
 - Each switch can pick any number it likes, as long as that number is not currently in use for some other connection on that port of that switch
 - Finally the setup message arrives at host B.
 - Assuming that B is healthy and willing to accept a connection from host A, it allocates an incoming VCI value, in this case 4.
 - This VCI value can be used by B to identify all packets coming from A

Switching and Forwarding: Virtual Circuit

- Now to complete the connection, everyone needs to be told what their downstream neighbour is using as the VCI for this connection
 - Host B sends an acknowledgement of the connection setup to switch 3 and includes in that message the VCI value that it chose (4)
 - Switch 3 completes the VC table entry for this connection and sends the acknowledgement on to switch 2 specifying the VCI of 7
 - Switch 2 completes the VC table entry for this connection and sends acknowledgement on to switch 1 specifying the VCI of 11
 - Finally switch 1 passes the acknowledgement on to host A telling it to use the VCI value of 5 for this connection

Switching and Forwarding: Virtual Circuit

- When host A no longer wants to send data to host B, it tears down the connection by sending a teardown message to switch 1
- The switch 1 removes the relevant entry from its table and forwards the message on to the other switches in the path which similarly delete the appropriate table entries
- At this point, if host A were to send a packet with a VCI of 5 to switch 1, it would be dropped as if the connection had never existed

Switching and Forwarding: Virtual Circuit

- Characteristics of VC
 - Since host A has to wait for the connection request to reach the far side of the network and return before it can send its first data packet, there is at least one RTT of delay before data is sent
 - While the connection request contains the full address for host B (which might be quite large, being a global identifier on the network), each data packet contains only a small identifier, which is only unique on one link.
 - Thus the per-packet overhead caused by the header is reduced relative to the datagram model
 - E.g. in ATM, the whole header is 5 bytes (out of a 53-byte frame)

Switching and Forwarding: Virtual Circuit

- Characteristics of VC (cont.)
 - If a switch or a link in a connection fails, the connection is broken and a new one will need to be established.
 - Also the old one needs to be torn down to free up table storage space in the switches
 - The issue of how a switch decides which link to forward the connection request on has similarities with the function of a routing algorithm

Switching and Forwarding: Virtual Circuit

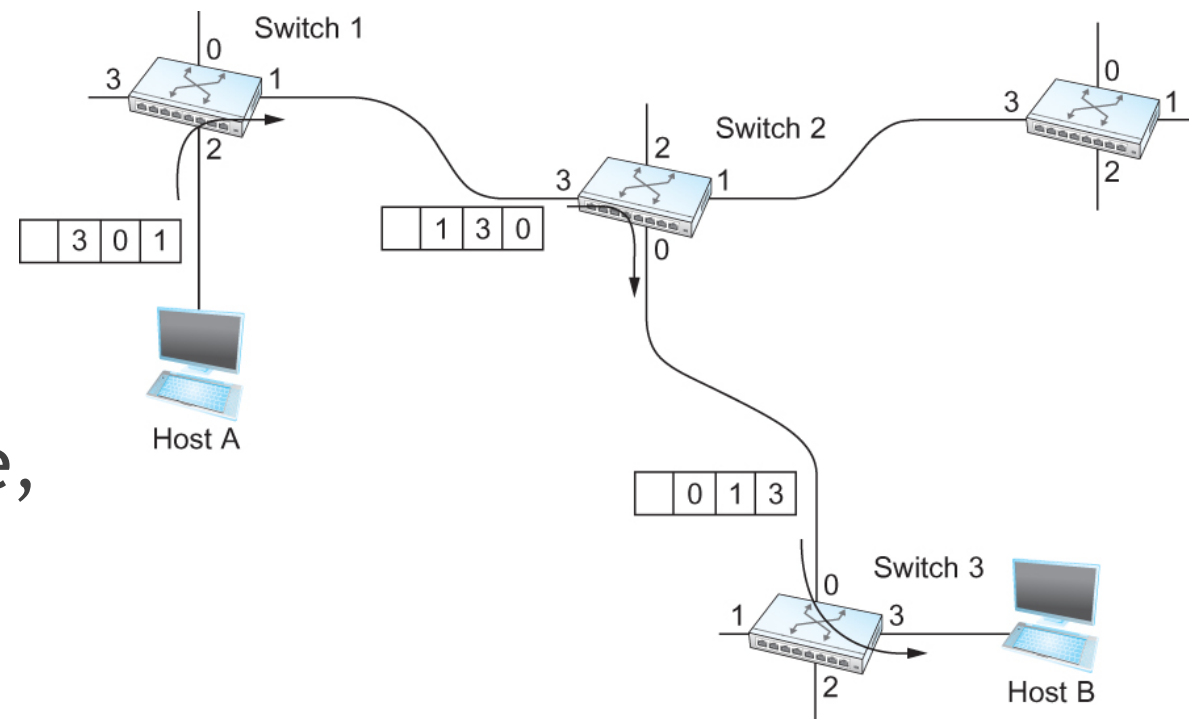
- Good Properties of VC
 - By the time the host gets the go-ahead to send data, it knows quite a lot about the network
 - For example, that there is really a route to the receiver and that the receiver is willing to receive data
 - It is also possible to allocate resources to the virtual circuit at the time it is established
 - E.g., in X.25, buffers are allocated to each virtual circuit when the circuit is initialized, and the circuit is rejected by a given node if not enough buffers are available

Switching and Forwarding: Virtual Circuit

- Comparison with the Datagram Model
 - Datagram network has no connection establishment phase and each switch processes each packet independently
 - Each arriving packet competes with all other packets for buffer space
 - If there are no buffers, the incoming packet must be dropped
- In VC, we could imagine providing each circuit with a different quality of service (QoS)
 - The network gives the user some kind of performance related guarantee
 - Switches set aside the resources they need to meet this guarantee
 - For example, a percentage of each outgoing link's bandwidth
 - Delay tolerance on each switch

Switching and Forwarding: Source Routing

- All the information about network topology that is required to switch a packet across the network is provided by the source host
- E.g.: the ports the packet is be forwarded over
- Pro: extremely simple switches; no connection is needed
- Cons: the routing is on the source, which has to know the network topology; less robust to failures



Bridges and LAN Switches

Bridges and LAN Switches

- Bridges and LAN Switches
 - Class of switches that is used to forward packets between shared-media LANs such as Ethernets
 - Known as **LAN switches**
 - Referred to as *Bridges*
 - usually by “switch” we intend LAN (i.e. level 2) switches, even if there can be switches of any level

Bridges and LAN Switches

- Suppose you have a pair of Ethernets that you want to interconnect
 - One approach is put a repeater in between them
 - A repeater just decodes/encodes bits
 - It might exceed the physical limitation of the Ethernet
 - No more than four repeaters between any pair of hosts
 - No more than a total of 2500 m in length is allowed
 - An alternative would be to put a node between the two Ethernets and have the node forward frames from one Ethernet to the other
 - This node is called a *Bridge*
 - A collection of LANs connected by one or more bridges is usually said to form an *Extended LAN*

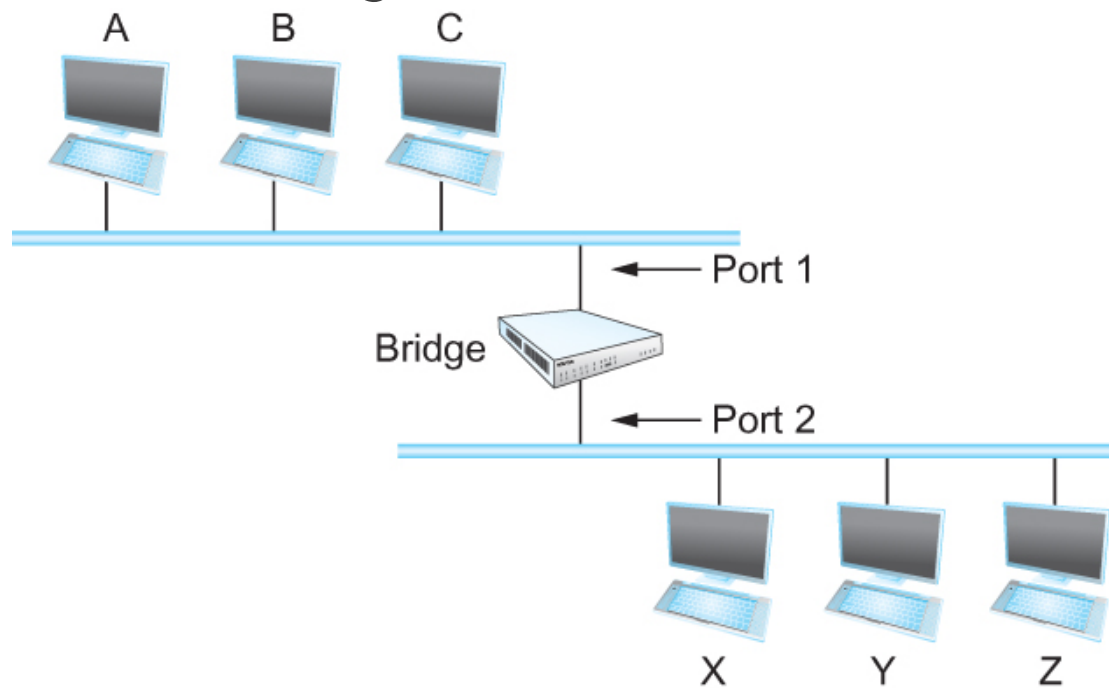
Bridges and LAN Switches

- Simplest Strategy for Bridges: accept LAN frames on their inputs and forward them out to all other outputs
 - This strategy is called **flooding**
 - Used by early bridges (*hubs*), now obsolete
- **Learning Bridges**
 - Observe that there is no need to forward to everyone all the frames that a bridge receives
 - instead, each frame can be forwarded only to the relevant host(s)



Bridges and LAN Switches

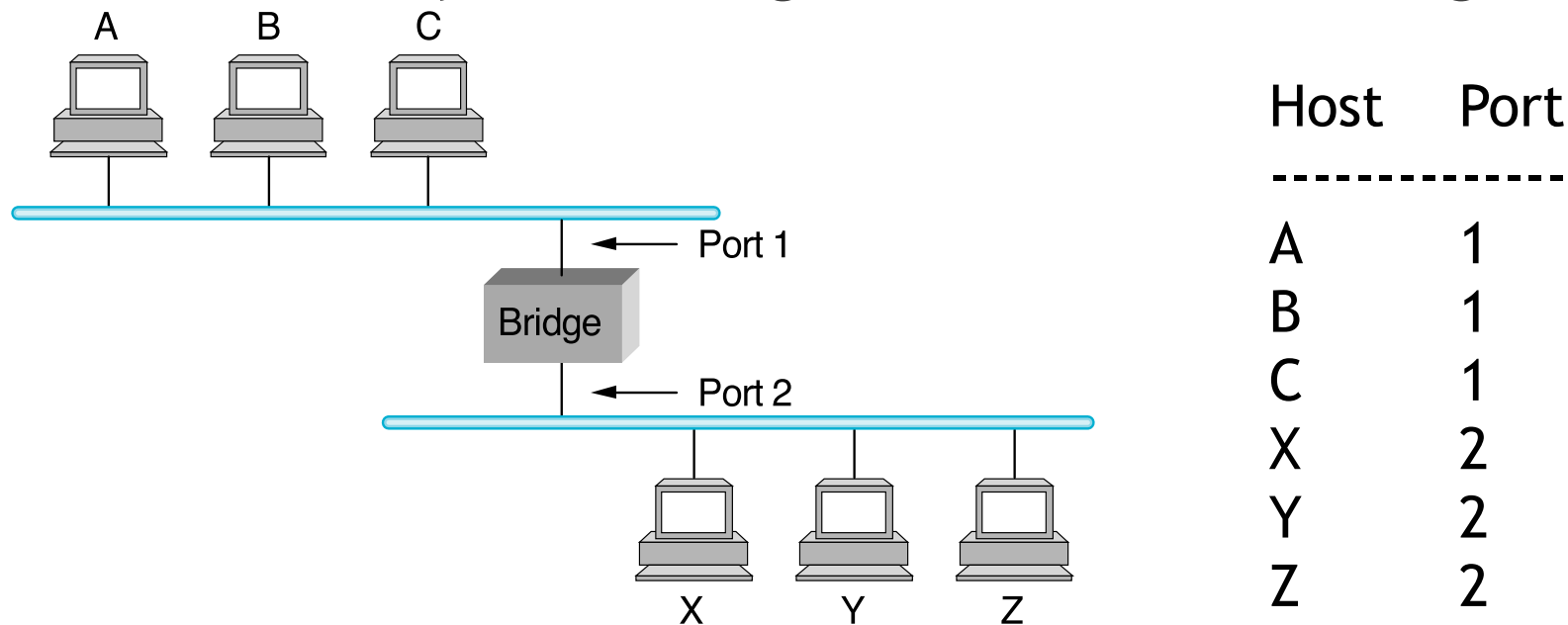
- Consider the following figure
 - When a frame from host A that is addressed to host B arrives on port 1, there is no need for the bridge to forward the frame out over port 2.



- How does a bridge come to learn on which port the various hosts reside?

Bridges and LAN Switches

- Solution: Download a *forwarding table* into the bridge



- Who does this configuration, for each bridge?
 - Human? Too much work for maintenance

Bridges and LAN Switches

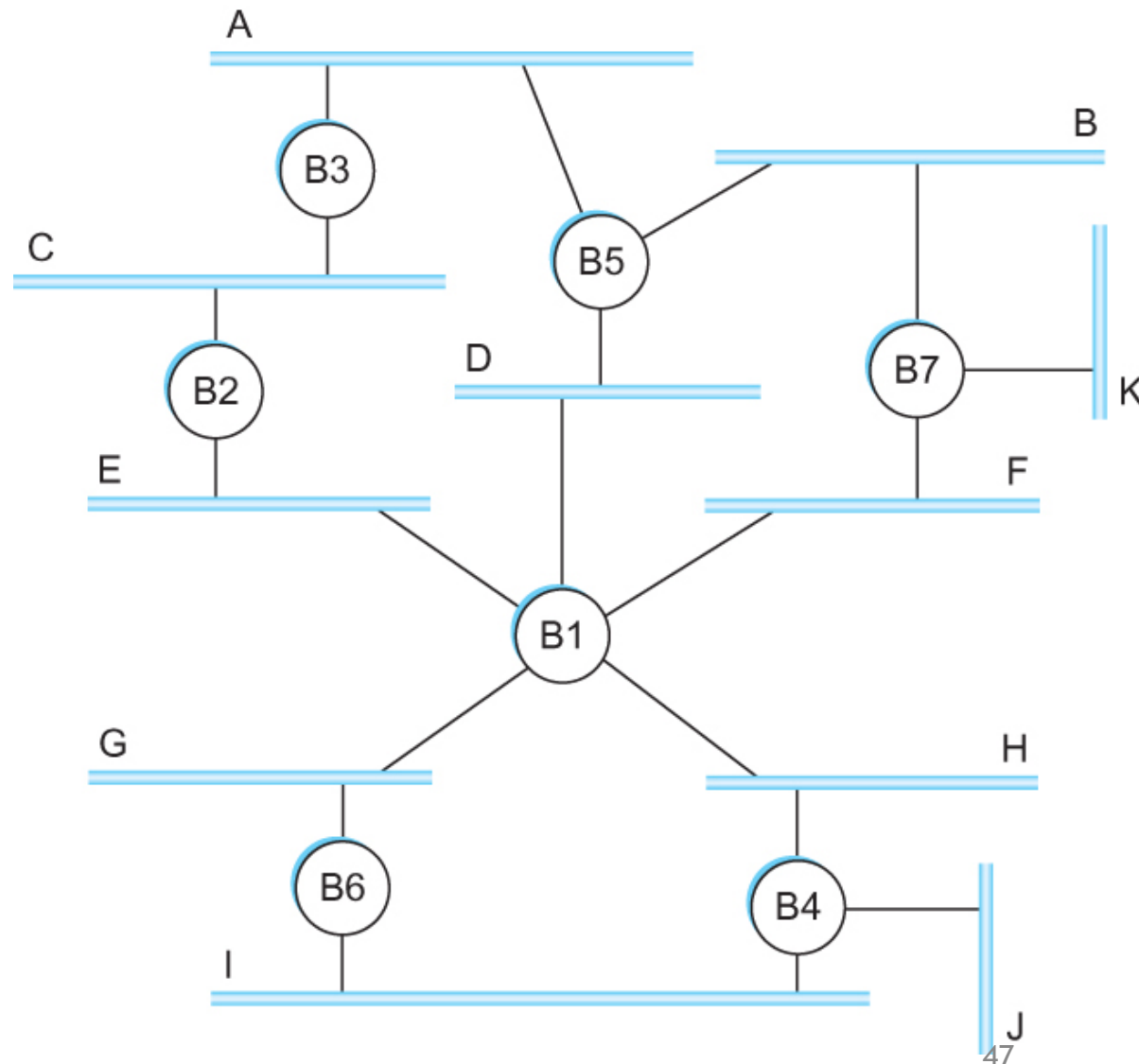
- The bridge can learn this information by *inspecting* the source address in *all the frames it observes at its ports*
- Record the information at the bridge and build the table
- When a bridge first boots, this table is empty
 - Entries are added over time
 - A timeout is associated with each entry
 - The bridge discards the entry after a specified period of time, to protect against the situation in which a host is moved from one network to another
- If the bridge receives a frame that is addressed to host not currently in the table, forwards the frame out on all other ports in flooding

Bridges and LAN Switches

- If the bridge receives a frame addressed to broadcast:
 - Flooding: forward it to all ports, apart the port where the frames come from
- If the bridge receives a frame addressed to a multicast address
 - Simple strategy: flooding: forward all multicast frames to all ports (apart the port where the frames come from), as broadcast, and let the host decide whether they are interested in or not
 - Can generate lot of unwanted traffic on interfaces
 - More sophisticated: Learn when no group members are associated to a port
 - Accomplished by having each member of group G send a frame to bridge multicast address with G in source field

Bridges and LAN Switches

- This strategy works fine as long as the extended LAN does not have a loop in it
 - Frames potentially loop through the extended LAN forever
 - Bridges B1, B4, and B6 form a loop



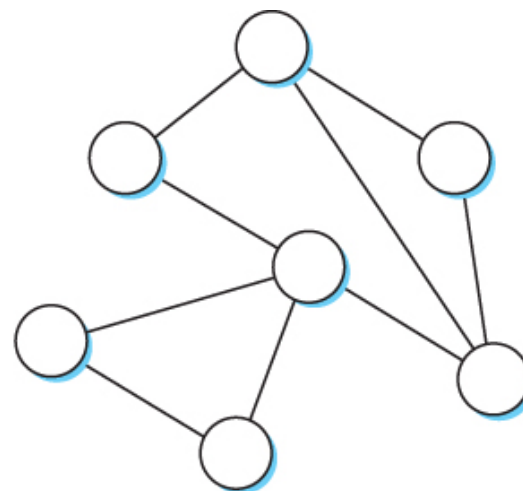
Bridges and LAN Switches

- How does an extended LAN come to have a loop in it?
 - Network is managed by more than one administrator
 - For example, it spans multiple departments in an organization
 - It is possible that no single person knows the entire configuration of the network
 - A bridge that closes a loop might be added without anyone knowing
 - Loops are built into the network to provide redundancy in case of failures
- Solution: **Distributed Spanning Tree Algorithm**

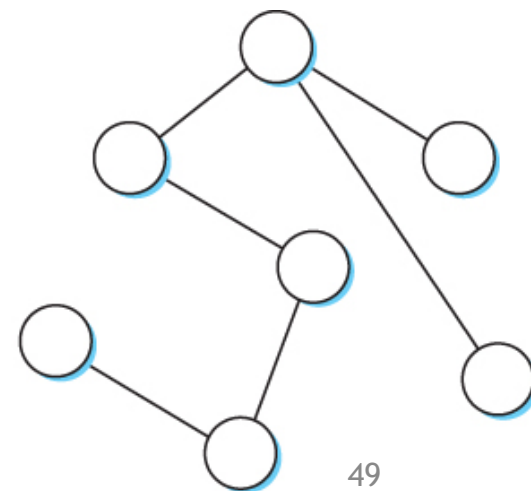
Spanning Tree Algorithm

- Think of the extended LAN as being represented by a graph that possibly has loops (cycles)
 - Nodes can be bridges and simple LANs
- A spanning tree is a sub-graph of this graph that covers all the vertices (of type simple LANs) but contains no cycles
 - Spanning tree keeps all the vertices of the original graph but throws out some of the edges
- Example of (a) a cyclic graph;
(b) a corresponding spanning tree.

(a)



(b)



Spanning Tree Algorithm

- “Rapid Spanning Tree”: a protocol used by a set of bridges to agree upon a spanning tree for a particular extended LAN
- Developed by Radia Perlman at Digital (1985)
- IEEE 802.1D (2004) specification for LAN bridges is based on this algorithm
- Used in almost all switches, nowadays
 - (but beware that in some networks there can be old switches without the Spanning Tree)

Spanning Tree Algorithm

- Main idea: each bridge decides the ports over which it is and is not willing to forward frames
 - By disabling ports from the topology, the extended LAN is reduced to a tree
 - It is even possible that an entire bridge will not participate in forwarding frames
- Algorithm is dynamic
 - The bridges are always prepared to reconfigure themselves into a new spanning tree if some bridges fail

Spanning Tree Algorithm

- Algorithm selects ports as follows:
 - Each bridge is given a unique identifier (usually the lowest id of its ports)
 - B1, B2, B3,...and so on
- The bridge with the smallest id will be elected as the root of the spanning tree
- Each bridge computes the *shortest path to the root* and notes which of its ports is on this path
 - This port is selected as the bridge's preferred path to the root, hence is called **root port**
- Finally, all the bridges connected to a given LAN elect a single designated bridge that will be responsible for forwarding frames upwards and downwards

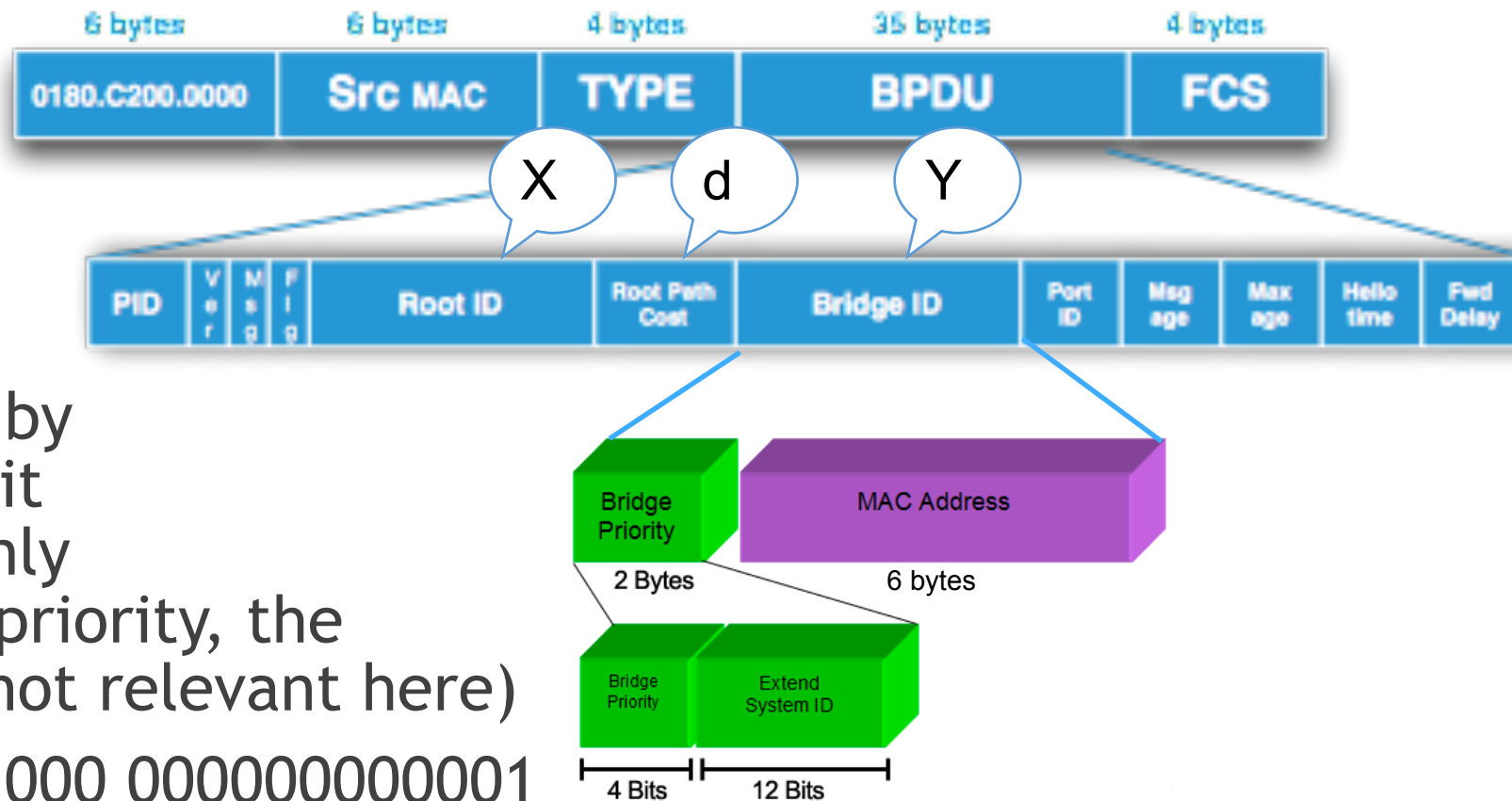
Spanning Tree Algorithm

- For each LAN, the bridge which is closest to the root is the designated bridge
 - If two or more bridges are equally close to the root, select bridge with the smallest id
- Each bridge may be connected to more than one LAN
 - So it participates in the election of a designated bridge for each LAN it is connected to.
 - Each bridge decides if it is the designated bridge relative to each of its ports
 - The bridge forwards frames over only those ports for which it is the designated bridge

Spanning Tree Algorithm

- Bridges reach this agreement by exchanging simple configuration messages (called BPDU) between immediate neighbours (i.e. over the shared segments)
- Each message is a triple (X, d, Y) where
 - Y is the identifier of the bridge generating the message
 - X is the identifier of the root bridge, according to Y
 - d is the distance from X to Y , in number of LANs to cross
- In reality, these messages are defined by 802.1D, carry also other data (like bridge priority, costs, etc) and are inside 802.3 frames with a special destination address

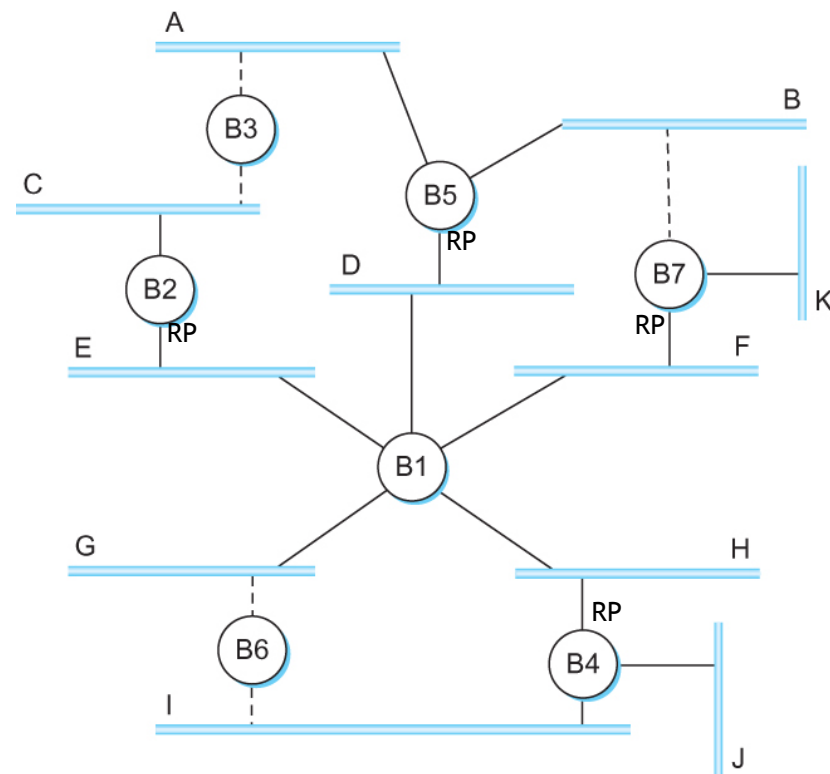
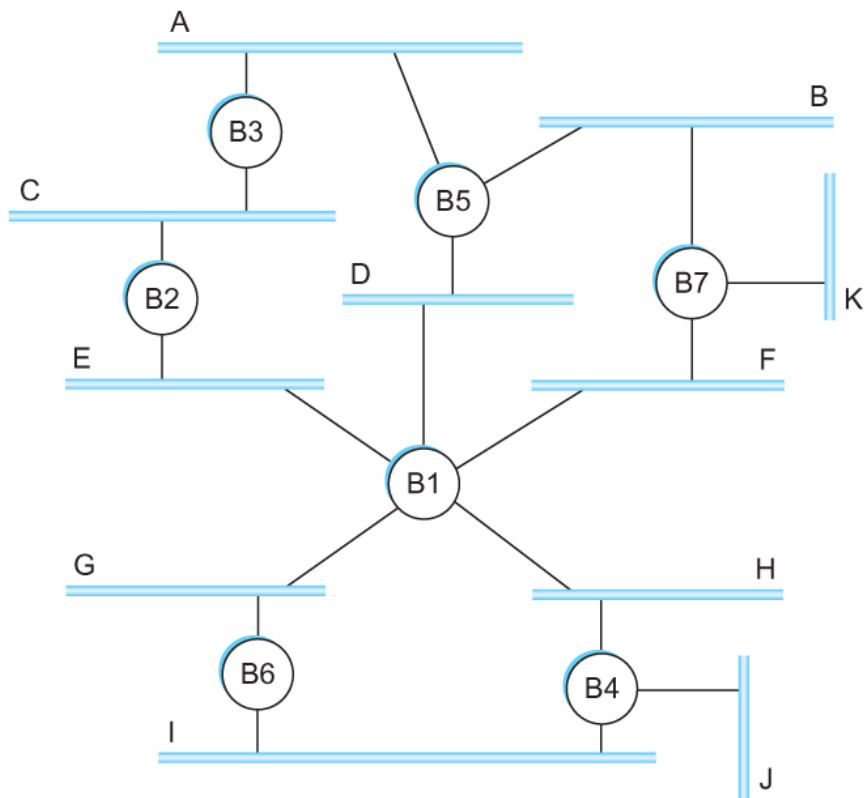
Spanning Tree Algorithm



- Each bridge ID is composed by the MAC address and a 16-bit bridge priority (of which, only the first 4 bits are the real priority, the others indicate the VLAN - not relevant here)
- Default priority is $32769 = 1000\ 0000000000001$
- It can be changed by the administrator, if he wants a specific bridge to become the root.
 - E.g., by putting $4097 = 0001\ 0000000000001$

Spanning Tree Algorithm

- B1 is the root bridge
- B3 and B5 are connected to LAN A, but B5 is the designated bridge
- B5 and B7 are connected to LAN B, but B5 is the designated bridge



Spanning Tree Algorithm

- Initially each bridge X thinks it is the root, so it sends a configuration message $(X, 0, X)$ on each of its ports identifying itself as the root and giving a distance to the root of 0
- Upon receiving a configuration message (Y, d, Z) over a particular port, the bridge checks to see if the new message is better than the current best configuration message recorded for that port
- The new configuration is better than the currently recorded information if
 - It identifies a root with a smaller id, or
 - It identifies a root with an equal id but with a shorter distance, or
 - The root id and distance are equal, but the sending bridge has a smaller id

Spanning Tree Algorithm

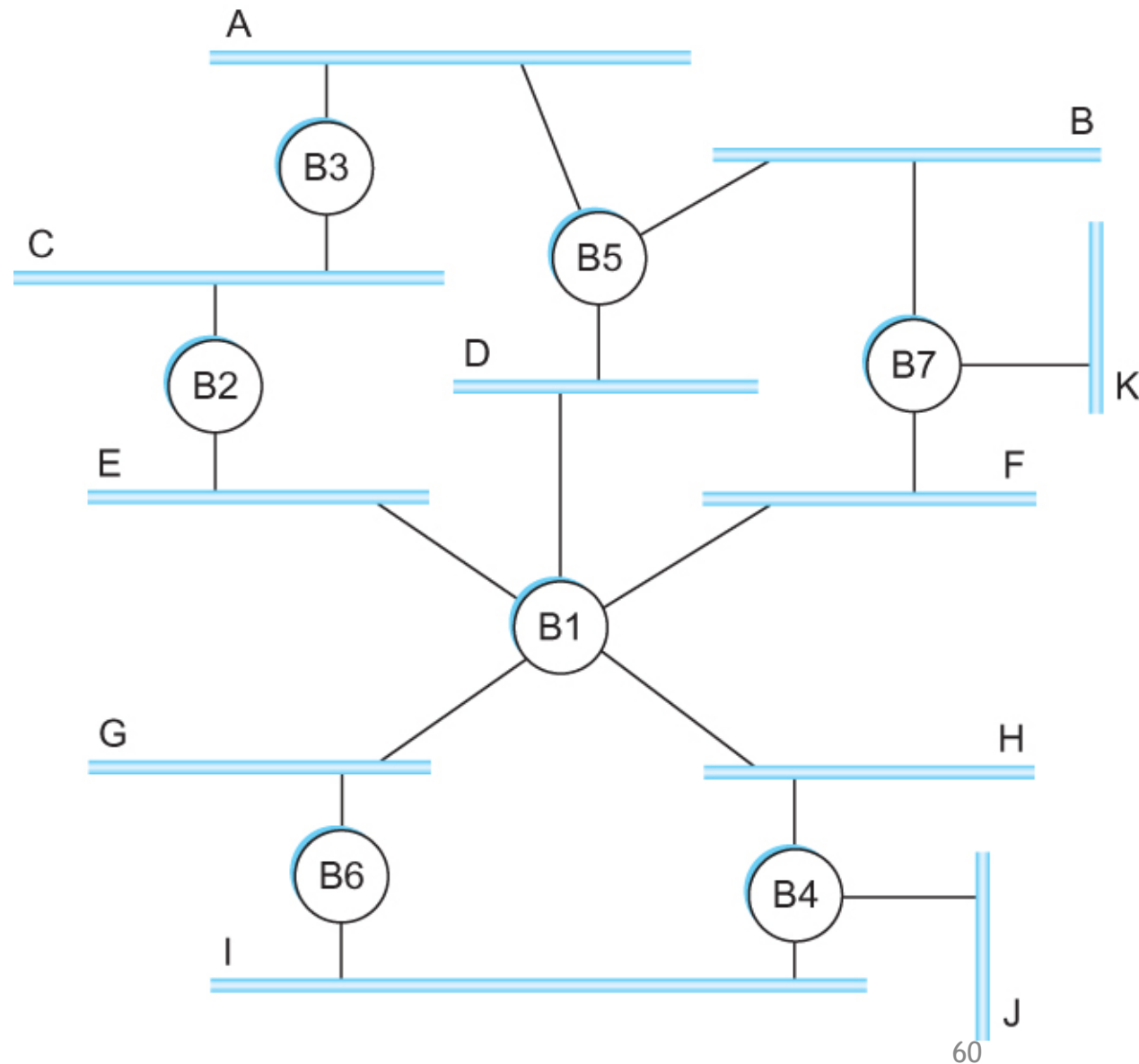
- If the new message is better than the currently recorded one,
 - It first adds 1 to the distance-to-root field
 - The bridge discards the old information and saves the new information
- When a bridge receives a configuration message indicating that it is not the root bridge (that is, a message from a bridge with smaller id)
 - The bridge stops generating configuration messages on its own
 - Only forwards configuration messages from other bridges after 1 adding to the distance field

Spanning Tree Algorithm

- When a bridge receives a configuration message that indicates it is not the designated bridge for that port
 - i.e., a message from a bridge that is closer to the root or equally far from the root but with a smaller id
 - The bridge stops sending configuration messages over that port
- When the system stabilizes,
 - Only the root bridge is still generating configuration messages.
 - Other bridges are forwarding these messages only over ports for which they are the designated bridge

Spanning Tree Algorithm

- Consider the situation when the power had just been restored to the building housing the following network
- All bridges would start off by claiming to be the root
- Denote a configuration message from node X in which it claims to be distance d from the root node Y as (Y, d, X)



RP = root port

Spanning Tree Algorithm

- Even after the system has stabilized, the root bridge continues to send configuration messages periodically
 - Other bridges continue to forward these messages
- When a bridge fails, the downstream bridges will not receive the configuration messages
- After waiting a specified period of time, they will once again claim to be the root and the algorithm starts again

Spanning Tree Algorithm

- Limitations of this algorithm
 - Although the algorithm is able to reconfigure the spanning tree whenever a bridge fails, it is not able to forward frames over alternative paths for the sake of routing around a congested bridge
 - Moreover, disabling ports in switches leads to longer paths between two LANs (e.g. consider the path between LANs B and K)

Limitation of Bridges

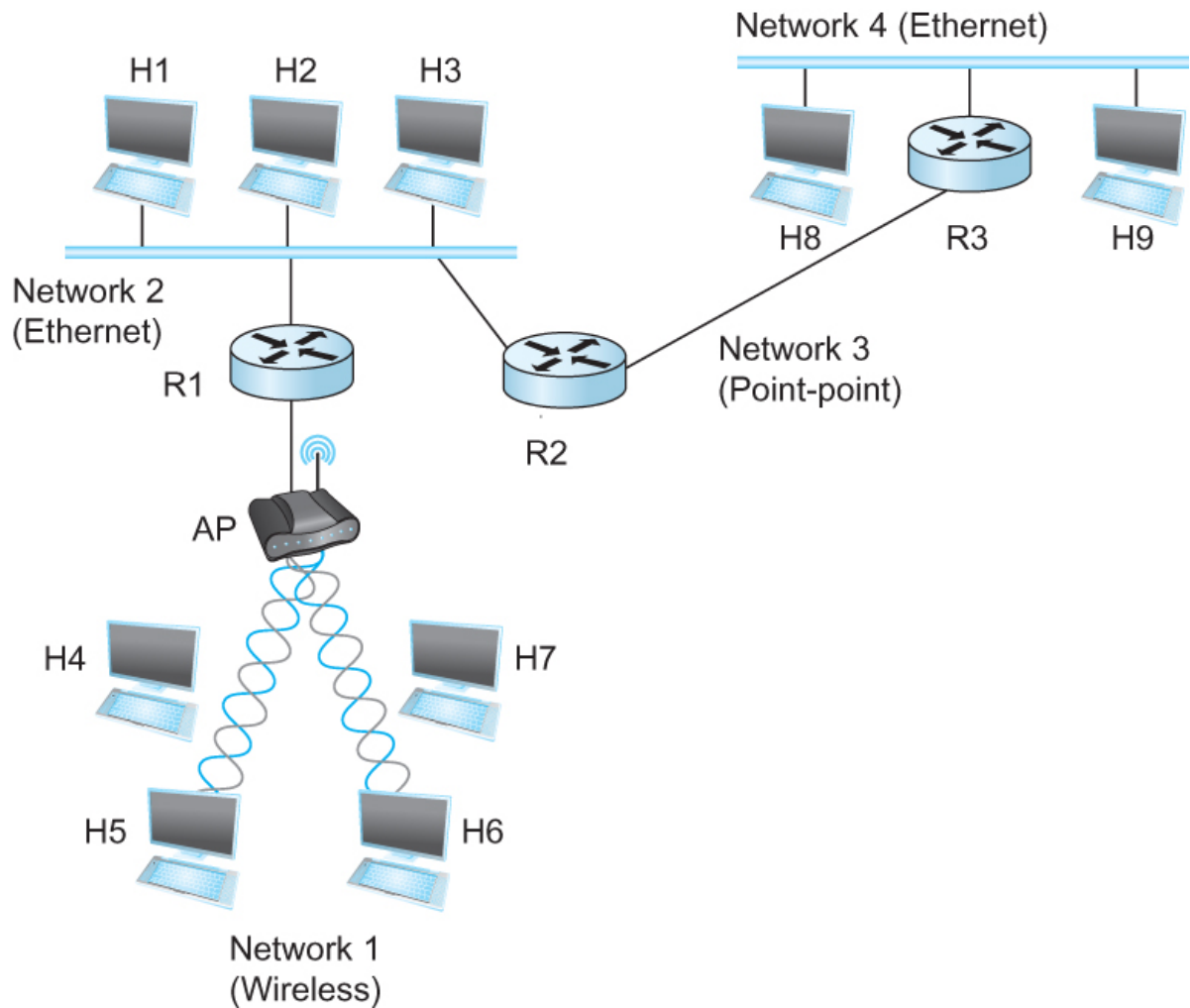
- Do not scale
 - Spanning tree algorithm does not scale (linear time, does not take advantage of hierarchy)
 - Broadcast does not scale: too much traffic on a large scale (nor we need it)
- Do not accommodate heterogeneity
 - all parts of the network must have the same kind of address, and similar features (e.g. broadcast)
- What if two segments admit different MTUs?
 - e.g. Ethernet (802.3): 1500 byte; WiFi (802.11): 2346 byte
 - in one direction, the bridge has to *fragment* a frame into 2 or more frames, which must be recomposed at the datalink level by receiver (*PAF fragmentation*)
- To overcome these limitations, we need to abstract from datalink details, and hence to move to level 3 – **Internetworking**



Internetworking

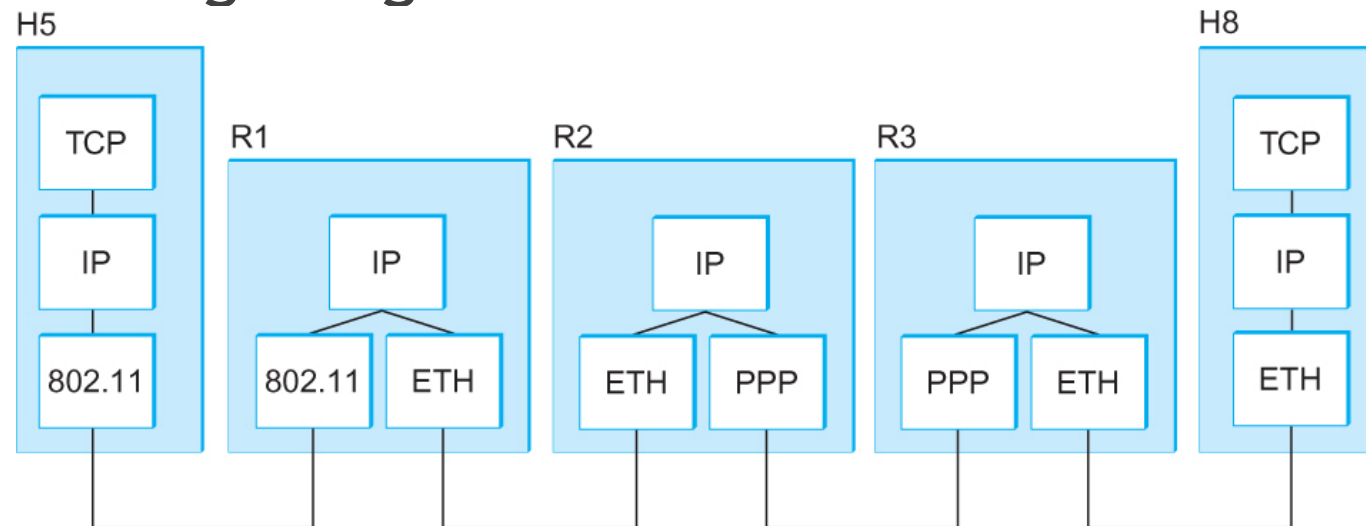
Internetworking

- What is an internetwork?
 - An arbitrary collection of networks interconnected to provide some sort of **host-to-host packet delivery service**
 - Example aside: a simple internetwork where H represents hosts and R represents routers

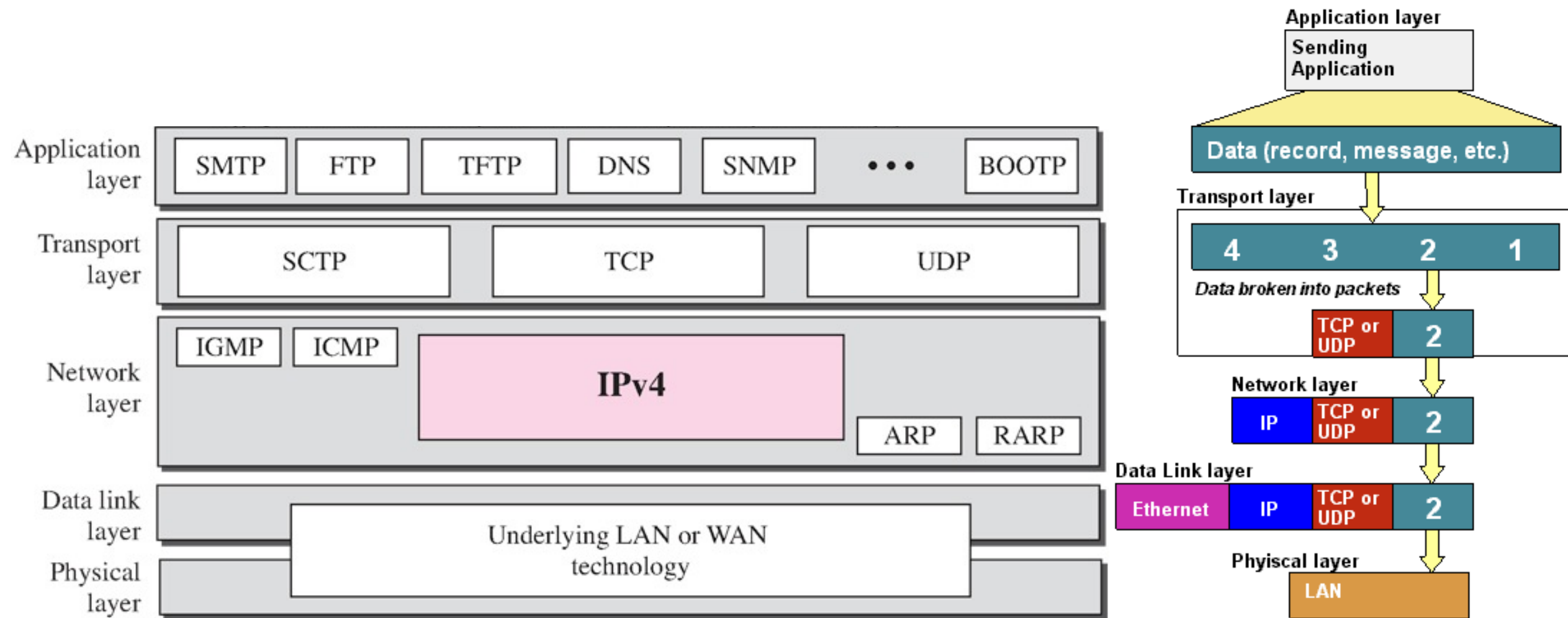


Internetworking

- In order to implement this abstraction we need a level 3 protocol
- Most used nowadays: IP (which stands for *Internet Protocol*)
 - Key tool used today to build scalable, heterogeneous internetworks
 - It runs on all the nodes in a collection of networks and defines the infrastructure that allows these nodes and networks to function as a single logical internetwork

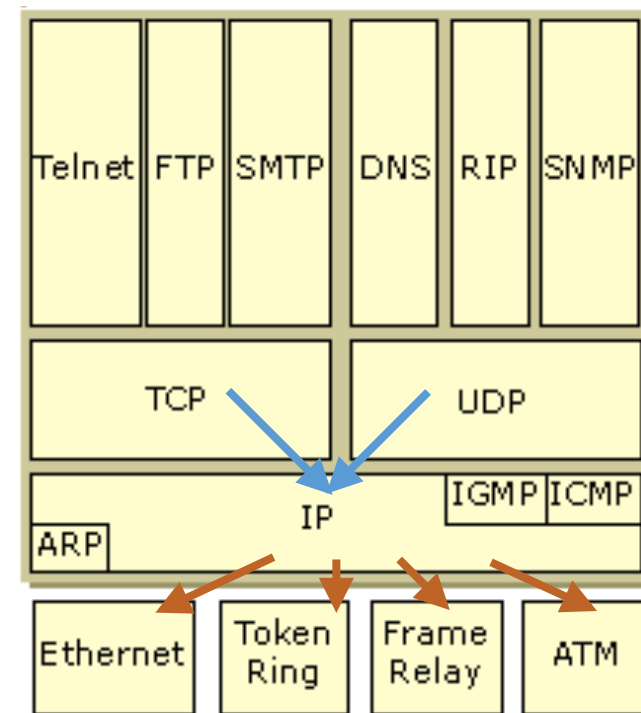


IP: position in TCP/IP stack



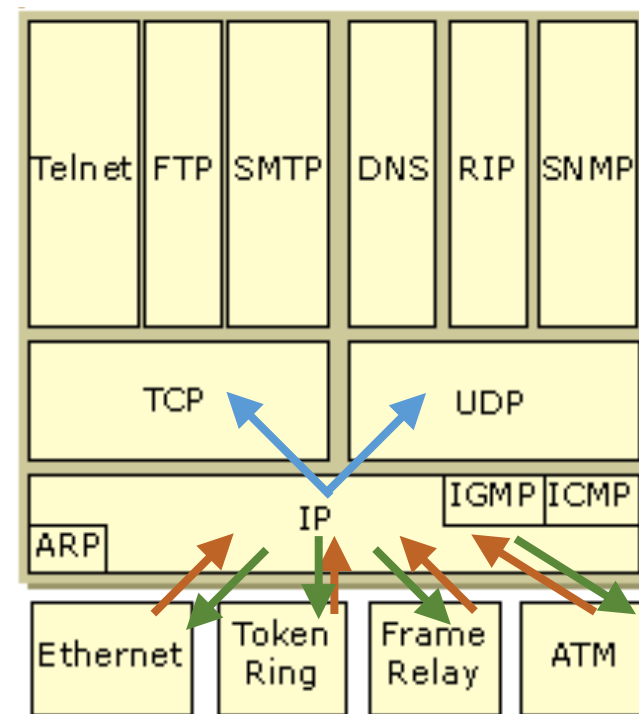
Internetworking

- Key aspect: the IP layer does also **packet forwarding**
- A packet (sometimes called “datagram”) can arrive to IP layer in two ways:
 - when an upper layer (e.g. TCP or UDP) require to send some data (“payload”) to another host:
 1. then this data is encapsulated in a IP datagram
 2. And the datagram is sent to the next node using an appropriate datalink protocol
 - Thus, an application can send/transmit data via any underlying interface, without knowing it

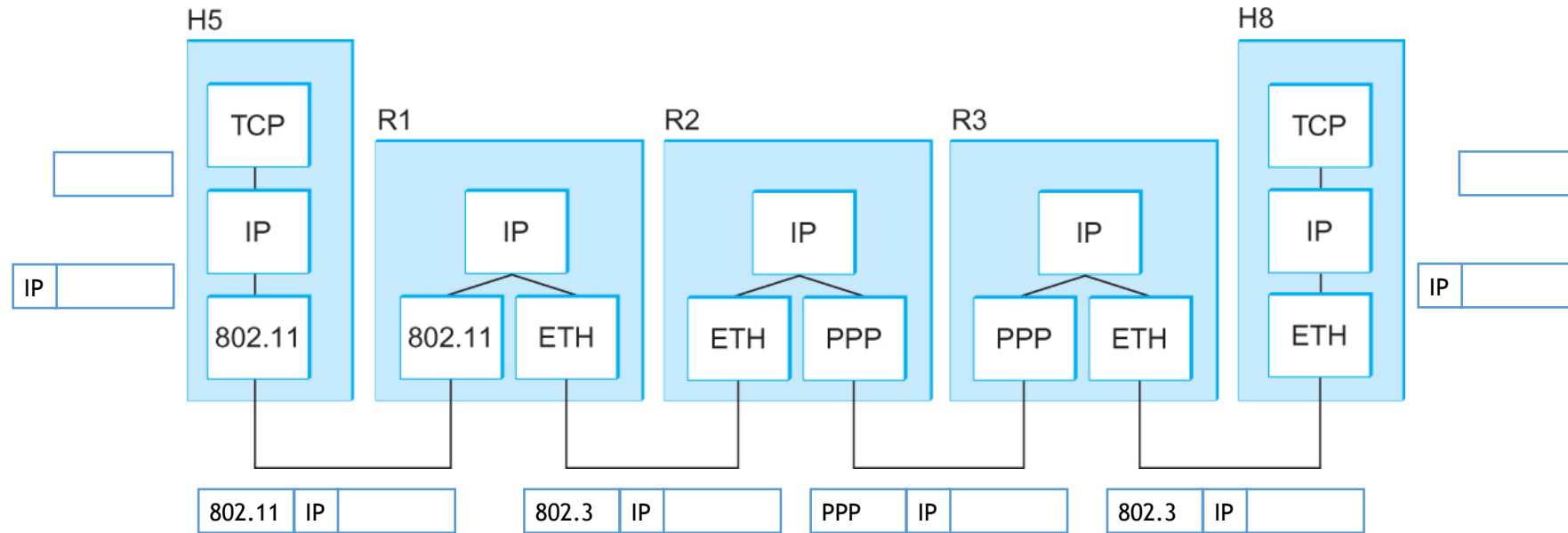


Internetworking

- The other situation is a packet received from a lower layer (i.e. from an interface).
 - if it is addressed to the local host (“myself”): the IP header is stripped and the payload is delivered up to the right transport protocol
 - if it is addressed to another host: it can be sent out using an interface, i.e. using an underlying datalink layer (if the host is instructed to forward these packets)



Internetworking - Routers

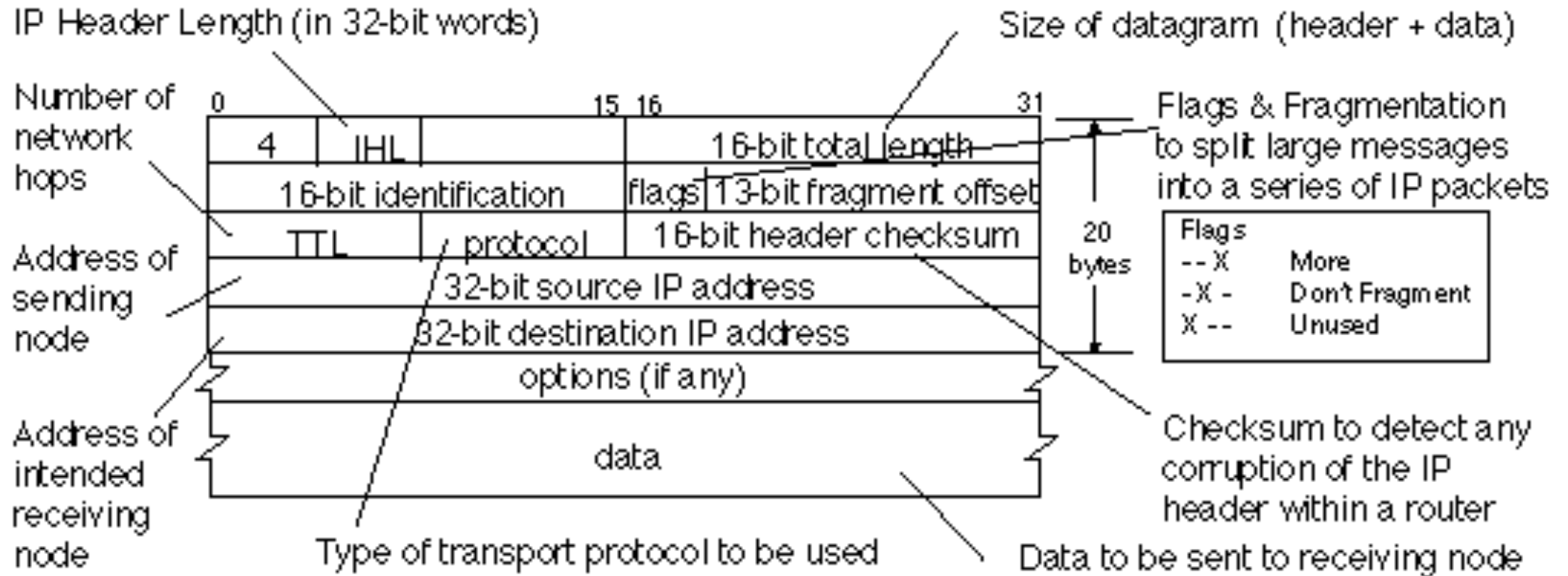


- A **router** is a host with multiple interfaces whose IP layer forwards packets from one interface to another (and possibly no upper layers)
- They operate with a **store-and-forward** strategy, but, differently from switches at level 2, they **abstract** from the underlying link technology
 - each link can use different technologies
 - The IP datagram is encapsulated/decapsulated and encapsulated again, etc.
 - Forwarding is decided on IP addresses, not on LAN addresses

IP Service Model

- Packet Delivery Model
 - Connectionless model for data delivery
 - Best-effort delivery (unreliable service)
 - packets are lost
 - packets are delivered out of order
 - duplicate copies of a packet are delivered
 - packets can be delayed for a long time
- Global Addressing Scheme
 - Provides a way to identify all hosts in the internetwork
 - Abstracts from the underlying layer 2 (MAC) addresses

Packet Format



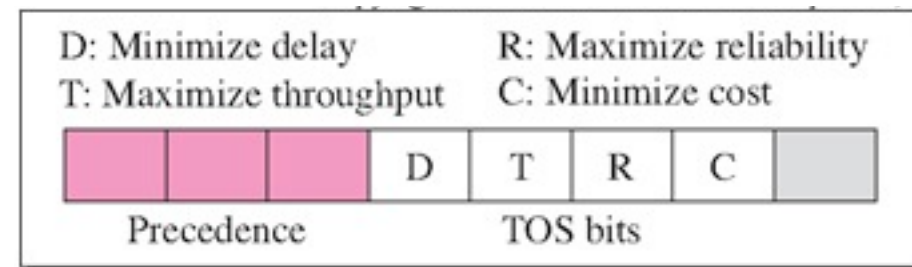
Datagram IPv4

- Version (4): currently 4 for IPv4 (or 6 for IPv6)
- Hlen (4): number of 32-bit words in header (hence real length is 4xHlen)
- TOS (8): type of service
- Length (16): total length of this packet, including header, in bytes
- Ident (16): used by fragmentation
- Flags (3) and Offset (13): used by fragmentation.
 - Flag bit 0: must be 0
 - Flag bit 1: Don't Fragment (DF)
 - Flag bit 2: More Fragments (MF)
- TTL (8): number of hops this datagram can travel
- Protocol (8): demux key (TCP=6, UDP=17)
- Checksum (16): of the header only
- DestAddr & SrcAddr (32 bits each)

Protocol	Value
ICMP	1
IGMP	2
TCP	6
UDP	17
OSPF	89

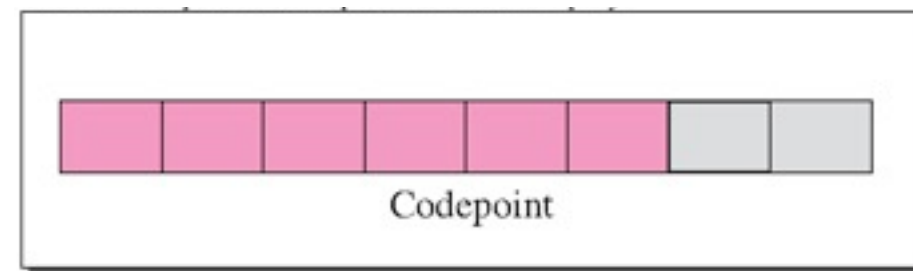
Datagram IPv4

- TOS - Type of Service (8 bit)
 - should be used to specify how to treat the datagram
 - Not very followed nowadays... we cannot trust the user!
- Original interpretation: TOS
 - Priority in case of congestion
 - network handling datagrams should have the precedence
 - Kind of service requested:
 - 0000: normal (default)
 - 1000 (D): minimize delay
 - 0100 (T): maximize throughput
 - 0010 (R): maximize reliability
 - 0001 (C): minimize cost
 - every transport protocol has an associated default service
 - TELNET, FTP (com), SMTP (com): D; SNMP: R; FTP (dati), SMTP (dati): T; ecc.



Datagram IPv4

- Modern interpretation: **Differentiated services (DSCP)**
 - New codes
 - If 3 rightmost bits = 000:
 - 3 leftmost bits are interpreted as priority, as in TOS
 - If 3 rightmost bits $dx \neq 000$:
 - 6 leftmost bit define 64 services
 - xxxxx0: standard IETF services
 - xxxx11: services defined by local authorities
 - xxxx01: temp. use, experimental

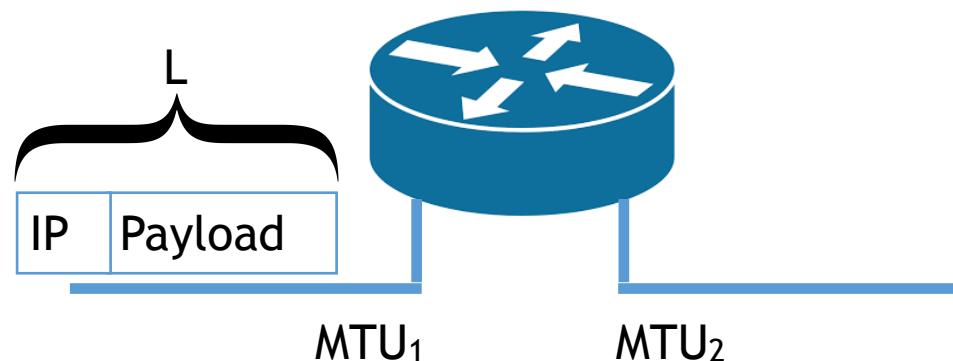


IP Fragmentation and Reassembly

- Each physical network has some MTU (*Maximum Transmission Unit*), which depends on the technology
- At transport level we do not know about MTU
- Even if we can discover the MTU of the local link, on the path from sender to destination we may cross a link with a smaller MTU
- Moreover, the “minimum” MTU from sender to destination can change during time, e.g. because packets may follow different paths, or the links change their features (e.g. PPP renegotiation)

Network/protocol	MTU (bytes)
Hyperchannel	65535
Token Ring	17914
FDDI	4352
WiFi	2346
Ethernet	1500
X.25	576
PPP	Negotiated

IP Fragmentation and Reassembly



- What should a router do if it has to forward a datagram which does not fit in a single frame on the output interface (i.e. $L > MTU_2$)?
 - If the Don't Fragment flag (DF) is 1, just drop the datagram and notify the sender (using ICMP); maybe the sender can reduce the size of the packets it sends
 - If $DF=0$, the router can proceed with **fragmentation**

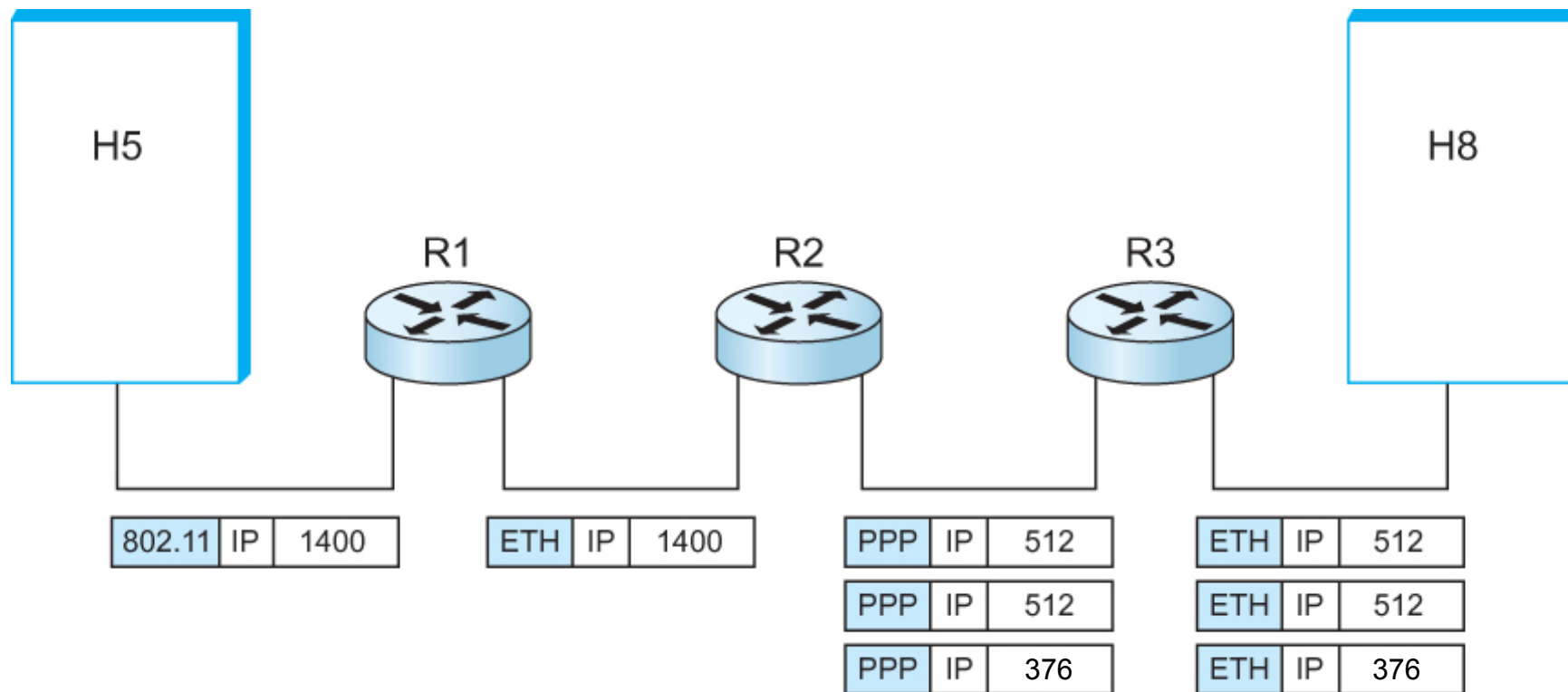
IP Fragmentation and Reassembly

- Datagrams are fragmented at network level, transparently for higher levels. Higher levels see only whole datagrams.
- Strategy
 - Fragmentation occurs in a router when it receives a datagram that it wants to forward over a network which has $MTU < \text{datagram length}$
 - All the fragments carry the same identifier in the Ident field
 - Fragments are self-contained packets; they travel independently from each other, and can be fragmented further
 - The size of each fragment is chosen by the router, such that $\text{header} + \text{payload} \leq MTU$
 - In general, the size of the payload is chosen to be the largest multiple of 8 less than $MTU - \text{header length}$
 - E.g.: if $MTU = 536$, $\text{header} = 20$ byte, then $536 - 20 = 516$, and the largest multiple of 8 less than 516 is 512. So each packet is $512 + 20$ bytes long

IP Fragmentation and Reassembly

- Reassembly is done at the receiving host:
 - When a fragment of a new identifier arrives, a buffer is allocated (and associated to that identifier)
 - Each fragment's payload is placed in this buffer, according to its offset
 - (If a fragment arrives twice, the last arrived overwrites the previous one)
 - When all fragments have arrived, the buffer containing the reconstructed payload is passed to the higher levels
 - No recovery for missing fragments: if some are still missing after the timeout (~15-60 seconds), all data is discarded and an ICMP message is sent to sender

IP Fragmentation and Reassembly



- IP datagrams traversing the sequence of physical networks

IP Fragmentation and Reassembly

- Header fields used in IP fragmentation.

(a) Unfragmented packet;

(b) corresponding fragmented packets.

Notice: offset field = real offset / 8 (e.g.
64 = 512 / 8)

hence in each fragment (but last one) the
length of payload is a multiple of 8.

(a)

Start of header			
Ident = x		0	Offset = 0
Rest of header			
1400 data bytes			

(b)

Start of header			
Ident = x		1	Offset = 0
Rest of header			
512 data bytes			

Start of header			
Ident = x		1	Offset = 64
Rest of header			
512 data bytes			

Start of header			
Ident = x		0	Offset = 128
Rest of header			
376 data bytes			

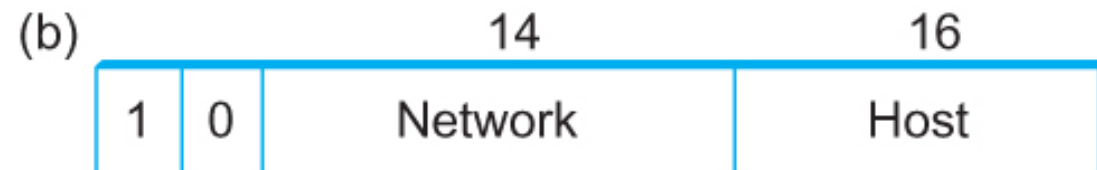
Global Addresses

- Properties
 - *globally unique*: an interface of an host may have many addresses, but an address cannot be assigned to more than one interface
 - *hierarchical*: the address is split in two parts
 - First part denotes the *network* – assigned uniquely to a single entity (*Autonomous System*) by central authorities
 - Second part denotes the *host* with the network
 - Each autonomous system can assign addresses within its networks as it prefers
- In IPv4: 32 bit, written using the dot notation
 - 10.3.2.4
 - 128.96.33.81
 - 192.12.69.77
- How to tell the Network number and the host number?

Network number	Host number
----------------	-------------

Global Addresses

- Originally, 3 classes of networks:
 - Class A:** addresses beginning with 0; next 7 bits are interpreted as network number; remaining 24 bits are the host number within network
 - Class B:** addresses beginning with 10; next 14 bits are interpreted as network number, the remaining 16 bits are the host number within network
 - Class C:** addresses beginning with 110; next 21 bits are interpreted as network number, the remaining 8 bits are the host number within network
 - (Addresses beginning with 111 are not used for host addressing: Class D (1110) is for multicast, and Class E (1111) is unused)



Global Addresses (IPv4)

- Within a network, two special addresses cannot be used for hosts:
 - first address (all “0” in host part) identifies the network (e.g. 158.110.0.0)
 - last address (all “1” in host part) is used for broadcast within the network (e.g. 158.110.255.255)
 - Remaining address can be assigned to hosts (e.g., 158.110.0.1-158.110.255.254)
- Number of hosts for each network
 - Class A: $2^7 = 128$ possible networks, with $2^{24} - 2 = 16777214$ hosts each
 - Class B: $2^{14} = 16384$ possible networks, with $2^{16} - 2 = 65534$ hosts each
 - Class C: $2^{21} = 2097152$ possible networks, with $2^8 - 2 = 254$ hosts each
- Overall: $2,1 \cdot 10^6$ networks (actually less, due to “special” networks), and theoretical limit of possible hosts: $3753869056 \approx 3,75 \cdot 10^9$

IP Datagram Forwarding

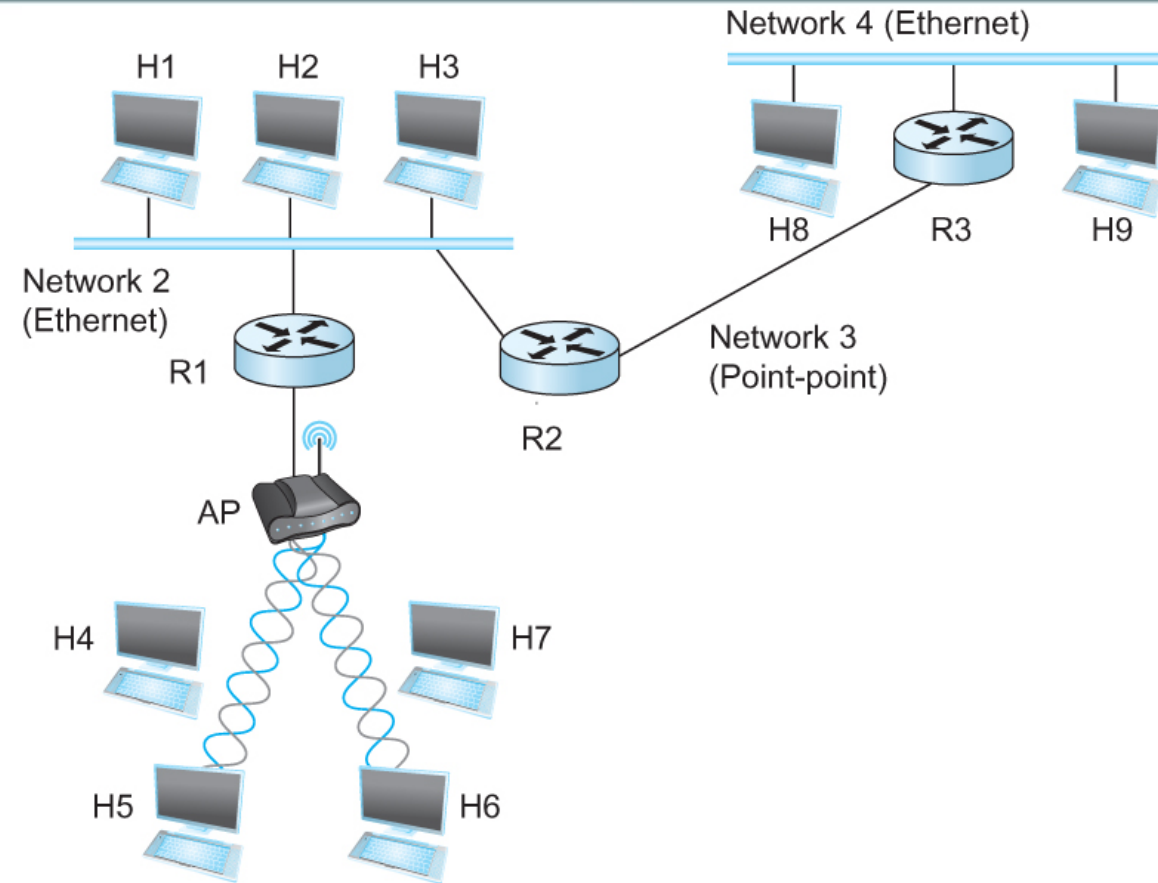
- Cannot be by destination address, like in Level 2 switches
 - Too many addresses (in the order of 10^9)
 - Tables would take too much memory, too slow to look up
 - (Actually, level 3 switches do exist, but are limited inside single AS; cannot scale to global size)
- Instead, forwarding is by *destination network*, that is, the network which the destination address belongs to
- forwarding table maps network number into next hop

IP Datagram Forwarding

- Strategy followed by each node:
 - every packet contains destination's address, from which we can get the destination network address
 - if the node is directly connected to destination network, then forward the packet to host
 - if not directly connected to destination network, then forward the packet to a router which should know how to handle it
- each host has a default router (*gateway*) and maintains a forwarding table
 - some algorithm will be needed for this

IP Datagram Forwarding

- Example: routing table for router R2
- In real tables, the destination (NetworkNum) is the network address
 - e.g. 158.110.0.0, 158.111.0.0, ...
- Hence each local network must be assigned a different network address
 - Hosts in different LANS have addresses in different networks



NetworkNum	NextHop
1	R1
2	Interface 1
3	Interface 0
4	R3

IP Datagram Forwarding

- **Algorithm:** (omitting TTL issues)
 - if (NetworkNum of destination == NetworkNum of one of my interfaces) then
 - directly deliver packet to destination over that interface
 - else if (NetworkNum of destination is in my forwarding table) then
 - deliver packet to NextHop router
 - else if there is a default router
 - deliver packet to default router
 - else drop packet
- For a host with only one interface and only one default router in its forwarding table, this simplifies to
 - if (NetworkNum of destination == my NetworkNum) then
 - deliver packet to destination directly
 - else
 - deliver packet to default router

Subnetting

- A and B networks can be too large for a single physical network
- Add another level to address/routing hierarchy: subnet
- Subnet masks define variable partition of host part of class A and B addresses, according to physical nets
- Subnets visible only within network (i.e. autonomous system)

Network number	Host number
----------------	-------------

Class B address

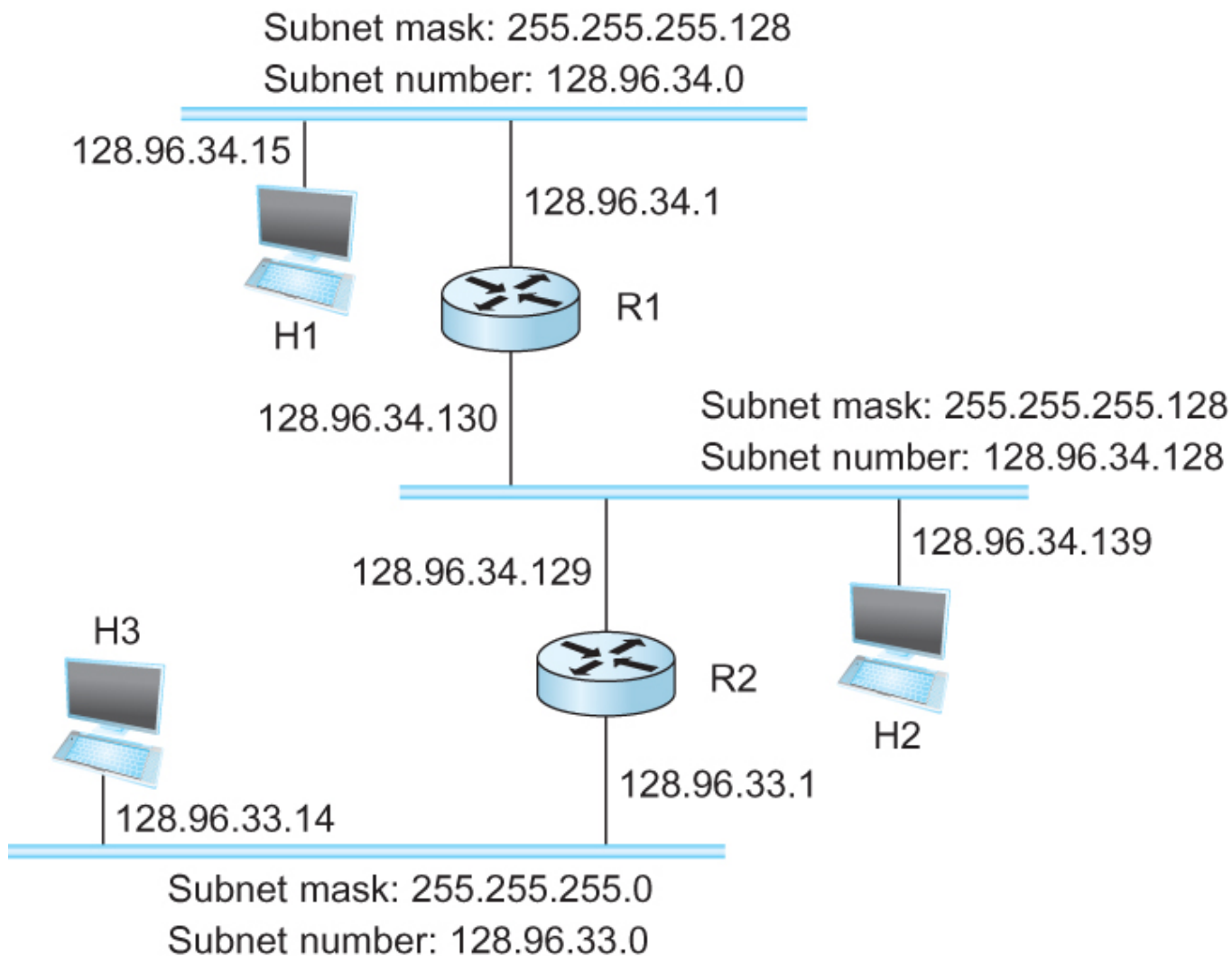
11111111111111111111111111111111	00000000
----------------------------------	----------

Subnet mask (255.255.255.0)

Network number	Subnet ID	Host ID
----------------	-----------	---------

Subnetted address

Subnetting



SubnetNumber	SubnetMask	NextHop
128.96.34.0	255.255.255.128	Interface 0
128.96.34.128	255.255.255.128	Interface 1
128.96.33.0	255.255.255.0	R2

Forwarding table at router R1

Subnetting

- Forwarding Algorithm

```
D = destination IP address
for each entry <SubnetNum, SubnetMask, NextHop>
    D1 = SubnetMask & D           // bitwise "and"
    if D1 = SubnetNum
        if NextHop is an interface
            deliver datagram directly to destination and return
        else
            deliver datagram to NextHop (a router)
        break
if no entry matches
    drop datagram
```

Subnetting

- Would use a default router if nothing matches
- Not necessary for all 1's in subnet mask to be contiguous
 - weird subnets can be done!
- Can put multiple subnets on one physical network
- Subnets are decided internally by network administrator, and not visible from the rest of the Internet
 - usually assigned by logical separations (e.g. different offices, branches, etc) and physical separations (different physical networks)

Esercizi

Nella rete 192.168.0.0/16 si vogliono definire tre sottoreti contigue, a partire dall'indirizzo 192.168.0.0. Le tre sottoreti devono essere le più piccole necessarie per contenere rispettivamente 50, 60, 70 indirizzi. Si diano gli indirizzi di tali sottoreti, completi di CIDR.

R: Per 50 e 60 indirizzi bastano 6 bit, quindi /26 ciascuna. Collettivamente stanno tutte e due nella 192.168.0.0/25. Invece quella da 70 indirizzi ha bisogno di una /25 separata. Quindi le tre sottoreti sono 192.168.0.0/26, 192.168.0.64/26, 192.168.0.128/25. Soluzione alternativa: 192.168.0.0/25, 192.168.0.128/26, 192.168.0.192/26.

Si considerino i seguenti due indirizzi IP: 192.168.5.9 e 192.168.6.14. (a) Qual è la sottorete più piccola (ossia con il minor numero di indirizzi) che contiene entrambi questi indirizzi? (dare indirizzo/CIDR). (b) Qual è l'indirizzo di broadcast di tale sottorete?

R: (a) 192.168.4.0/22. (b) 192.168.7.255.

Classless Inter-Domain Routing

- A technique that addresses two scaling concerns in the Internet
 - The growth of backbone routing table as more and more network numbers need to be stored in them
 - Potential exhaustion of the 32-bit address space
- Address assignment efficiency
 - Arises because of the IP address structure with class A, B, and C addresses
 - Forces us to hand out network address space in fixed-size chunks of three very different sizes
 - A network with two hosts needs a class C address
 - Address assignment efficiency = $2/254 = 0.78\%$
 - A network with 256 hosts needs a class B address
 - Address assignment efficiency = $256/65535 = 0.39\%$

Classless Addressing

- Exhaustion of IP address space centers on exhaustion of the class B network numbers
- Possible solution
 - Do not accept any *Autonomous System* (AS) that requests a class B network address unless they can show a need for something close to 64K addresses
 - Instead give them an appropriate number of class C addresses
 - For any AS with at least 256 hosts, we can guarantee an address space utilization of at least 50%
- What is the problem with this solution?

Classless Addressing

- Problem with this solution: Excessive storage requirement at the routers
- If a single AS has, say, 16 class C network numbers assigned to it, every Internet backbone router needs 16 entries in its routing tables for that AS, even if the path to every one of these networks is the same
- If we had assigned a class B network to the AS
 - The same routing information can be stored in one entry
 - Efficiency = $16 \times 255 / 65536 = 6.2\%$

Classless Addressing

- **Classless Inter-Domain Routing (CIDR)** tries to balance the desire to minimize the number of routes that a router needs to know against the need to hand out addresses efficiently.
- CIDR uses *aggregate routes*
 - Uses a single entry in the forwarding table to tell the router how to reach a lot of different networks
 - Breaks the rigid boundaries between address classes

Classless Addressing

- Consider an AS with 16 class C network numbers.
- Instead of handing out 16 addresses at random, hand out a block of contiguous class C addresses
- Suppose we assign the class C network numbers from 192.4.16.0 through 192.4.31.0
- Observe that top 20 bits of all the addresses in this range are the same (11000000 00000100 0001)
 - We have created a 20-bit network number (which is in between class B network number and class C number)
- Requires to hand out blocks of class C addresses that share a common prefix

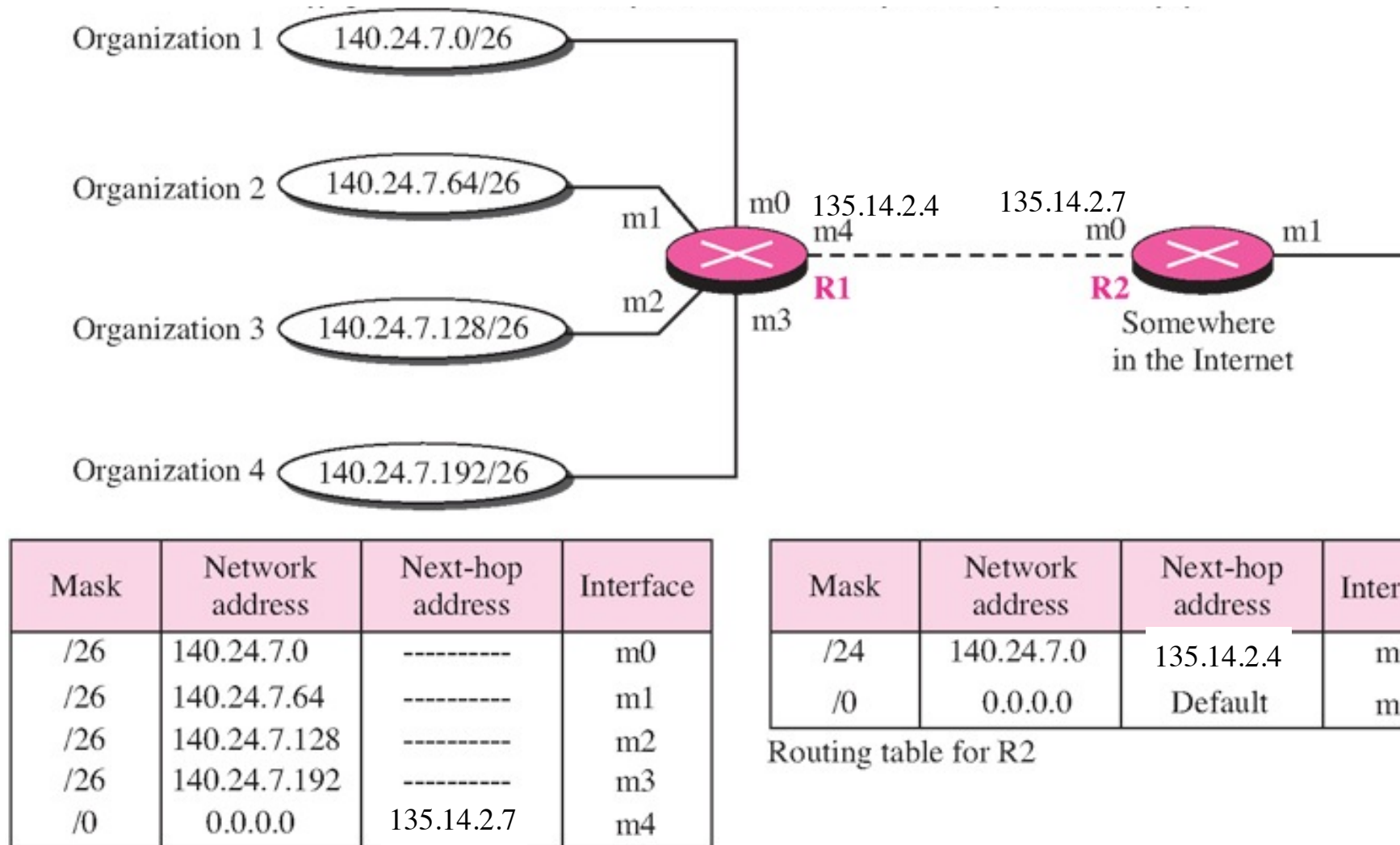
Classless Addressing

- Requires to hand out blocks of class C addresses that share a common prefix
- The convention is to place a /X after the prefix where X is the prefix length in bits
 - For example, the 20-bit prefix for all the networks 192.4.16.0 through 192.4.31.0 is represented as **192.4.16/20**
 - By contrast, if we wanted to represent a single class C network number, which is 24 bits long, we would write it 192.4.16.0/24 (or simply 192.4.16)

Classless Addressing

- How do the routing protocols handle this classless addresses
 - It must understand that the network number may be of any length
- Represent network number with a single pair <length, value>
- All routers must understand CIDR addressing
 - IP forwarding mechanism assumes that it can find the network number in a packet and then look up that number in the forwarding table
 - We need to change this assumption in case of CIDR
 - CIDR means that prefixes may be of any length, from 2 to 32 bits

Classless Addressing: route aggregation

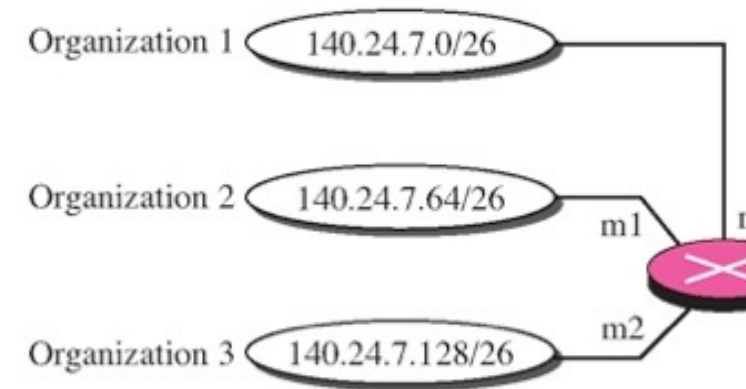


- Route aggregation with CIDR

IP Forwarding Revisited

- It is also possible to have prefixes in the forwarding tables that overlap
 - Some addresses may match more than one prefix
- For example, in the forwarding table of a router we might find 140.24.7.0/24 and 140.24.7.192/26 in this order
 - A packet destined to 140.24.7.200 clearly matches both prefixes.
- Most routers choose following the principle of “longest match” (140.24.7.192/26 in this case)
- Some routers scan sequentially the forwarding table, and apply the first matching rule
 - In this case, a packet destined to 140.24.7.200 would match 140.24.7.0/24 and not 140.24.7.192/26

Longest match forwarding

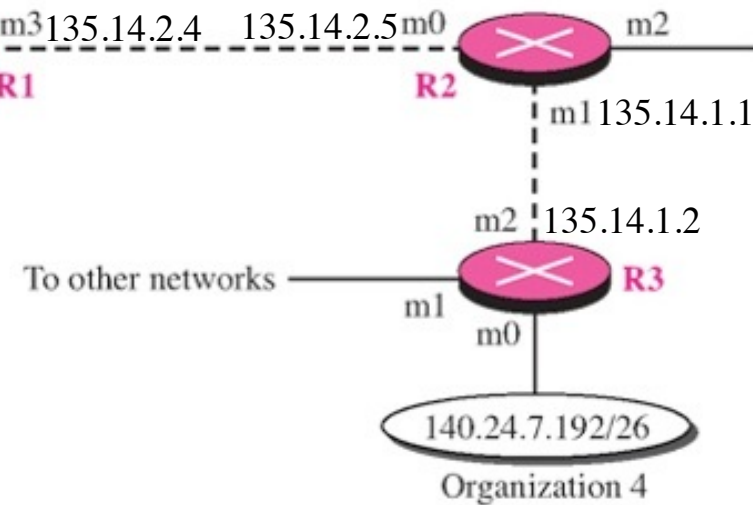


Mask	Network address	Next-hop address	Interface
/26	140.24.7.0	-----	m0
/26	140.24.7.64	-----	m1
/26	140.24.7.128	-----	m2
/0	0.0.0.0	135.14.2.5	m3

Routing table for R1

Routing table for R2

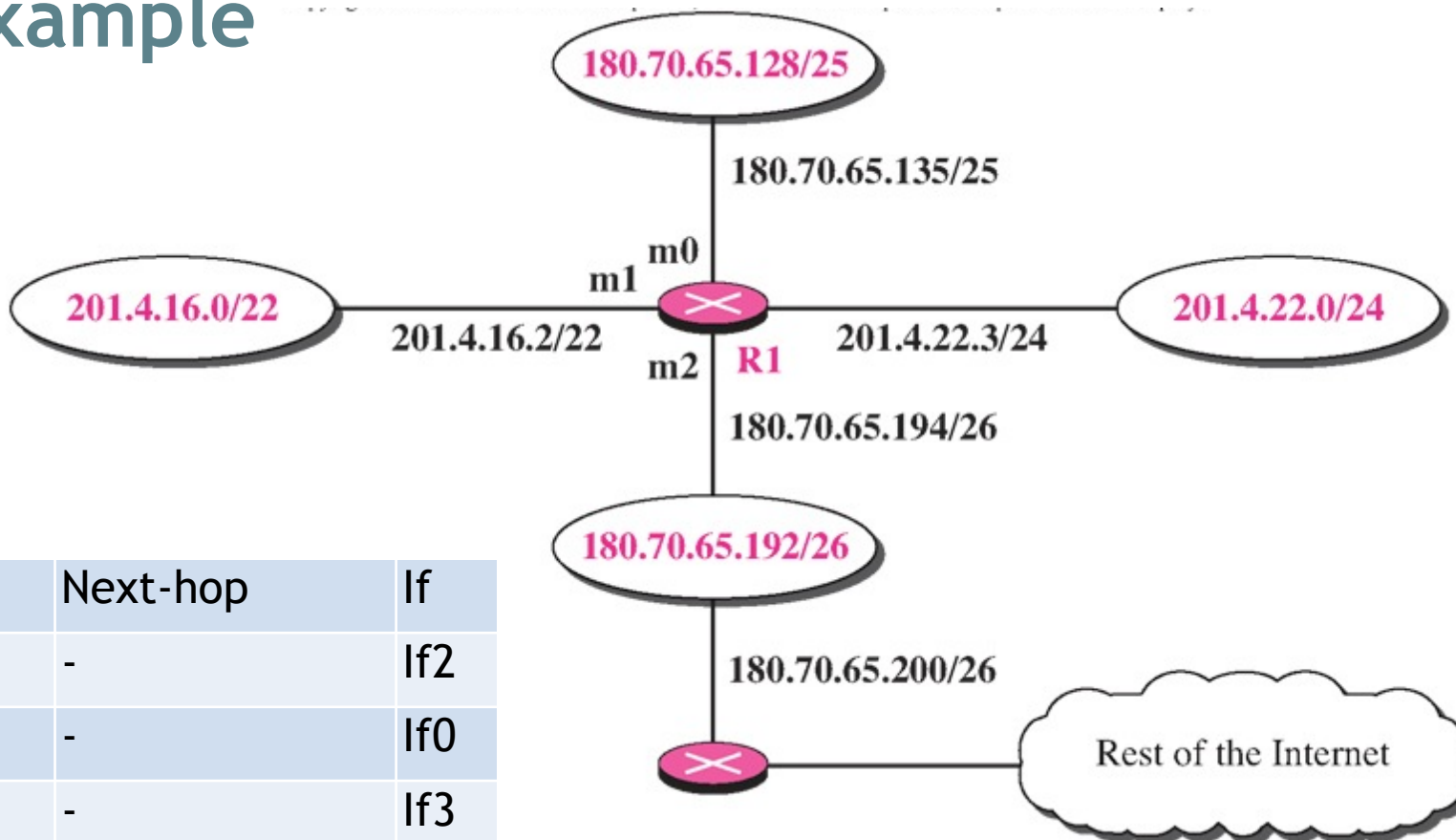
Mask	Network address	Next-hop address	Interface
/26	140.24.7.192	135.14.1.2	m1
/24	140.24.7.0	135.14.2.4	m0
/??	???????	?????????	m1
/0	0.0.0.0	Default	m2



Mask	Network address	Next-hop address	Interface
/26	140.24.7.192	-----	m0
/??	???????	?????????	m1
/0	0.0.0.0	135.14.1.1	m2

Routing table for R3

Another example



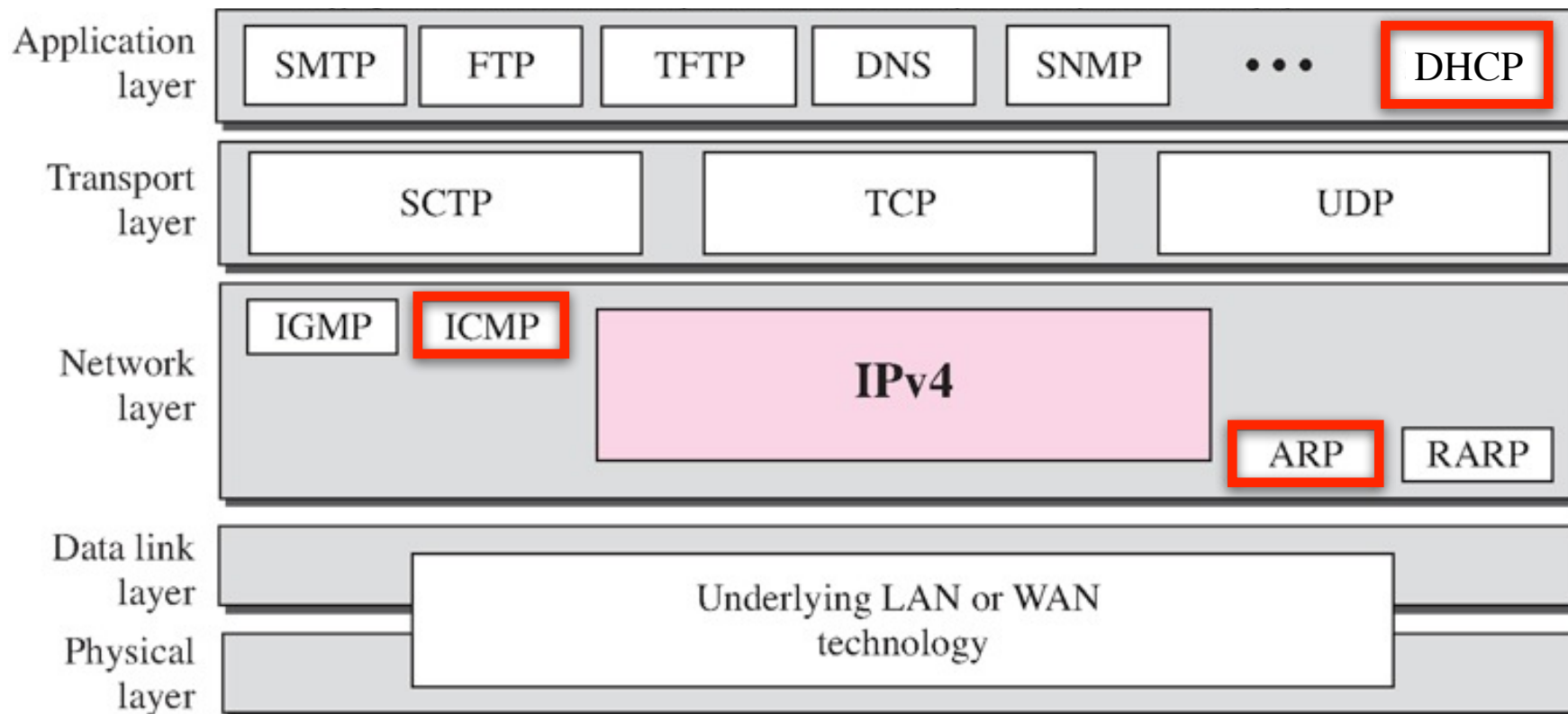
Destination	Next-hop	If
180.70.65.192/26	-	If2
180.70.65.128/25	-	If0
201.4.22.0/24	-	If3
201.4.16.0/22	-	If1
*	180.70.65.200	if2

Special use IP addresses (RFC 5735)

- **0.0.0.0/8**: non-routable, invalid address. Used to denote “any IP address”
- **10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16**: Used for private networks - not routable on the Internet
- **127.0.0.0/8**: Loopback (i.e. ‘myself’)
- **169.254.0.0/16**: autoconfiguration on a local link when DHCP not available
- **x.y.z.y/31**: this subnet has exactly 2 addresses, in theory none for hosts (0 is the net, 1 is the broadcast), but in practice the two addresses can be used as actual host addresses (used only in point-to-point links, RFC 3021)
- **224.0.0.0/4**: Class D: Multicast addresses
- **240.0.0.0/4**: Class E: reserved for future uses (never actually used...)

IP Ancillary protocols

IP Ancillary protocols

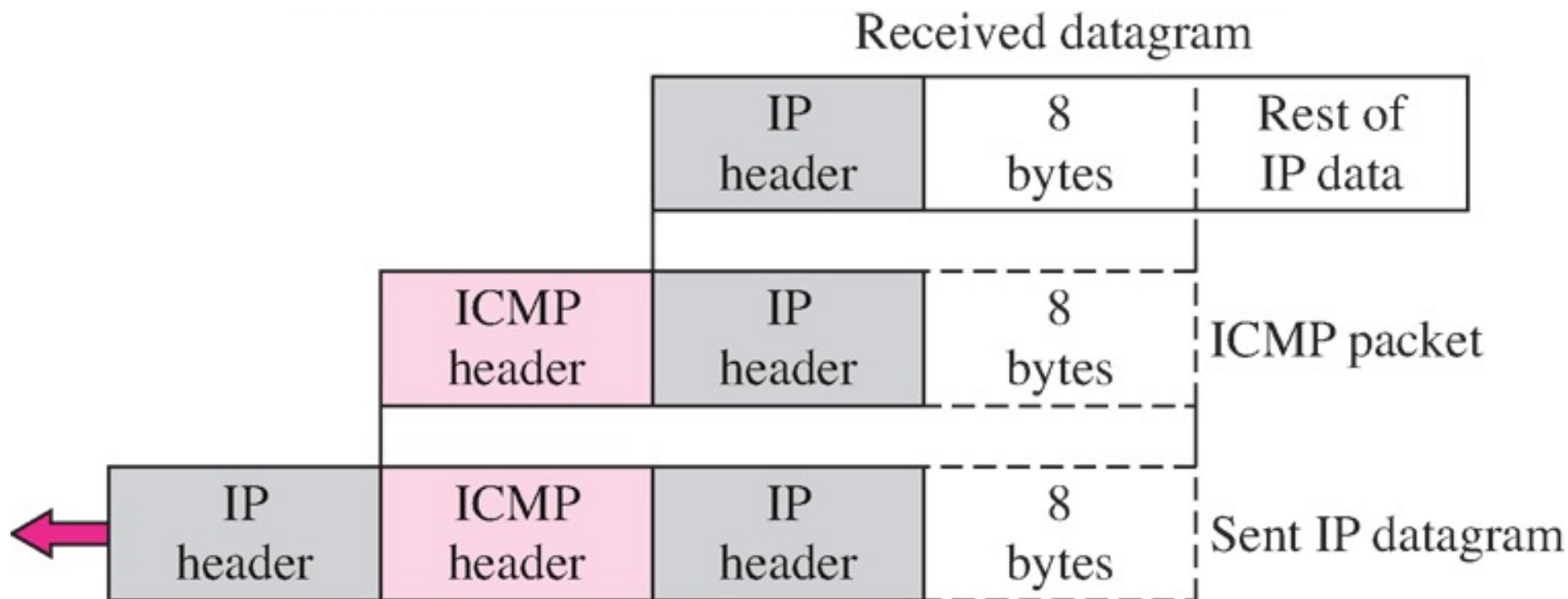


Network signalling and management: Internet Control Message Protocol (ICMP)

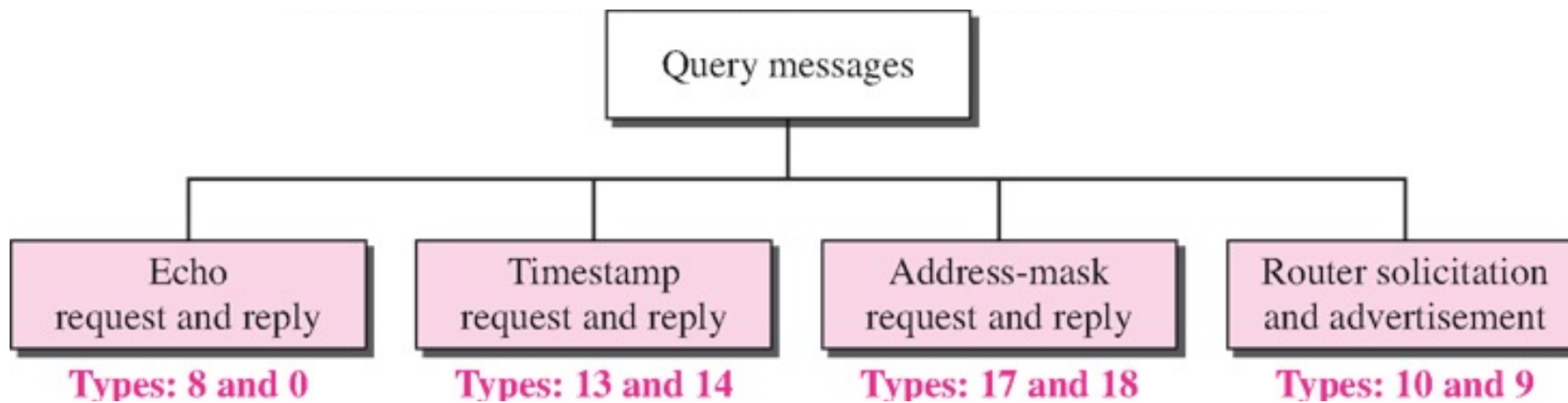
- Defines a collection of error and notification messages that are sent back to the source host
 - whenever a router or host is unable to process an IP datagram successfully
 - Destination host unreachable due to link /node failure
 - Reassembly process failed
 - TTL had reached 0 (and datagram has been discarded)
 - IP header checksum failed
 - when there is better route for that destination
 - From router to the source host (ICMP-Redirect)
- In all cases, only the source host is notified

Internet Control Message Protocol (ICMP)

- The initial part of the rejected datagram is included in the ICMP message, for facilitating analysis and countermeasures



ICMP request messages



- Echo (“ping”)
 - check whether a host is reachable
- Timestamp
 - for measuring round trip time (RTT)
- Mask discovery, router discovery
 - implemented by DHCP, nowadays

Ping

- Sends several ICMP Echo requests
 - computes round trip time

```
└─ ping www.google.com
PING www.google.com (216.58.198.36): 56 data bytes
64 bytes from 216.58.198.36: icmp_seq=0 ttl=54 time=25.934 ms
64 bytes from 216.58.198.36: icmp_seq=1 ttl=54 time=46.239 ms
64 bytes from 216.58.198.36: icmp_seq=2 ttl=54 time=25.844 ms
64 bytes from 216.58.198.36: icmp_seq=3 ttl=54 time=25.701 ms
64 bytes from 216.58.198.36: icmp_seq=4 ttl=54 time=27.354 ms
64 bytes from 216.58.198.36: icmp_seq=5 ttl=54 time=46.450 ms
64 bytes from 216.58.198.36: icmp_seq=6 ttl=54 time=29.715 ms
64 bytes from 216.58.198.36: icmp_seq=7 ttl=54 time=26.358 ms
64 bytes from 216.58.198.36: icmp_seq=8 ttl=54 time=26.206 ms
64 bytes from 216.58.198.36: icmp_seq=9 ttl=54 time=26.120 ms
64 bytes from 216.58.198.36: icmp_seq=10 ttl=54 time=27.245 ms
^C
--- www.google.com ping statistics ---
11 packets transmitted, 11 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 25.701/30.288/46.450/7.646 ms
```

Traceroute

```
prompt> traceroute fhda.edu
traceroute to fhda.edu (153.18.8.1), 30 hops max, 38 byte packets
 1Dcore.fhda.edu          (153.18.31.254)  0.995 ms 0.899 ms 0.878 ms
 2Dbbackup.fhda.edu       (153.18.251.4)   1.039 ms 1.064 ms 1.083 ms
 3tiptoe.fhda.edu         (153.18.8.1)     1.797 ms 1.642 ms 1.757 ms
```

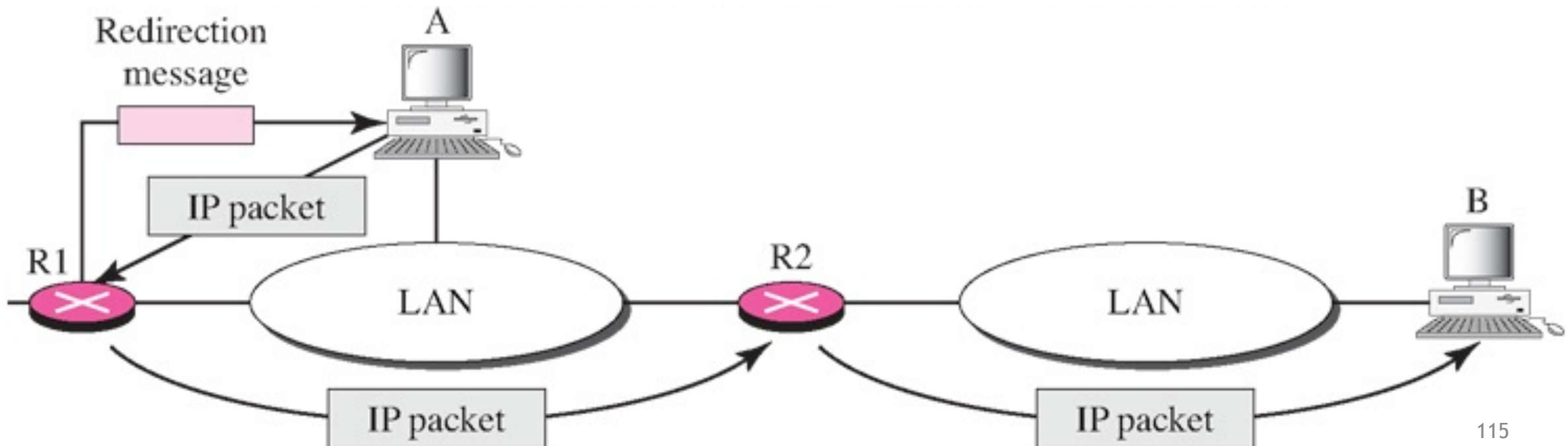
- Useful tool for discovering the route to an host
- Uses ICMP error messages and TTL
 - Sends a message with TTL=1
 - First router drops it, and send the sender an ICMP Error
 - Thus the sender discovers the address of first router
 - Sends a message with TTL=2
 - First router decrements TTL, and forwards it to next router
 - Second router drops it, and send the sender an ICMP Error
 - Thus the sender discovers the address of second router
- ... etc.

Traceroute

- This technique does not work for destination host
 - even if $TTL=0$, the packet has arrived and no ICMP error is sent back
- For the last step, an UDP packet on port 1 is sent
 - port 1 is not valid, hence it generates an ICMP error of “port not reachable”

ICMP-Redirect

- Router R1 receives a datagram for a destination for which there is a better route
- datagram is delivered, but source is notified about the right route
- the host updates its route table accordingly



Address Translation Protocol (ARP)

- Map IP addresses into physical addresses
 - destination host
 - next hop router
- Techniques
 - encode physical address in host part of IP address
 - table-based
 - Can be managed directly only for small networks

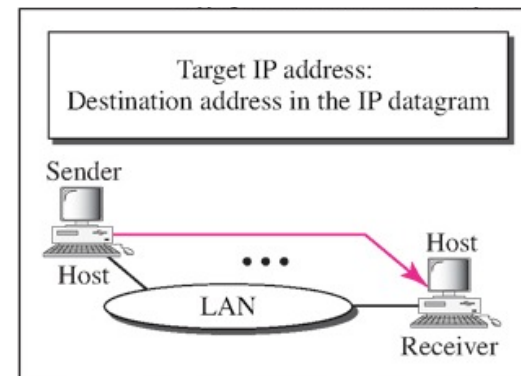
Logic Addr	Physic Address
192.44.80.1	FF:6E:A0:13:6C:A4
192.44.80.2	67:4A:6D:1A:B4:33
192.44.80.3	1A:27:44:8F:C2:14
192.44.80.4	1F:56:AB:C3:12:34
192.44.80.5	F2:65:29:04:72:60
192.44.80.6	2E:70:0E:A2:53:66
...	...

Address Translation Protocol (ARP)

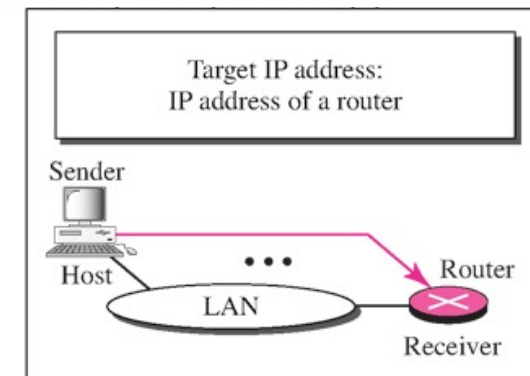
- **ARP (Address Resolution Protocol)**
 - handles automatically the table of IP to physical address bindings
 - broadcast request if IP address not in table
 - target machine responds with its physical address
 - table entries are discarded if not refreshed
 - invoked by IP module, when a datagram has to be delivered to a host of the same network, but the address is not present in ARP table

Address Translation Protocol (ARP)

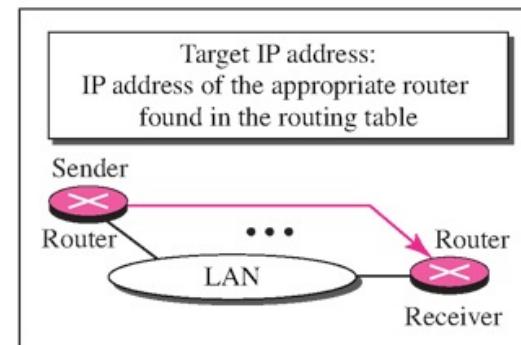
- Which IP address should we resolve?
 - for direct delivery: the destination's
 - for indirect delivery: next hop's IP address
- thus IP datagram is encapsulated in a datalink frame whose MAC address may be not that of destination
- the MAC address changes at each link the packet traverses, while the IP address remains unchanged



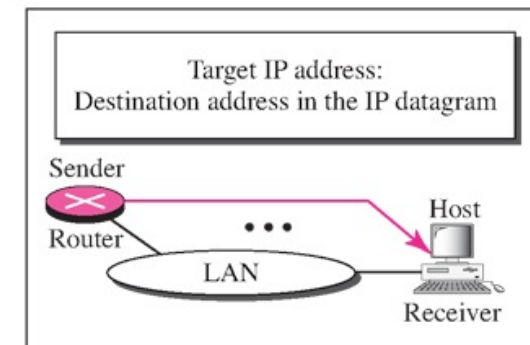
Case 1. A host has a packet to send to another host on the same network.



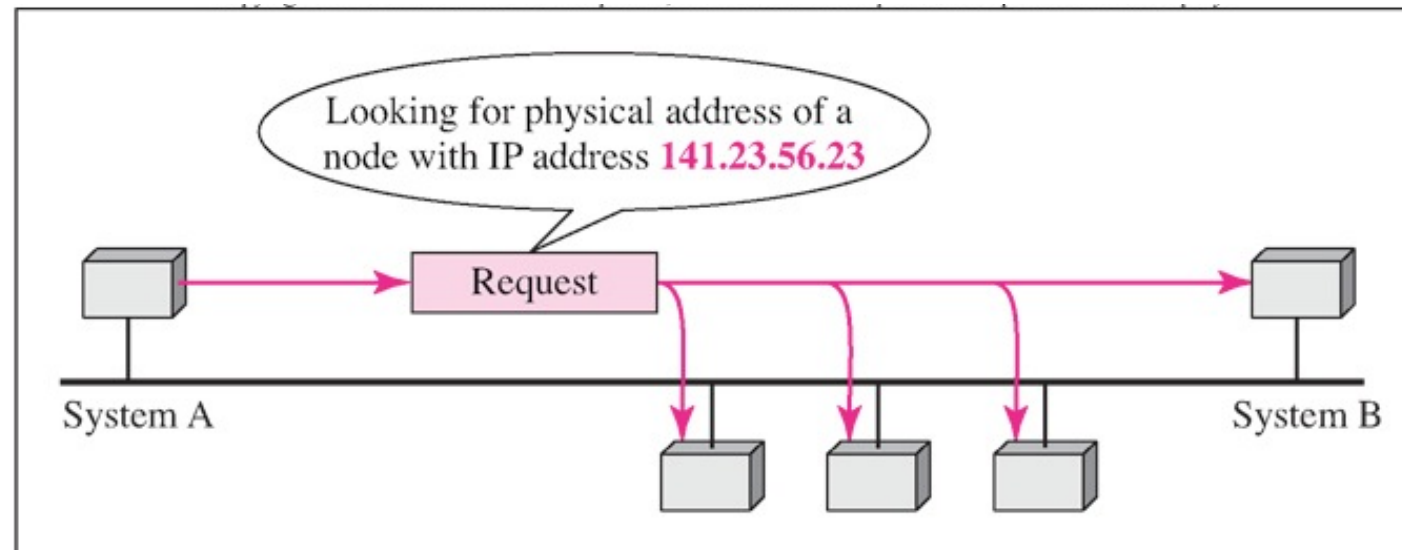
Case 2. A host wants to send a packet to another host on another network. It must first be delivered to a router.



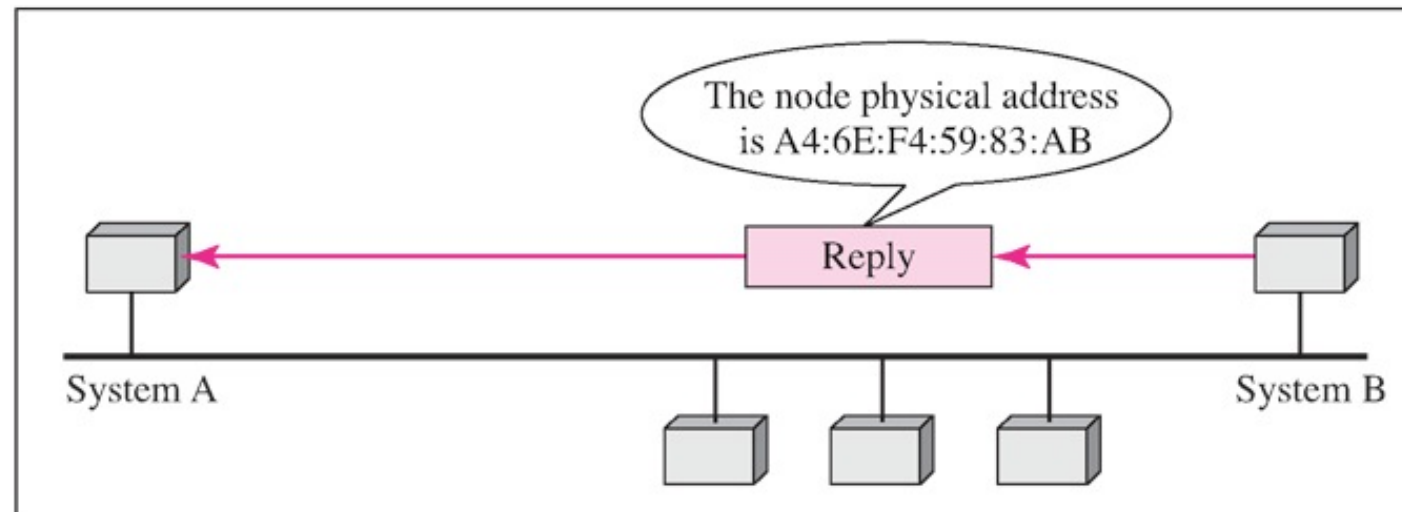
Case 3. A router receives a packet to be sent to a host on another network. It must first be delivered to the appropriate router.



Case 4. A router receives a packet to be sent to a host on the same network.



a. ARP request is broadcast



b. ARP reply is unicast

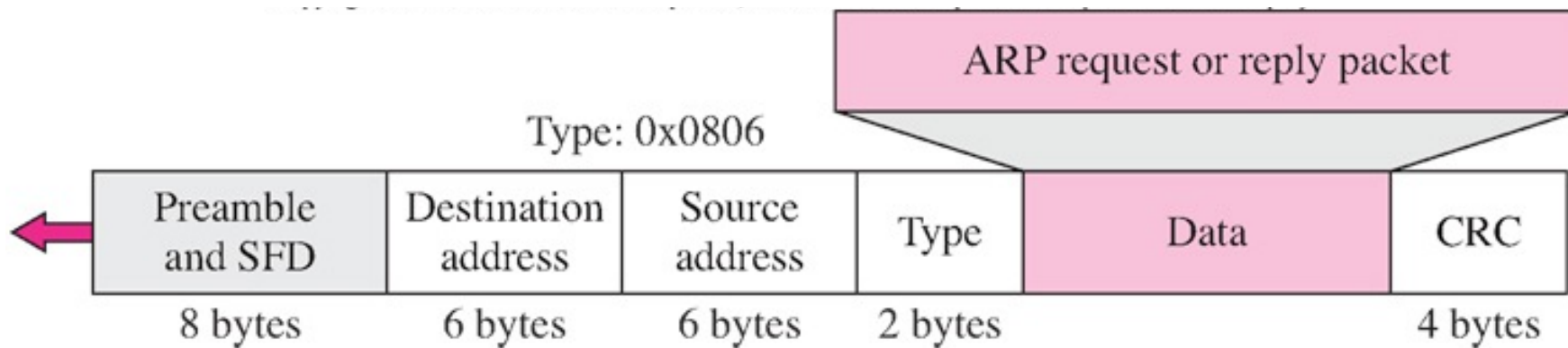
ARP Packet Format

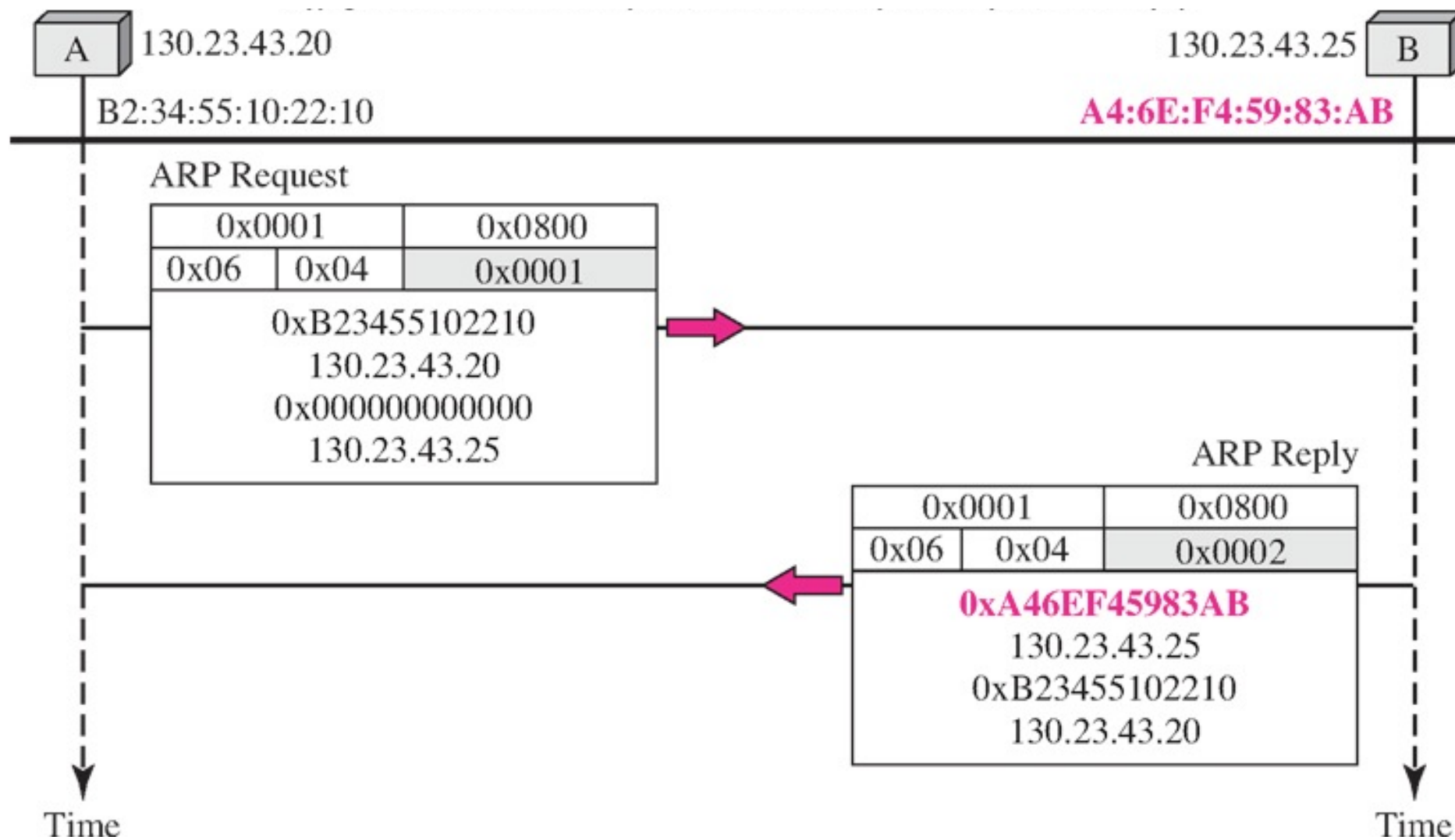
- HardwareType: type of physical network (e.g., Ethernet)
- ProtocolType: type of higher layer protocol (e.g., IP)
- HLEN & PLEN: length of physical and protocol addresses
- Operation: request or response
- Source/Target Physical/Protocol addresses
- TargetHardwareAddr is empty in request

0	8	16	31
Hardware type = 1		ProtocolType = 0x0800	
HLen = 48	PLen = 32		Operation
SourceHardwareAddr (bytes 0–3)			
SourceHardwareAddr (bytes 4–5)		SourceProtocolAddr (bytes 0–1)	
SourceProtocolAddr (bytes 2–3)		TargetHardwareAddr (bytes 0–1)	
TargetHardwareAddr (bytes 2–5)			
TargetProtocolAddr (bytes 0–3)			

Address Translation Protocol (ARP)

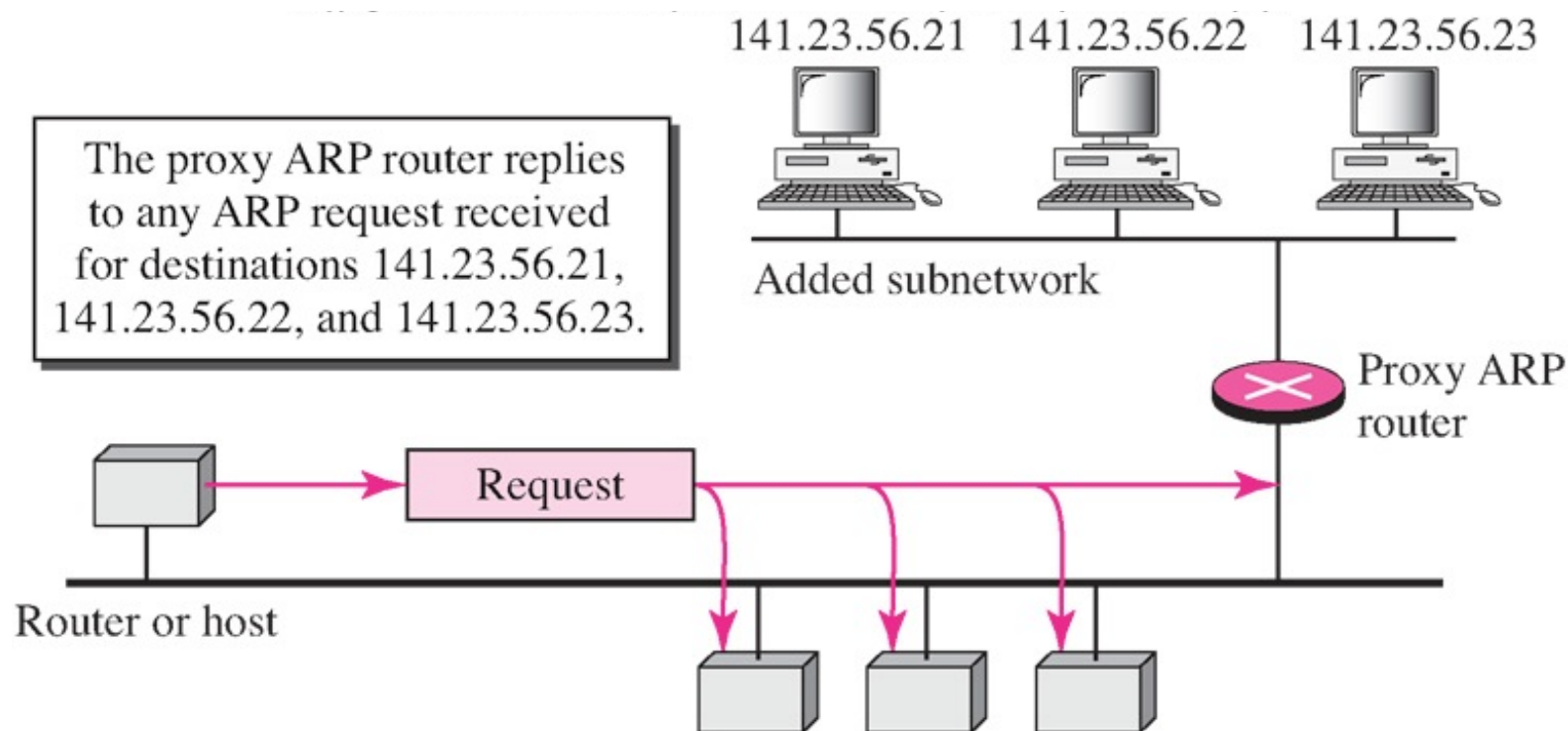
- ARP packets do not use IP (obviously)
- are directly encapsulated in datalink (e.g. Ethernet) frames





Proxy ARP

- A proxy ARP is a host representing other hosts (often entire networks)
- it answers with its own MAC to ARP requests for the “hidden” hosts
- it forwards the frames it receives to the proper destinations, like a switch (but changing MAC address)

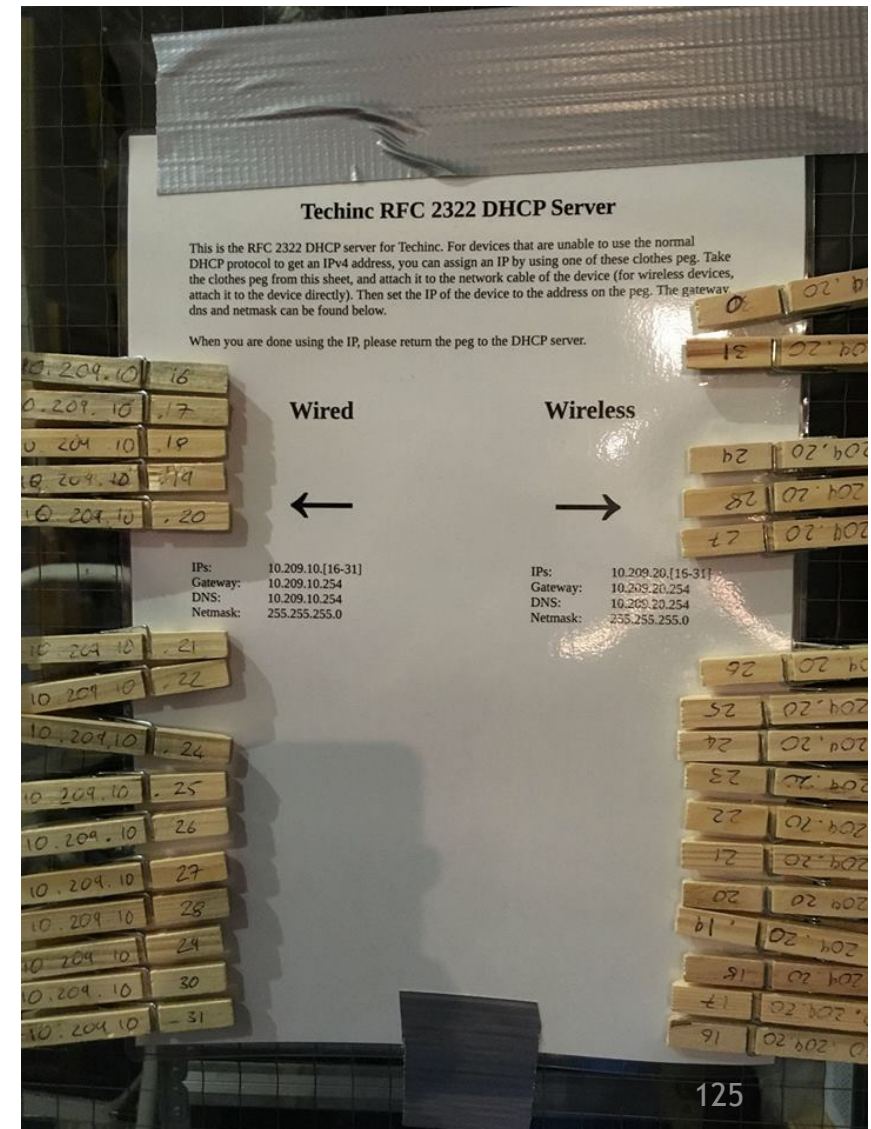


Host Configurations

- Ethernet addresses are configured into network by manufacturer and they are unique
- IP addresses must be unique on a given internetwork but also must reflect the structure of the internetwork
- Most host Operating Systems provide a way to manually configure the IP information for the host
- Drawbacks of manual configuration
 - A lot of work to configure all the hosts in a large network
 - Configuration process is error-prone
- Automated Configuration Process is required

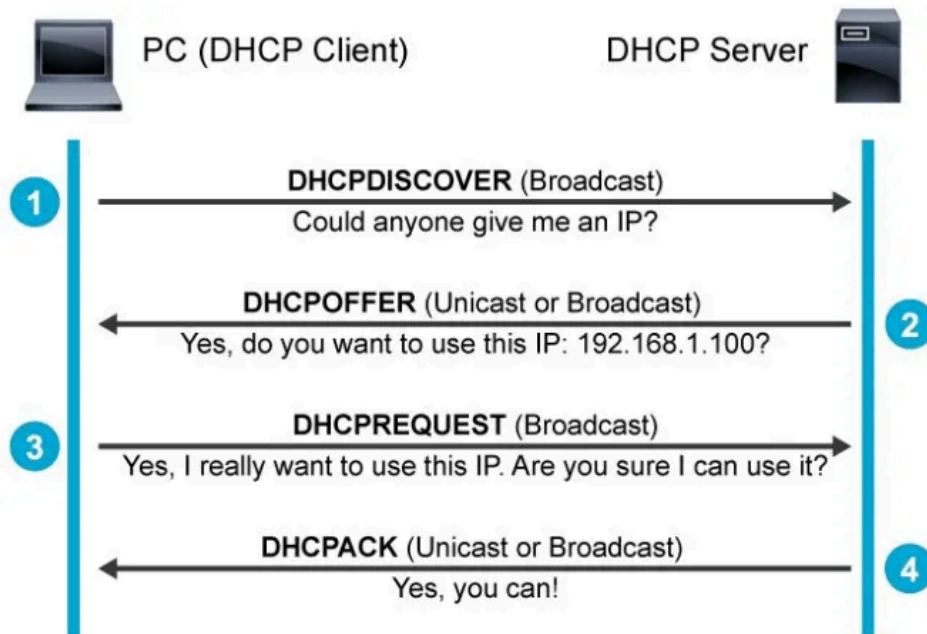
Dynamic Host Configuration Protocol (DHCP)

- DHCP server is responsible for providing configuration information to hosts
 - Other solutions (obsoleted): RARP, BOOTP
- For automatic configuration, at least one DHCP server for an administrative domain
- DHCP server maintains a pool of available addresses, which are assigned to hosts
 - statically (e.g. based on MAC addresses)
 - dynamically (time-limited “leases”)



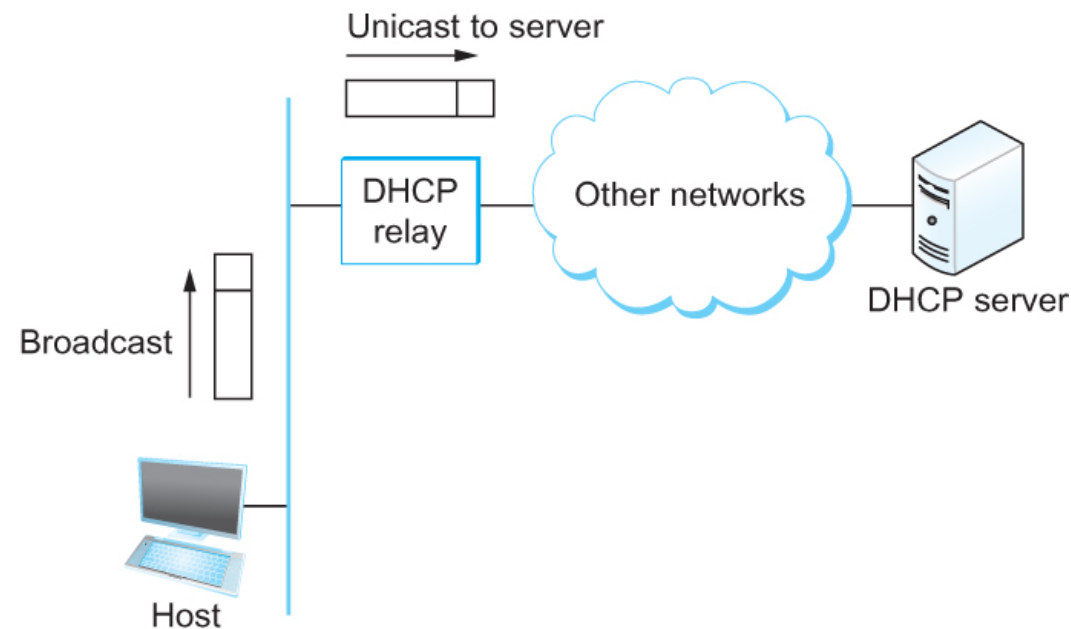
DHCP

- Newly booted or attached host sends DHCP DISCOVER message
 - destination IP: 255.255.255.255 (“limited broadcast”, ignored by routers)
 - source IP: irrelevant. Message contains also MAC address of source host
- DHCP server’s answer contains the offered IP address
 - this message is broadcasted, or delivered to the source host only (using its MAC, not its IP - it has no IP yet)
- The client ask the server for confirmation
- If confirmed, the DHCP client sets the local IP address (and other parameters, like gateway, DNS, etc.) on the host



DHCP

- A DHCP server can serve several networks at once, even if not directly connected
- A *DHCP relay agent* is needed for each network, in order to listen for DHCP requests
- DHCP relay agent do not assign addresses; they simply unicast the messages to DHCP server and waits for the response, to be sent in the local networks

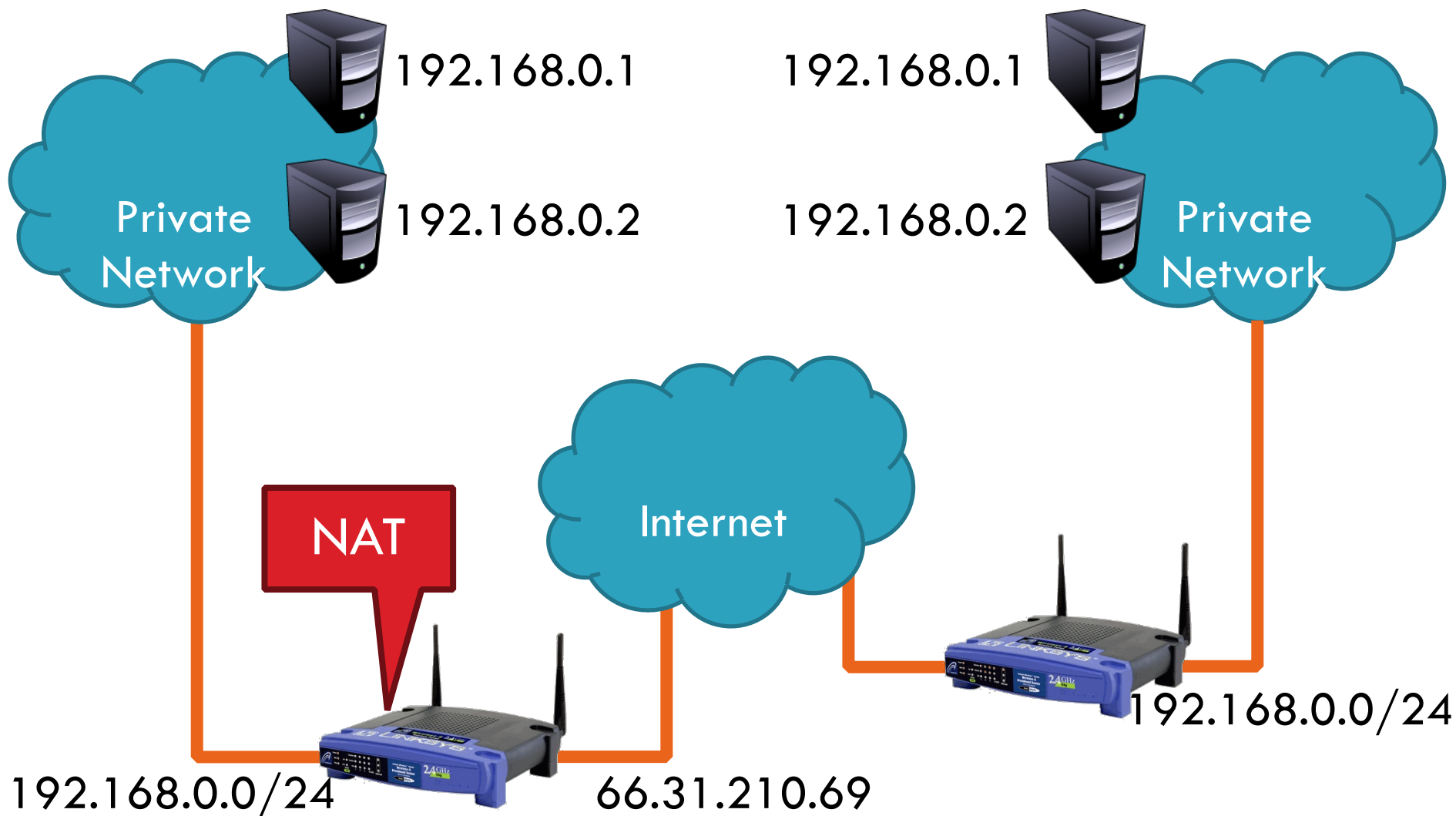


Private IP Networks and NAT

Private IP Networks and NAT

- Idea: create a range of private IPs that are separate from the rest of the network
 - Use the private IPs for internal routing
 - Use a special router to bridge the LAN and the WAN
- Properties of private IPs
 - **Not globally unique**
 - Usually taken from non-routable IP ranges (why?)
- Typical private IP ranges
 - 10.0.0.0/8, that is, 10.0.0.1 - 10.255.255.255
 - 172.16.0.0/12, that is, 172.16.0.1 - 172.31.255.255
 - 192.168.0.0/16, that is, 192.168.0.1 - 192.168.255.255

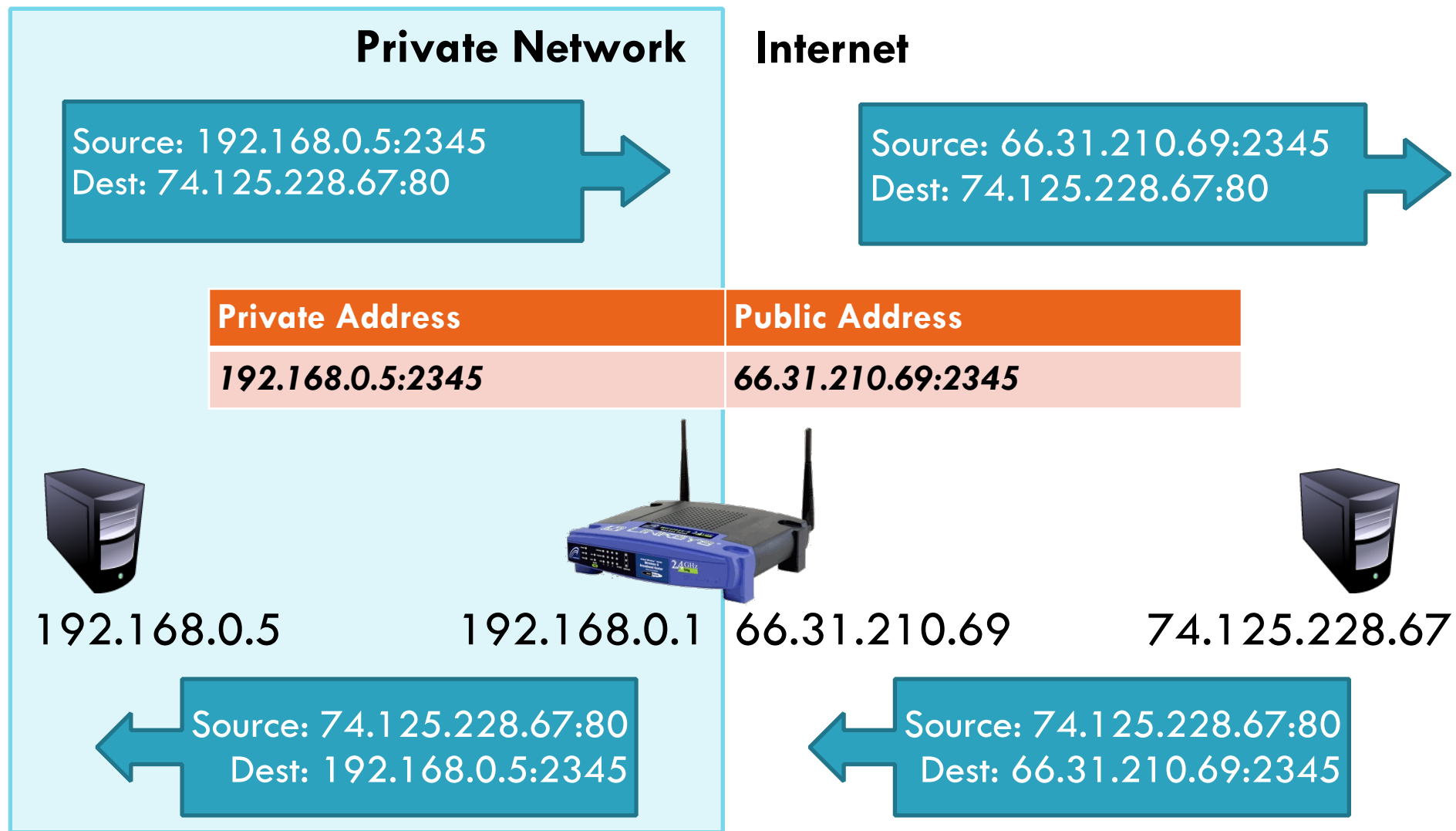
Private Networks



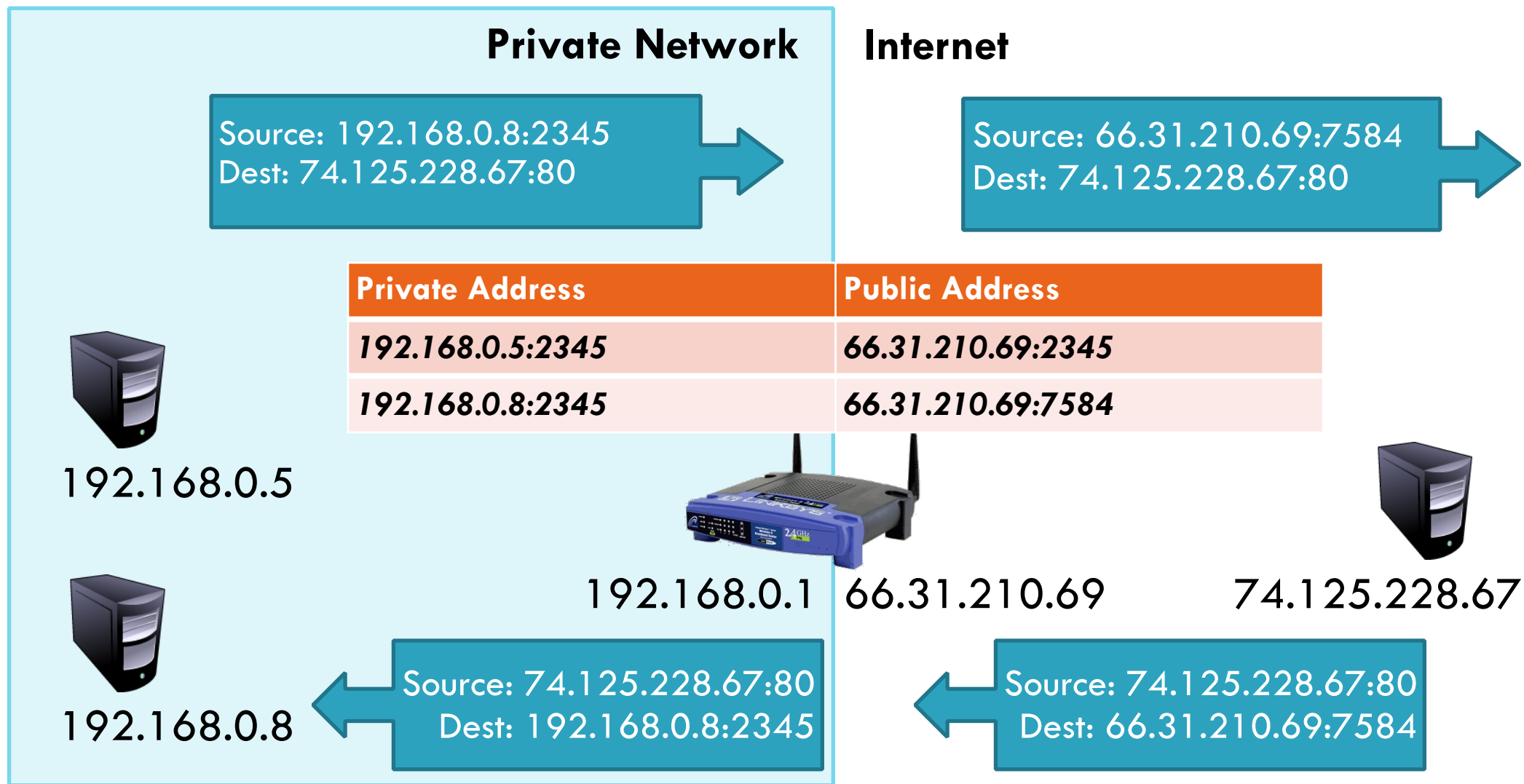
Network Address Translation (NAT) with Port Mapping

- NAT allows hosts on a private network to communicate with the Internet
 - Warning: connectivity is not seamless
- Special router at the boundary of a private network replaces internal IPs with external IP: **“Network Address Translation”**
 - May also replace TCP/UDP port numbers: **“Port Mapping”**
- So the header of each IP packet is modified by the NAT
- The router has to maintain a table of active flows
 - Outgoing packets initialize a table entry
 - Incoming packets are rewritten based on the table

Basic NAT Operation (“full cone”)

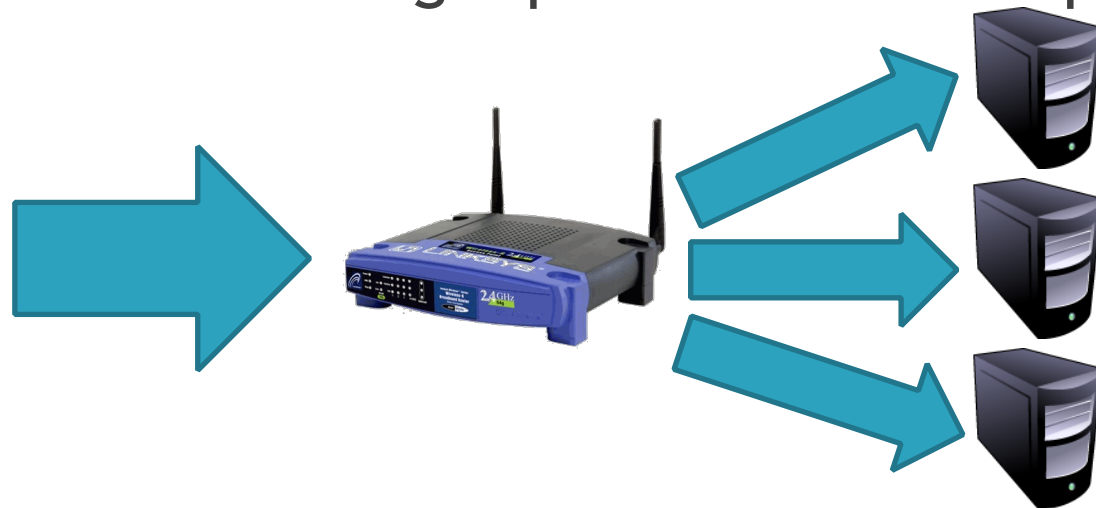


Basic NAT Operation (“full cone” with PAT)

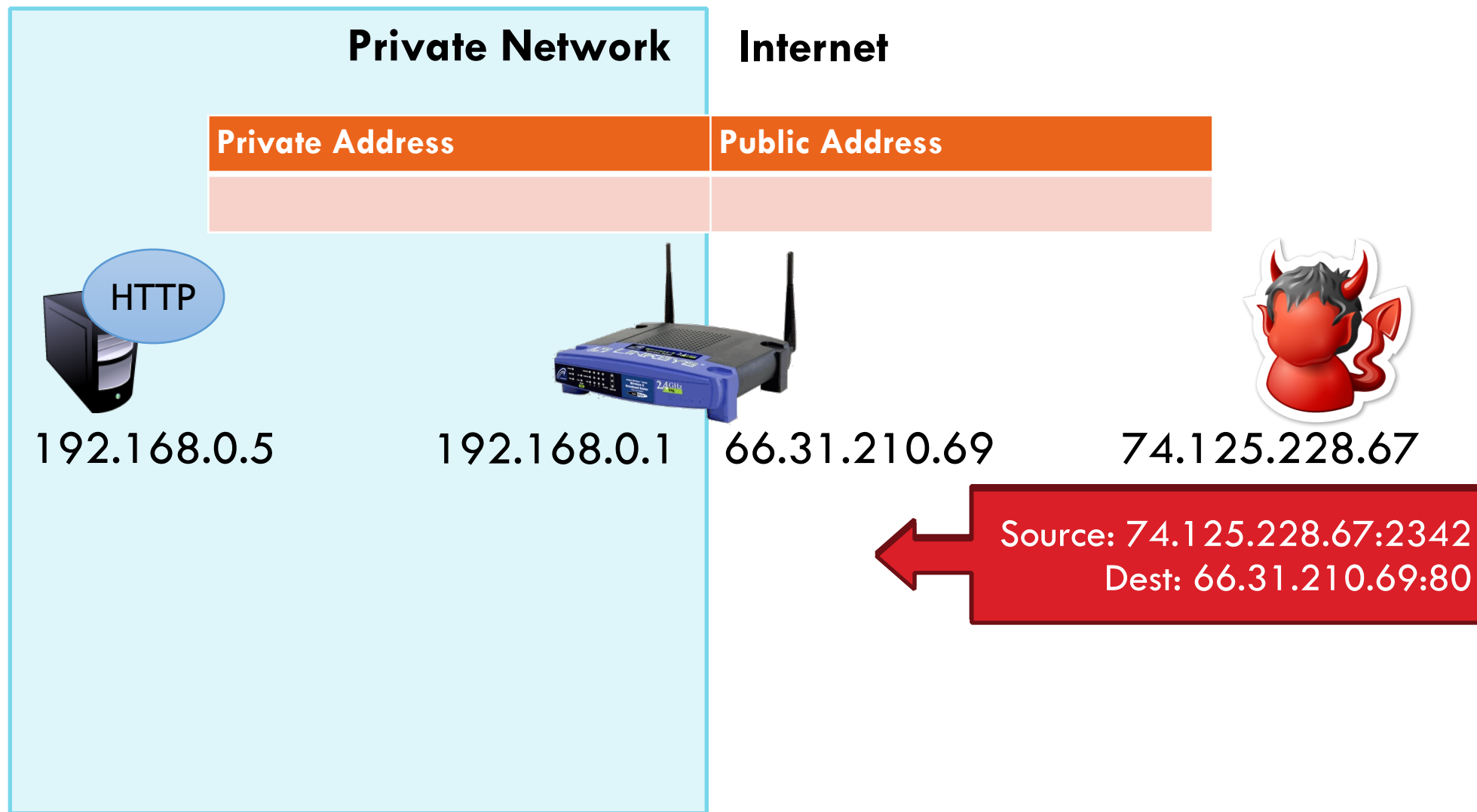


Applications of NATs

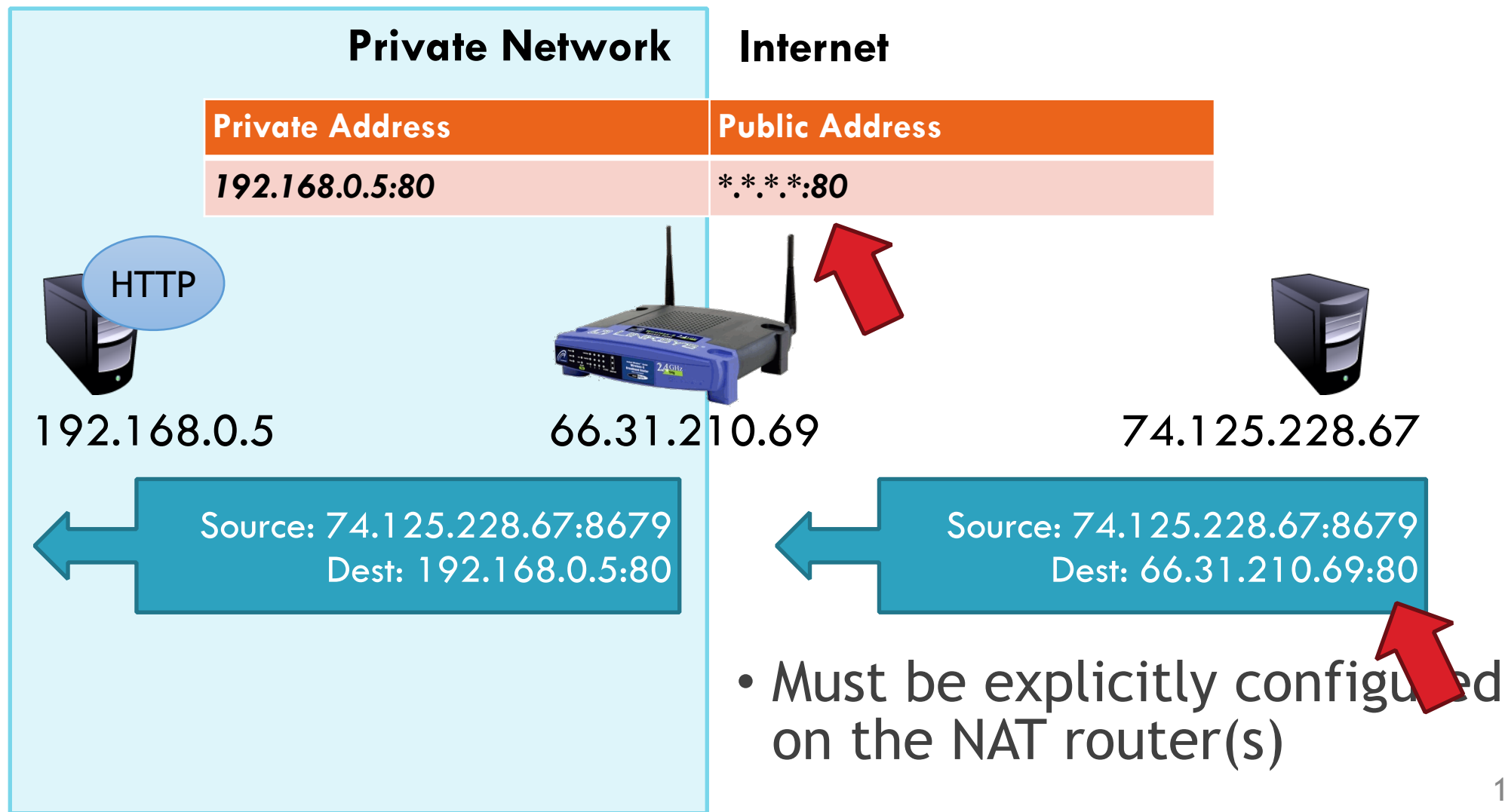
- Allows sharing of one public IP among several private addresses
- Allow migration between ISPs
 - Even if the public IP address changes, you don't need to reconfigure the machines on the LAN
 - Only the router's public IP address changes
- Load balancing
 - Forward traffic from a single public IP to multiple private hosts



Natural “Firewall”

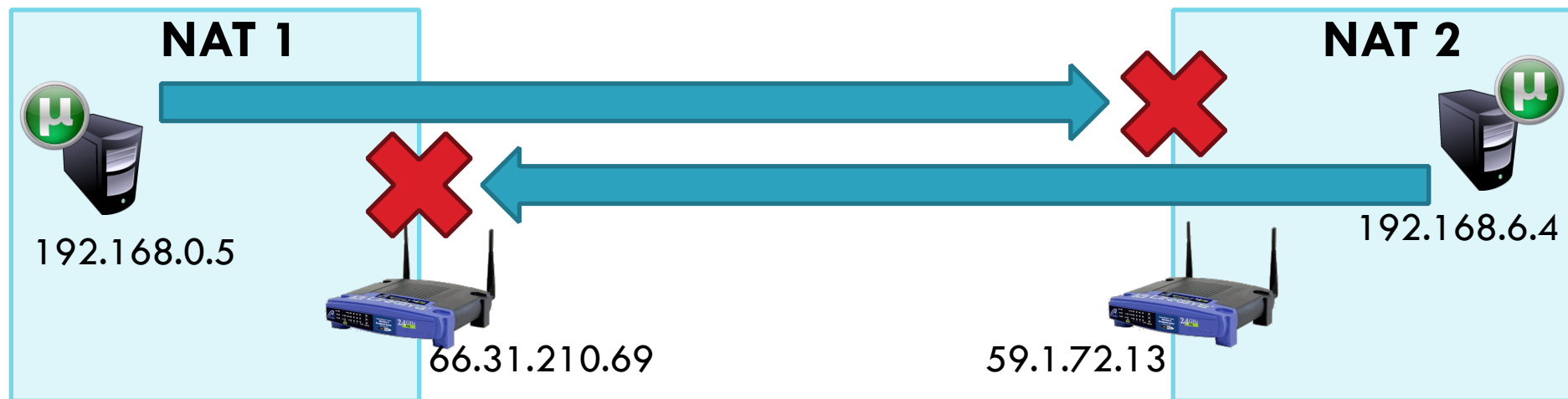


Port Forwarding



Hole Punching

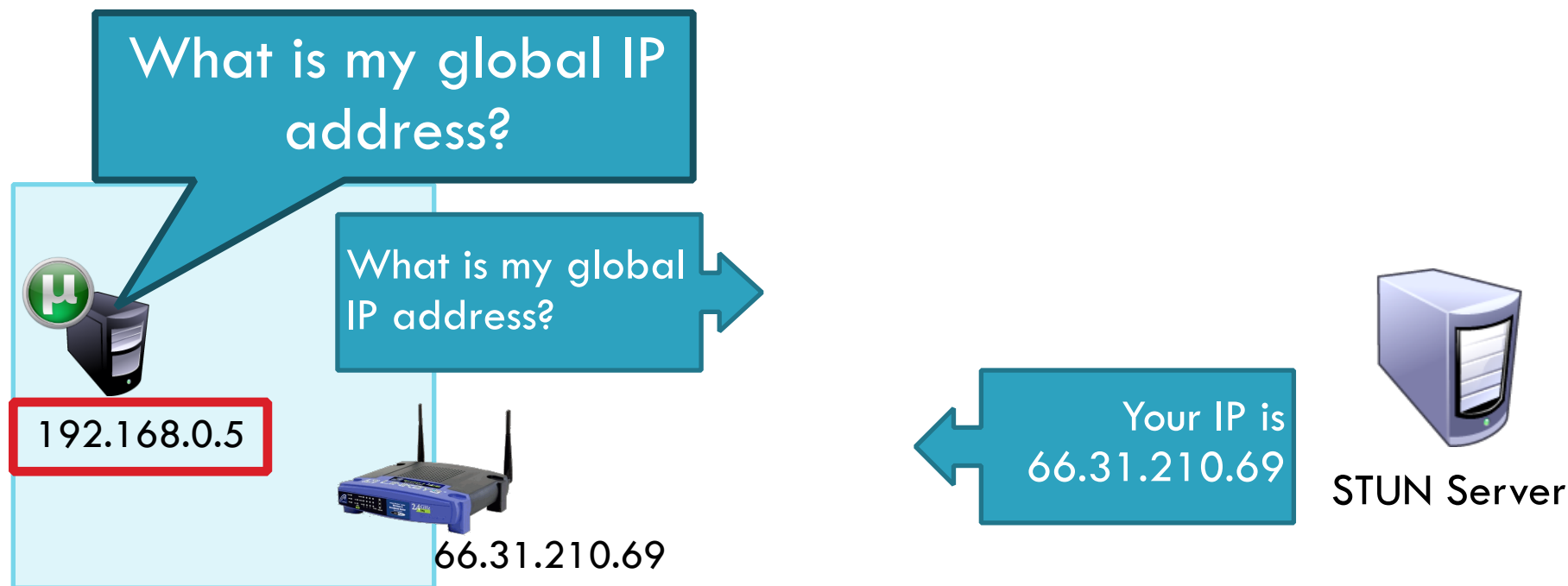
- Problem: How to enable connectivity through NATs?



- Two application-level protocols for hole punching
 - ▣ STUN
 - ▣ TURN

STUN

- **Session Traversal Utilities for NAT**
 - Use a third-party to echo your global IP address
 - Also used to probe for symmetric NATs/firewalls
 - i.e. are external ports open or closed?



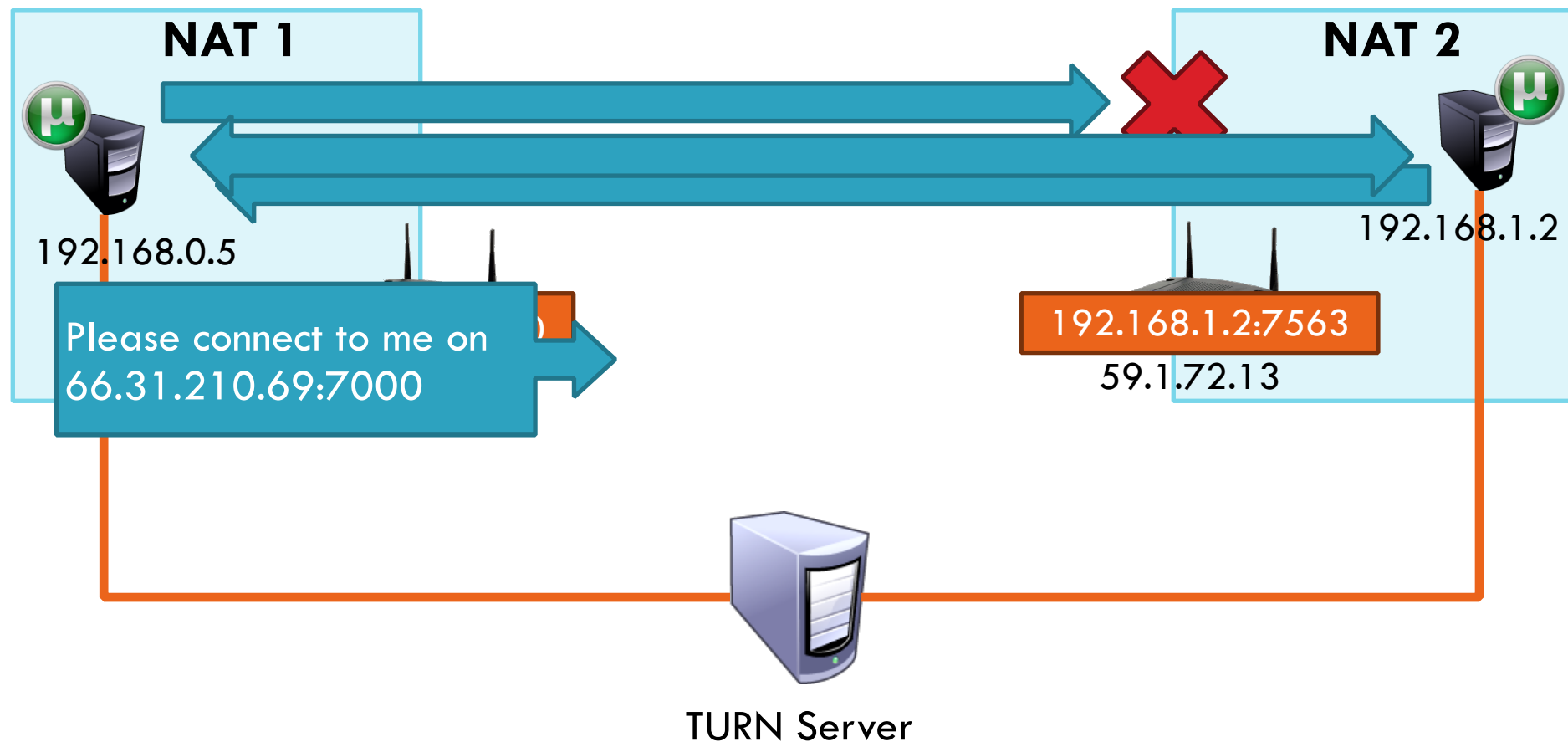
Problems With STUN

- Only useful in certain situations
 - Only one peer is behind a symmetric NAT, the other is public or with port forwarding
 - Both peers are behind partial NATs
- Not useful when both peers are fully behind full NATs: knowing the respective public addresses is not enough for establishing connection, because NAT tables are empty



TURN

- Traversal Using Relays around NAT



Concerns About NAT

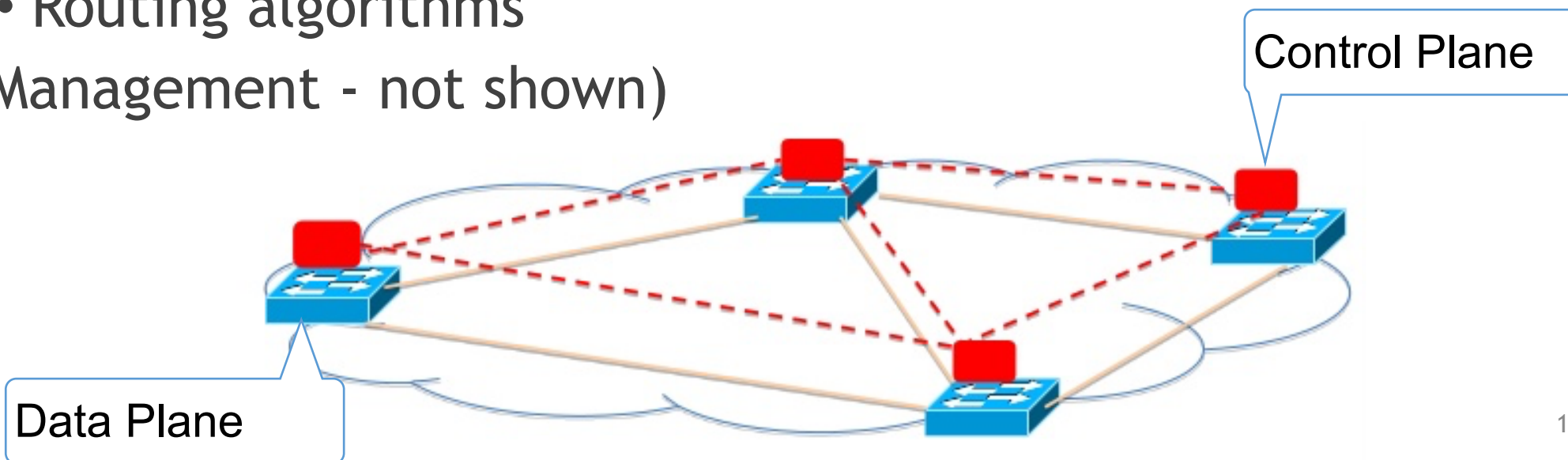
- Performance/scalability issues
 - Per flow state!
 - Modifying IP and Port numbers means NAT must recompute IP and TCP checksums
 - Extra traffic for STUN, TURN
- Breaks the layered network abstraction
- Breaks end-to-end Internet connectivity
 - 192.168.*.* addresses are private
 - Cannot be routed to on the Internet
 - Problem is worse when **both** hosts are behind NATs, or in nested NATs!
- Overall, NAT is a “necessary evil”
 - Almost all networks established in the last 20 years are behind one or more NATs
 - It would not be needed if we had enough public IP address
 - Network separation can be obtained using firewalls



Routing

ROUTING

- Data flow is controlled by switches and routers and contains the following basic elements:
 - Data Forwarding Plane
 - algorithms for packet forwarding
 - **Control Plane**
 - Routing algorithms
 - (Management - not shown)



Forwarding versus Routing Algorithms

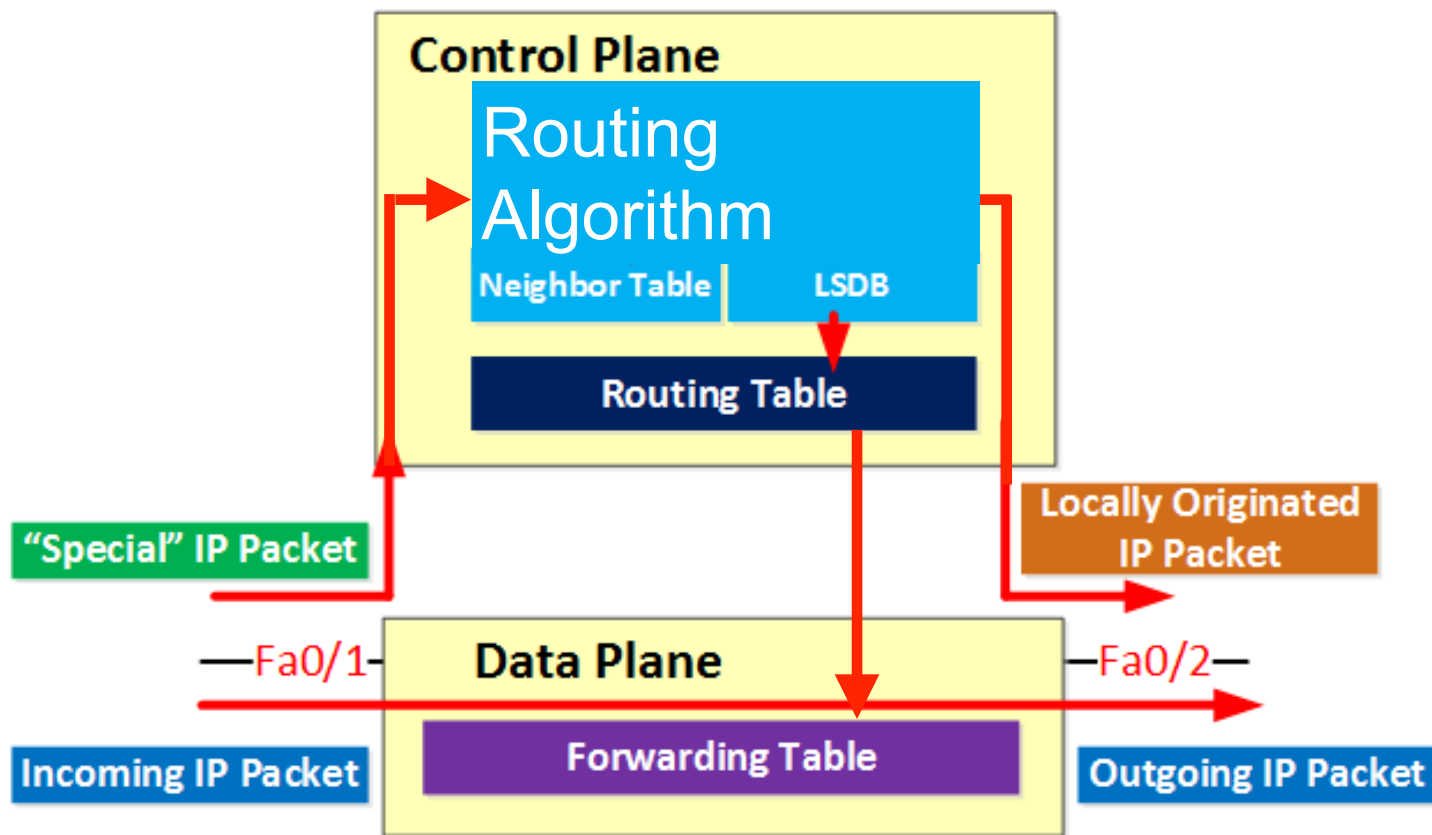
- Forwarding:
 - to select an output port for each packet, based on destination address and forwarding table
 - must be simple and fast
 - possibly implemented in hardware
- Routing:
 - process by which routing table is built
 - routing table is a precursor of forwarding table
 - run as a background process (in user space)
 - can be complex

Forwarding table VS Routing table

- Forwarding table
 - Used when a packet is being forwarded and so must contain enough information to accomplish the forwarding function
 - A row in the forwarding table contains the mapping from a network number to an outgoing interface and some MAC information, such as Ethernet Address of the next hop
- Routing table
 - Built by the routing algorithm as a precursor to build the forwarding table
 - Generally contains mapping from network numbers to next hops

Forwarding table VS Routing table

- Forwarding table is obtained from routing table
- Routing table is built by routing algorithms using informations received from other nodes



Forwarding table VS Routing table

(a) routing table: used by control plane, contains level 3 data, with infos about costs, distances, etc.

(a)

Prefix/CIDR	Cost	Next Hop
18/8	3	171.69.245.10
151.85/16	4	195.84.38.5

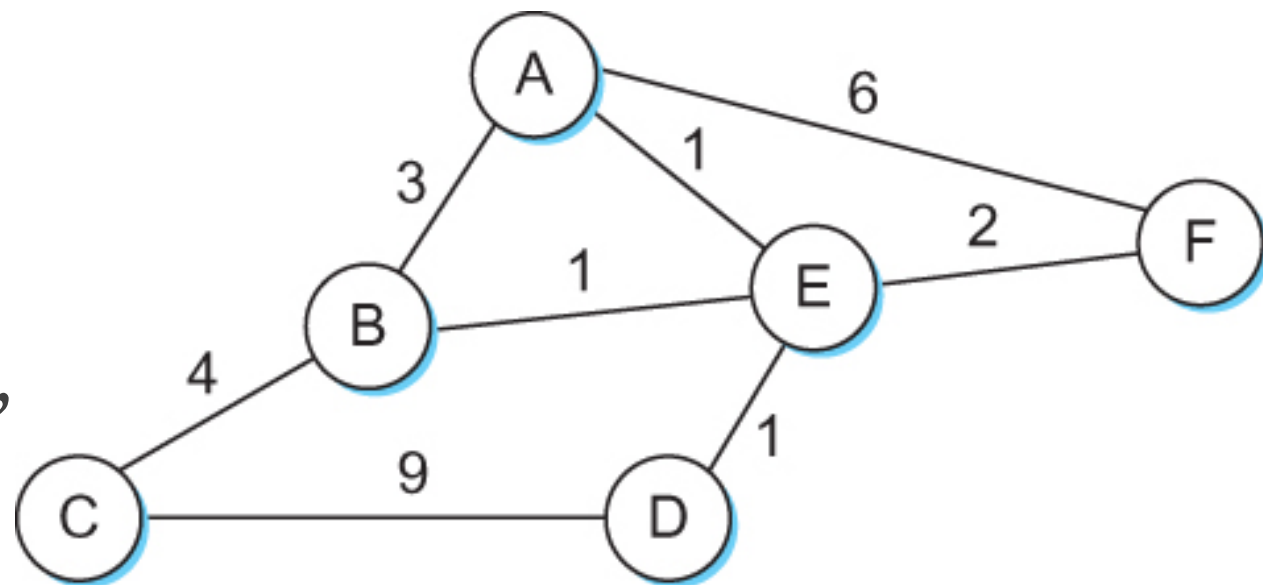
(b) forwarding table: used by data plane, so it contains data for forwarding packets: which interface, address of next hop, etc.

(b)

Prefix/CIDR	Interface	Next Hop
18/8	if0	84:3f:96:43:ad:6c
151.85/16	if2	6a:5c:04:22:5d:90

Routing

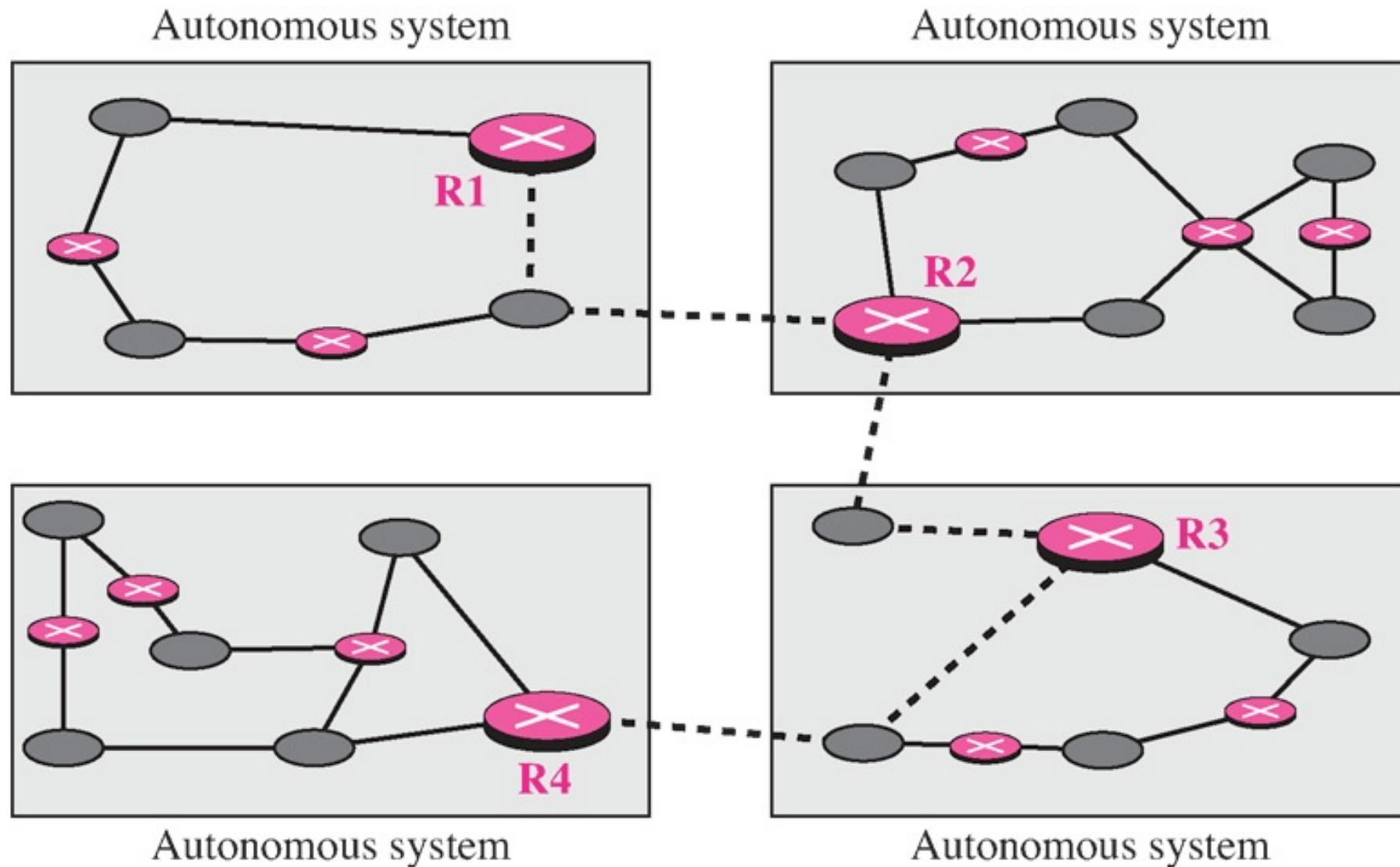
- Network as a Graph
 - nodes = routers connected to (basic) networks
 - links = connections
 - costs = speed, latency, costs, loss rate... many options
- The basic problem of routing is to **find the lowest-cost path between any two nodes**
 - Where the cost of a path equals the sum of the costs of all the edges that make up the path



Routing algorithms

- Difficult to have a unique algorithm for the whole Internet
 - Scalability issues
 - Different administrative domains may choose different routing policies
- Better separate in two levels
 - inside *Autonomous Systems*
 - each admin is responsible of routing algorithms
 - not too large networks => simpler algorithms (called *intra-domain*, or *interior* protocols)
 - among Autonomous Systems
 - **reachability** issues rather than efficiency
 - more complex algorithms (called *inter-domain*, or *exterior* protocols)

Routing algorithms



Routing

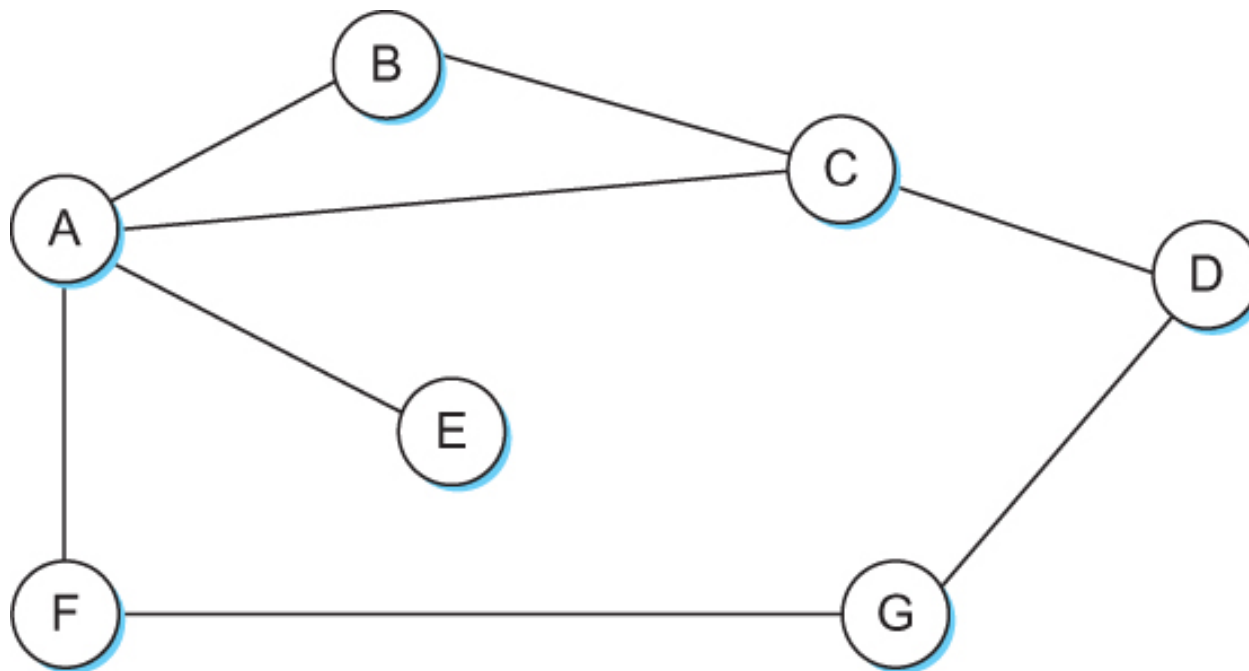
- For a simple network, we can calculate all shortest paths and load them into some nonvolatile storage on each node.
- Such a static approach has several shortcomings
 - It does not deal with node or link failures
 - It does not consider the addition of new nodes or links
 - It implies that edge costs cannot change

Routing

- Solution: a **distributed** and **dynamic** protocol for constructing routing tables at each
- Three main classes of protocols
 - **Distance Vector**
 - Used in interior protocols like RIP
 - **Link State**
 - Used in interior protocols like OSPF and IS-IS
 - **Path vector**
 - Used in exterior protocols like BGP - next chapter

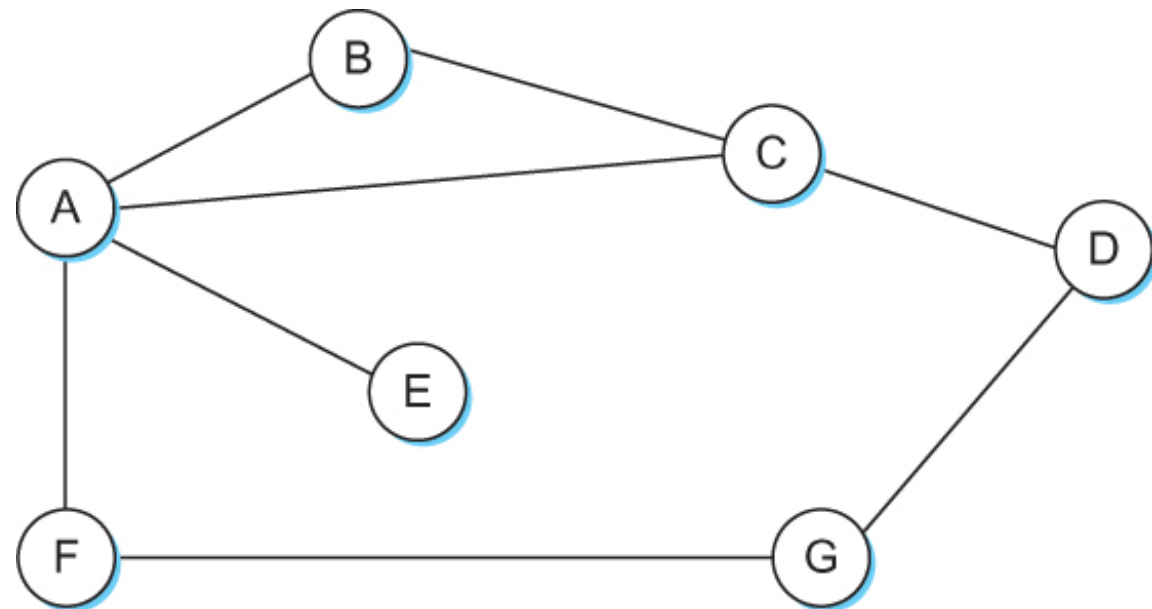
Distance Vector

- Starting assumption is that each node knows the cost of the link to each of its directly connected neighbors
- Each node constructs a one dimensional array (a vector) containing the “distances” (costs) to all other nodes and distributes that vector to its immediate neighbors



Distance Vector

Information Stored at Node	Distance to Reach Node						
	A	B	C	D	E	F	G
A	0	1	1	∞	1	1	∞
B	1	0	1	∞	∞	∞	∞
C	1	1	0	1	∞	∞	∞
D	∞	∞	1	0	∞	∞	1
E	1	∞	∞	∞	0	∞	∞
F	1	∞	∞	∞	∞	0	1
G	∞	∞	∞	1	∞	1	0

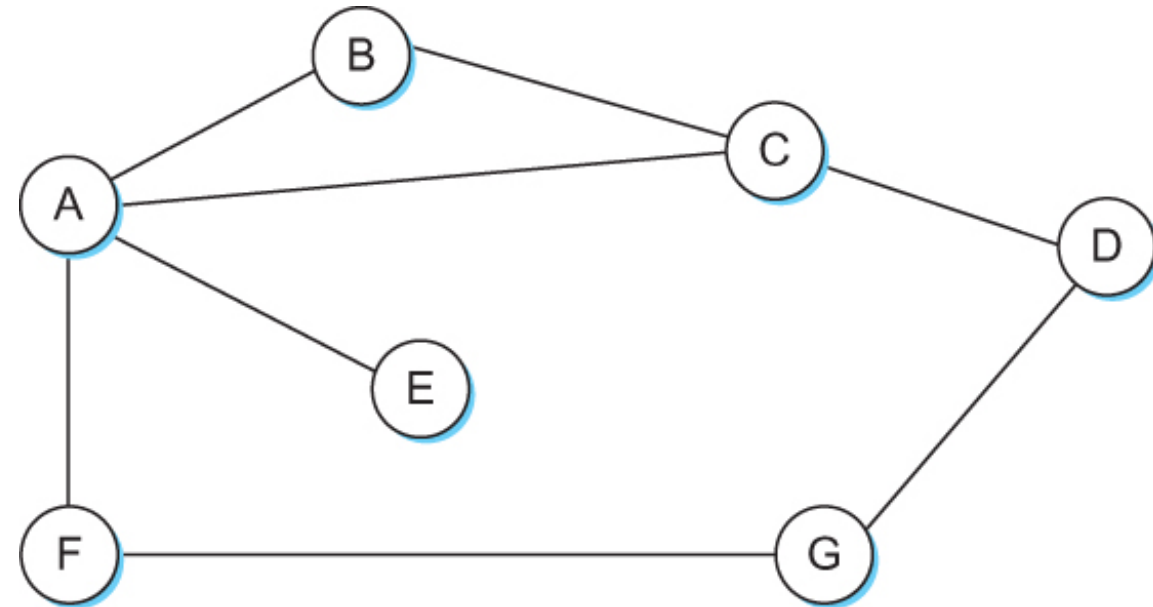


- Initial distances stored at each node (global view)

Distance Vector

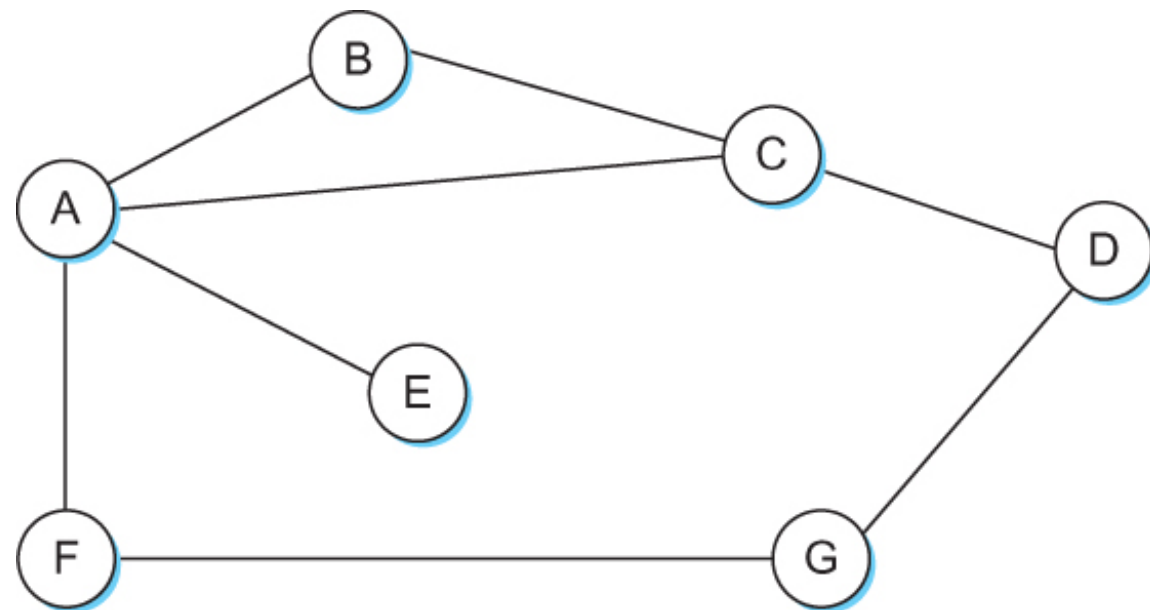
Destination	Cost	NextHop
B	1	B
C	1	C
D	∞	—
E	1	E
F	1	F
G	∞	—

Initial routing table at node A



Distance Vector

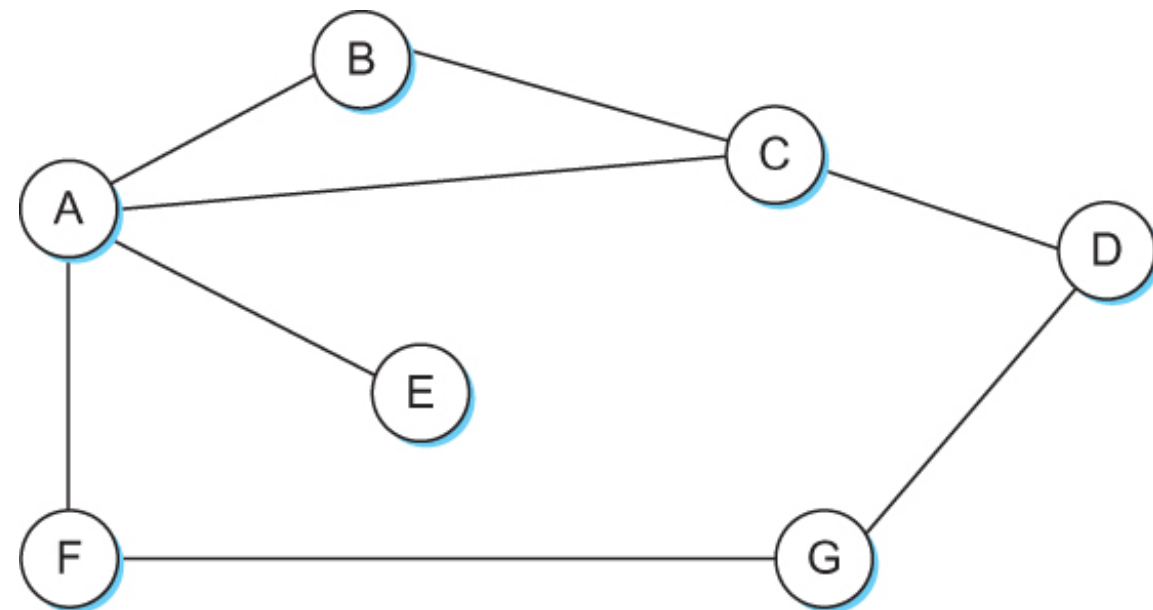
Destination	Cost	NextHop
B	1	B
C	1	C
D	2	C
E	1	E
F	1	F
G	2	F



- Final routing table at node A

Distance Vector

Information Stored at Node	Distance to Reach Node						
	A	B	C	D	E	F	G
A	0	1	1	2	1	1	2
B	1	0	1	2	2	2	3
C	1	1	0	1	2	2	2
D	2	2	1	0	3	2	1
E	1	2	2	3	0	2	3
F	1	2	2	2	2	0	1
G	2	3	2	1	3	1	0



- Final distances stored at each node (global view)

Distance Vector

- The distance vector routing algorithm is sometimes called as **Bellman-Ford** algorithm:
 - Every T seconds each router sends its vector to its neighbors
 - when a router receives a vector from a neighbor router, updates its table based on the new information: for each entry
 - if we have a better route than the one already known, the entry is replaced
 - if the best route is still through the neighbor router, the cost is updated
- (Stabilizes in $O(|N| * |L|)$), where N=nodes, L=links)

Distance Vector

- V = vector received by neighbor R
- T = my local routing table
- $\text{cost}(R)$ = cost of the link towards R
- Update Algorithm when we receive a vector V from R :

for each entry $(D, c, N) \in T$:

$c' = V(D) + \text{cost}(R)$

if $R = N$ then

 replace (D, c, N) in T with (D, c', N)

else if $c' < c$ then

 replace (D, c, N) in T with (D, c', R)

Distance Vector

- ❖ Problems include:
 - ❖ fast response to good new
 - ❖ slow response to bad news.
- ❖ Also too many messages to update

- When a node detects a link failure
 - F detects that link to G has failed
 - F sets distance to G to infinity and sends update to A
 - A sets distance to G to infinity since it uses F to reach G
 - A receives periodic update from C with 2-hop path to G
 - A sets distance to G to 3 and sends update to F
 - F decides it can reach G in 4 hops via A



Distance Vector

- Slightly different circumstances can prevent the network from stabilizing
 - Suppose the link from A to E goes down
 - In the next round of updates, A advertises a distance of infinity to E, but B and C advertise a distance of 2 to E
 - Depending on the exact timing of events, the following might happen
 - Node B, upon hearing that E can be reached in 2 hops from C, concludes that it can reach E in 3 hops and advertises this to A
 - Node A concludes that it can reach E in 4 hops and advertises this to C
 - Node C concludes that it can reach E in 5 hops
 - and so on... **count to infinity!**

Count-to-infinity Problem

- Use some relatively small number as an approximation of infinity
- For example, the maximum number of hops to get across a certain network is never going to be more than 16
 - Used in actual implementations (RIP)
- When the number reaches the limit, stop incrementing and treat it as ∞
- Does not work if the network outgrows the limit
 - hence good only for limited networks, like AS

Count-to-infinity Problem

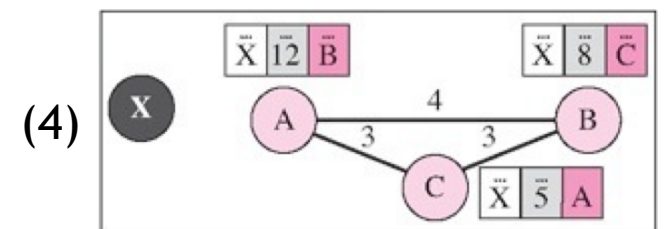
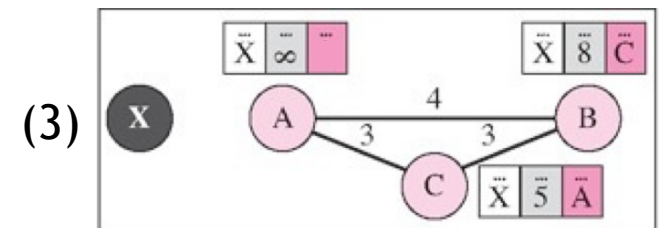
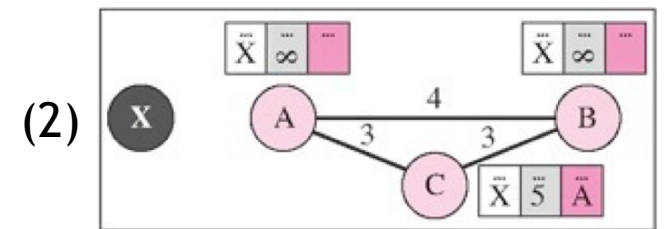
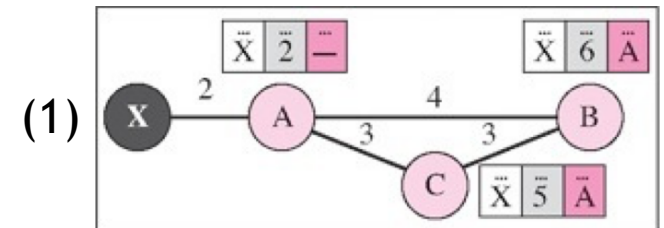
- One technique to improve the time to stabilize routing is called **split horizon**
 - When a node sends a routing update to its neighbors, it does not send those routes it learned from each neighbor back to that neighbor
 - For example, if B has the route (E, 2, A) in its table, then it knows it must have learned this route from A, and so whenever B sends a routing update to A, it does not include the route (E, 2) in that update

Count-to-infinity Problem

- Split horizon does not allow a node to know whether the neighbors receive its messages
- In a stronger version of split horizon, called *split horizon with poison reverse*
 - B actually sends that back route to A, but it puts negative information in the route to ensure that A will not eventually use B to get to E
 - For example, B sends the route (E, ∞) to A

Count-to-infinity Problem

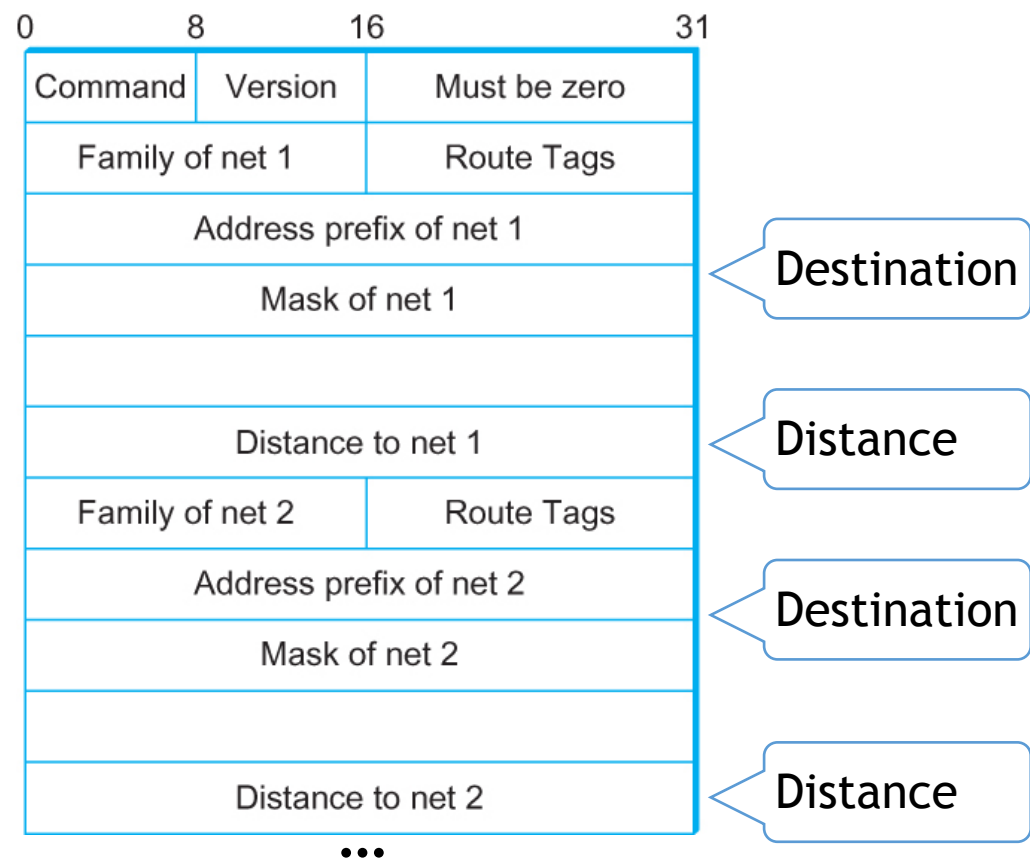
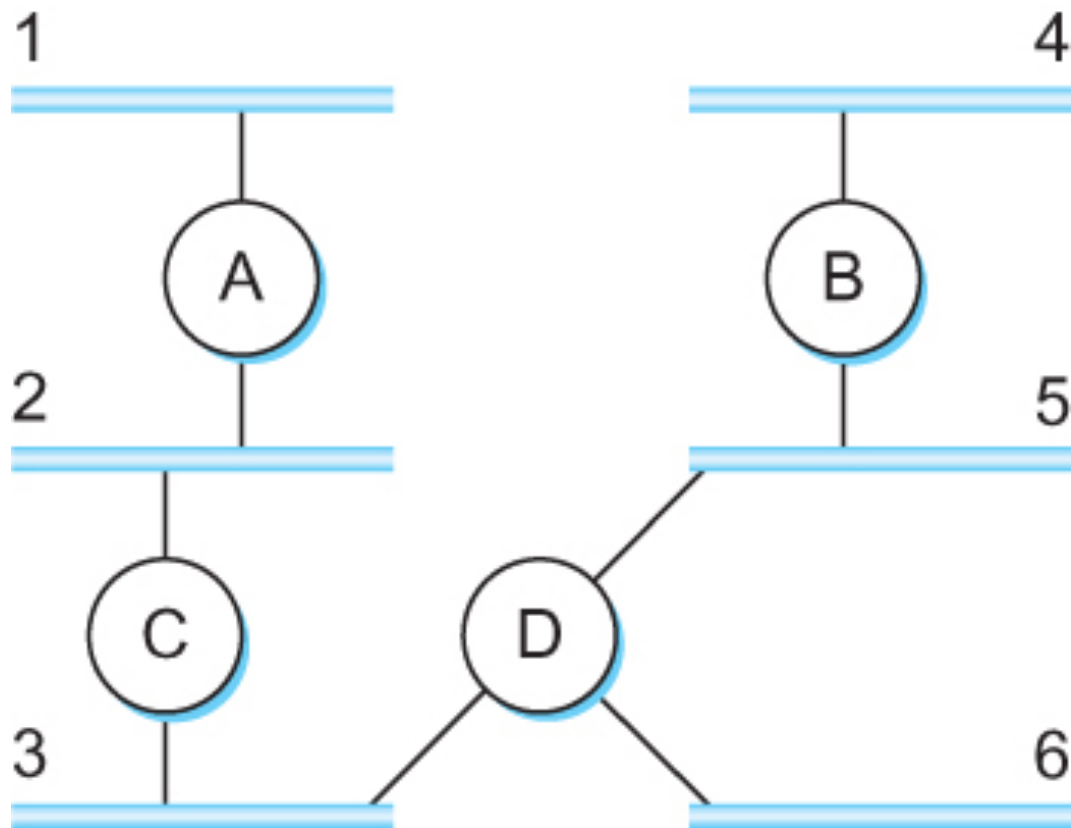
- Split horizon techniques does not solve the problem with loops of 3 or more nodes. E.g., in the situation aside:
 - Before failure
 - After A-X failure, and A has sent the route to B (but not to C)
 - After C sends the route to B
 - After B sends the route to A
- For this reason, usually “finite infinite” is used in actual implementations



Routing Information Protocol (RIP)

- RIP = Routing Information Protocol (currently v2)
 - Implemented as application level protocol, run over UDP (port 520), although it could be implemented directly over IP
- Used to be common solution for intradomain routing
 - Implemented on *nix systems by routed(8)
- Adopts distance vector technique with finite infinite
 - final destinations are network address
 - nodes are routers directly connected to networks
 - Next hop is the address of a router, or null if the destination is in the same network
 - Each hop counts 1
 - Limit value for infinity is 16 (hence it works on networks with diameter ≤ 15)

Routing Information Protocol (RIP)

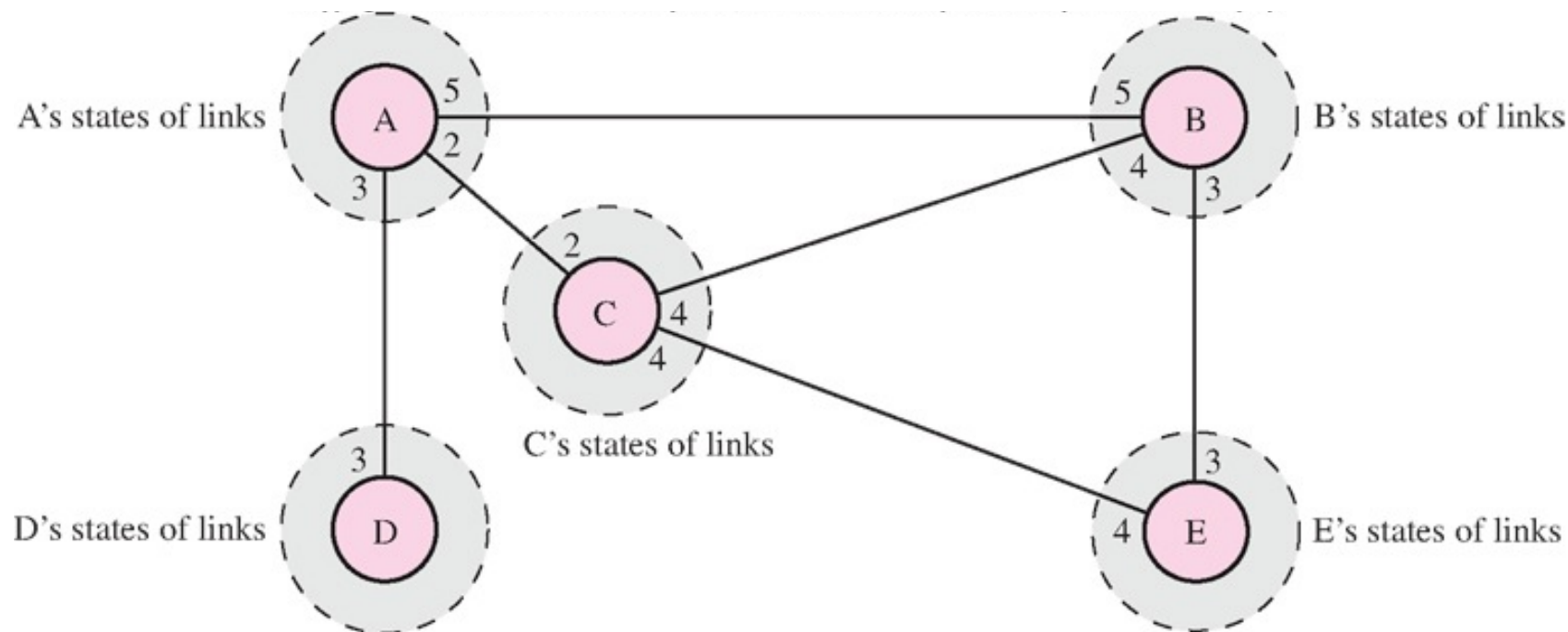


- Example Network running RIP

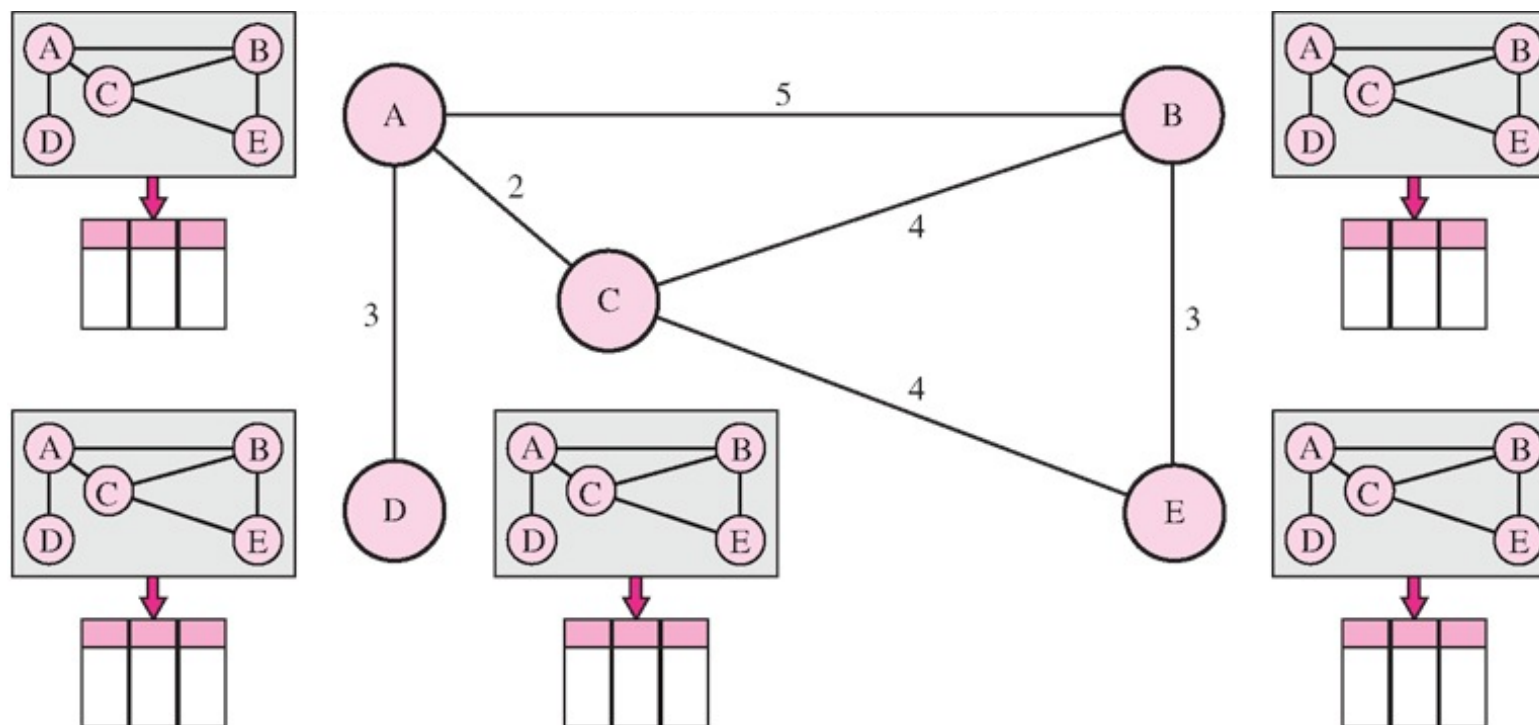
RIPv2 Packet Format

Link State Routing

- Completely different approach: each node constructs a **complete view** of the network (nodes and links)
 - Different nodes can have different views
- Each node checks the status of its direct connections, and informs the others of any change



- When a node has a complete view of the network it can compute the minimal paths for each node
 - E.g. using Dijkstra algorithm



Link State Routing

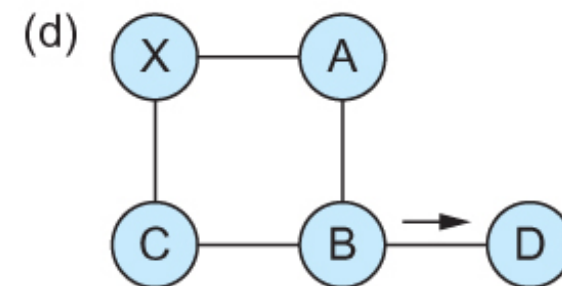
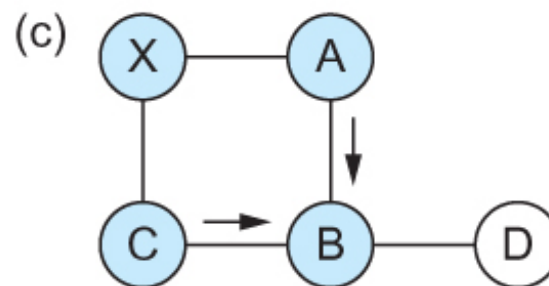
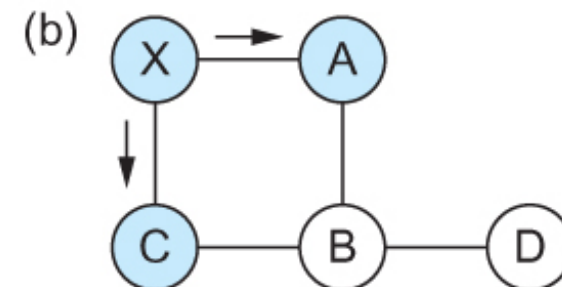
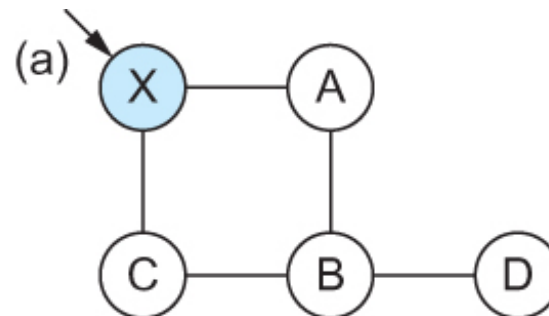
- Strategy: Inform periodically all nodes (not just neighbors) information about directly connected links (not entire routing table).
- **Link State Packet (LSP):** contains
 - id of the node that created the LSP
 - cost of link to each directly connected neighbor
 - sequence number (SEQNO)
 - time-to-live (TTL) for this packet
- Packets are propagated by *Reliable Flooding*
 - store most recent LSP from each node
 - forward LSP to all nodes but one that we have received the packet from
 - generate new LSP periodically; increment SEQNO
 - start SEQNO at 0 when reboot
 - decrement TTL of each stored LSP; discard when TTL=0

Link State

- **Reliable Flooding**

- Flooding of link-state packets

- (a) LSP arrives at node X;
- (b) X floods LSP to A and C;
- (c) A and C flood LSP to B (but not X);
- (d) flooding is complete



Shortest Path Routing

- Dijkstra's Algorithm - Assume non-negative link weights
 - N : set of nodes in the graph
 - $l(i, j)$: the non-negative cost associated with the edge between nodes $i, j \in N$ and $l(i, j) = \infty$ if no edge connects i and j
 - Let $s \in N$ be the starting node which executes the algorithm to find shortest paths to all other nodes in N
 - Two variables used by the algorithm
 - M : set of nodes incorporated so far by the algorithm
 - $C(n)$: the cost of the path from s to each node n

Shortest Path Routing

- Dijkstra's Algorithm

$M = \{s\}$

for each n in $N - \{s\}$

$C(n) = l(s, n)$

while ($N \neq M$)

$M = M \cup \{w\}$ such that $C(w)$ is the minimum for
all w in $(N-M)$

For each n in $(N-M)$

$C(n) = \text{MIN} (C(n), C(w) + l(w, n))$

- Complexity: $O(|I| + |N|\log(|N|))$

Shortest Path Routing

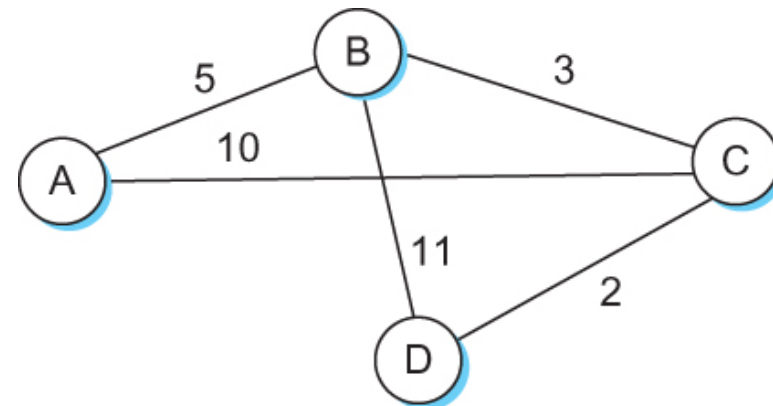
- In practice, each router computes its routing table directly from the LSP's it has collected using a realization of Dijkstra's algorithm called the **forward search algorithm**
- Specifically each router maintains two lists, known as Tentative and Confirmed
- Each of these lists contains a set of entries of the form (Destination, Cost, NextHop)

Shortest Path Routing - Algorithm

1. Initialize the Confirmed list with an entry for myself; this entry has a cost of 0
2. For the node just added to the Confirmed list in the previous step, call it node Next, select its LSP
3. For each neighbor (Neighbor) of Next, calculate the cost (Cost) to reach this Neighbor as the sum of the cost from myself to Next and from Next to Neighbor
 - If Neighbor is currently on neither the Confirmed nor the Tentative list, then add (Neighbor, Cost, Nexthop) to the Tentative list, where Nexthop is the direction I go to reach Next
 - If Neighbor is currently on the Tentative list, and the Cost is less than the currently listed cost for the Neighbor, then replace the current entry with (Neighbor, Cost, Nexthop) where Nexthop is the direction I go to reach Next
4. If the Tentative list is empty, stop. Otherwise, pick the entry from the Tentative list with the lowest cost, move it to the Confirmed list, and return to Step 2.

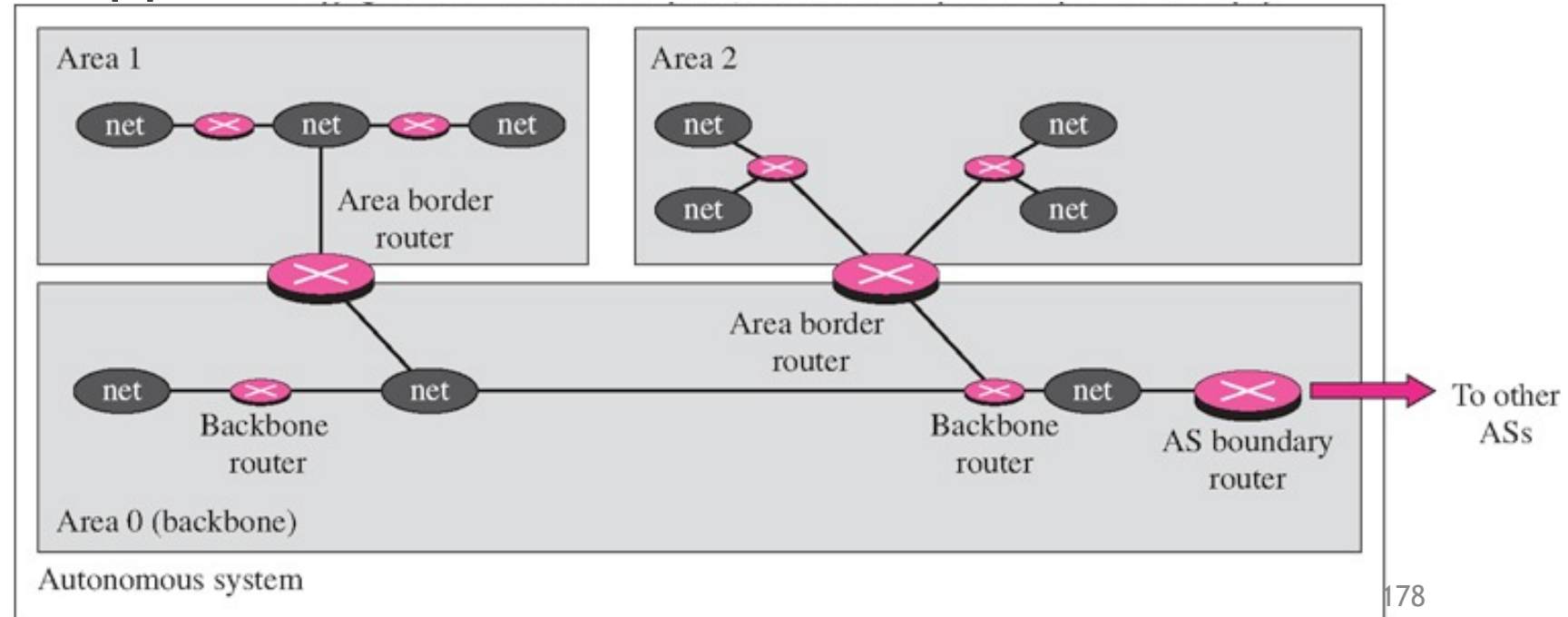
Shortest Path Routing

Step	Confirmed	Tentative	Comments
1	(D,0,-)		Since D is the only new member of the confirmed list, look at its LSP.
2	(D,0,-)	(B,11,B) (C,2,C)	D's LSP says we can reach B through B at cost 11, which is better than anything else on either list, so put it on Tentative list; same for C.
3	(D,0,-) (C,2,C)	(B,11,B)	Put lowest-cost member of Tentative (C) onto Confirmed list. Next, examine LSP of newly confirmed member (C).
4	(D,0,-) (C,2,C)	(B,5,C) (A,12,C)	Cost to reach B through C is 5, so replace (B,11,B). C's LSP tells us that we can reach A at cost 12.
5	(D,0,-) (C,2,C) (B,5,C)	(A,12,C)	Move lowest-cost member of Tentative (B) to Confirmed, then look at its LSP.
6	(D,0,-) (C,2,C) (B,5,C)	(A,10,C)	Since we can reach A at cost 5 through B, replace the Tentative entry.
7	(D,0,-) (C,2,C) (B,5,C) (A,10,C)		Move lowest-cost member of Tentative (A) to Confirmed, and we are all done.



Open Shortest Path First (OSPF)

- Protocol Open Shortest Path First
 - implemented on *nix by gated(8)
 - directly over IP (no UDP), with authentication to avoid redirection attacks: a host can redirect traffic by flooding false information
- adopts the Link State approach
- The AS is divided in *areas* and *backbone*
- each area is connected to backbone or other areas by a **border router**



Open Shortest Path First (OSPF)

- Administrator can define link costs in various ways
 - speed, cost, load charge, latency, reliability, ...
- OSPF allows LSP to carry different kind of costs, even for the same links
- A router can keep several routing tables (and correspondingly several forwarding tables), one for each intended kind of service: it can make different forwarding choices for different services
- which table to apply is decided upon
 - TOS/DSCP (rare)
 - QoS, priority (if supported)
 - flow tag and priority in IPv6
 - Automatic flow kind detection (*traffic shaping*)

Open Shortest Path First (OSPF)

- OSPF Header Format

0	8	16	31
Version	Type	Message length	
SourceAddr			
Areald			
Checksum		Authentication type	
Authentication			

OSPF Link State Advertisement

LS Age		Options		Type = 1
Link-state ID				
Advertising router				
LS sequence number				
LS checksum			Length	
0	Flags	0	Number of links	
Link ID				
Link data				
Link type	Num_TOS		Metric	
Optional TOS information				
More links				

Summary

- We have looked at some of the issues involved in building scalable and heterogeneous networks by using switches and routers to interconnect links and networks.
- To deal with heterogeneous networks, we have discussed in details the service model of Internetworking Protocol (IP) which forms the basis of today's routers.
- We have seen some ancillary protocols (ICMP, NAT, DHCP)
- We have discussed in details two major classes of routing algorithms
 - Distance Vector (RIPv2)
 - Link State (OSPF)